UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA AERONÁUTICA Y DEL ESPACIO

GRADO EN INGENIERÍA AEROESPACIAL

TRABAJO FIN DE GRADO

Comparative analysis of computational performance in simulation with parallel processing techniques

AUTOR: Pedro RODRÍGUEZ JIMÉNEZ

ESPECIALIDAD: Ciencias y Tecnologías Aeroespaciales (CTA)

COTUTOR: Javier Luis GONZÁLEZ MONGE

TUTOR DEL TRABAJO: Juan Antonio HERNÁNDEZ RAMOS

Julio de 2025

I dedicate this work to my family and friends, for all the support they have given me throughout these years of my bachelor's degree at ETSIAE, as well as for their constant encouragement to carry out and complete this bachelor's thesis. This achievement is not solely the result of the past year, but rather the culmination of a journey that began 5 years ago when I first set foot in the school back in 2020.

This journey has profoundly shaped who I am today, not only by strengthening my knowledge but also by enriching me on a personal level. These years at the school have given me the opportunity to grow, to meet amazing people, to work on exciting personal projects, and to gain international experience during my exchange semester at Tongji University in Shanghai. All these experiences together have broadened my perspective and opened new horizons.

I would also like to express my gratitude to my tutor, Juan Antonio Hernández Ramos, and to Javier González Monge for their invaluable support throughout the entire process. Even though the circumstances were not always ideal in terms of proximity, due to my stay in China, their guidance and weekly assistance have been essential for making progress, step by step.

Completing this stage means closing a chapter in my life that has been present for a long time, and it brings me great satisfaction to finally conclude it so that I can move forward.

Abstract

This thesis presents a comprehensive computational performance analysis comparing matrix-vector (MxV) and matrix-matrix (MxM) operations across CPU and GPU architectures, with practical applications to numerical wave equation solvers. The study is motivated by the fundamental limitation that matrix-vector operations suffer from low operational intensity, leading to memory-bound performance, while matrix-matrix operations achieve higher arithmetic intensity and better computational efficiency.

The research is structured in two complementary blocks. Block I conducts systematic synthetic performance benchmarks on both CPU and GPU architectures. Through detailed GFLOPS measurements across varying problem sizes, the study demonstrates that MxM operations consistently outperform MxV operations due to superior operation-to-data ratios. On CPU, MxM operations achieve compute-bound behavior with effective cache utilization, while MxV operations remain memory-bound with limited scaling. GPU results show similar trends but with different performance characteristics due to architectural differences in memory hierarchy and parallel processing capabilities.

Block II validates these findings through real-world applications using wave equation solvers implemented with global interpolation methods. The study develops 1D and 2D wave equation solvers using Lagrange polynomials and differentiation matrices, comparing three computational approaches: (1) 1D solver using matrix-vector formulation, (2) 2D solver using efficient matrix-matrix formulation, and (3) 2D solver using looped matrix-vector operations. Performance measurements confirm that the matrix-matrix approach achieves significantly higher computational efficiency, with the 2D matrix-matrix implementation reaching compute-bound performance while matrix-vector alternatives remain memory-bound.

Key contributions include: (1) comprehensive benchmarking methodology for comparing MxV vs MxM operations across architectures, (2) theoretical and empirical analysis of operational intensity impacts on performance, (3) demonstration of cache hierarchy effects and memory bandwidth limitations, (4) development of high-order accurate wave equation solvers using global interpolation, and (5) validation that synthetic benchmark insights translate directly to practical PDE applications.

The results provide clear evidence that matrix-matrix formulations should be preferred over matrix-vector approaches in numerical algorithms whenever mathematically feasible, offering substantial performance improvements across both CPU and GPU platforms. This work establishes computational guidelines for numerical method design and demonstrates the critical importance of algorithmic structure in achieving optimal performance on modern computing architectures.

Table of Contents

Chapter 1

Introduction

1.1 Objectives and Scope of the Study

This bachelor thesis conducts a comprehensive analysis of computational performance in numerical computing, with particular emphasis on understanding the Operation to Data Ratio and its implications for matrix-vector versus matrix-matrix formulations. The study is structured around two fundamental blocks.

Block I: Synthetic Performance Analysis

- **CPU Performance Analysis:** Detailed study of single-core and multi-core CPU performance characteristics, comparing matrix-vector and matrix-matrix operations. Analysis includes memory bandwidth limitations and cache hierarchies.
- **GPU Performance Analysis:** Parallel examination of GPU architectures, exploring the same matrix operation comparisons with focus on memory coalescing, thread occupancy, and throughput-oriented design advantages.
- **CPU vs GPU Comparative Study:** Direct performance comparison between architectures, identifying scenarios where each excels and understanding the fundamental architectural differences that drive performance characteristics.

Block II: Real-World Application

- **Wave Equation Solver:** Implementation of 1D and 2D wave equation solvers as representative computational physics problems requiring intensive numerical operations.
- **Global Interpolation Formulation:** Development of interpolation matrices that can operate on both vector and matrix data structures, providing a direct testbed for comparing operation types as in synthetic benchmarks.
- **Performance Validation:** Verification that real-world applications exhibit the same performance characteristics observed in synthetic benchmarks.

This dual-block approach ensures that performance insights gained from controlled synthetic benchmarks translate meaningfully to practical computational scenarios.

1.2 How to Navigate this Bachelor Thesis

This bachelor thesis is designed as an interactive computational document that combines traditional academic exposition with hands-on experimentation. Rather than merely presenting results, this work enables readers to reproduce, verify, and extend every computational experiment on their own hardware.

Repository Structure and Philosophy

The accompanying repository mirrors the structure of this document, with each chapter and section corresponding to executable code modules. This design philosophy ensures that:

- Every benchmark can be reproduced on the reader's local machine
- Architecture-specific differences can be explored and compared
- Performance claims are immediately verifiable rather than taken on faith
- Code implementations are transparent and available for modification

Interactive Exploration

The repository includes a launcher script (`main.jl`) that provides an interactive menu system mirroring this thesis structure. Readers can:

1. Navigate through synthetic performance benchmarks (Chapters 2-5)
2. Execute wave equation solvers with different formulations (Chapters 6-8)
3. Compare their hardware performance against the baseline results presented here
4. Modify parameters to explore performance boundaries on their specific systems

Getting Started

To begin exploring:

1. Install Julia following the setup guide in Appendix A.1
2. Clone the bachelor thesis repository from the provided link here
3. Execute `main.jl` from the root directory
4. Use the interactive menu to navigate and execute any section of interest

This approach transforms the traditional thesis reading experience into an active computational investigation, where theoretical concepts are immediately testable and performance claims are subject to empirical verification on the reader's own hardware.

Chapter 2

Fundamentals of Computational Performance

2.1 Operation to data ratio: Operational Intensity

As already said, there is an inherent limitation in matrix-vector multiplication (which is not due to CPU capacity but rather a data access bottleneck issue). This limitation arises because the operation-to-data ratio in matrix-matrix operation is higher than in a matrix-vector operation.

This hypothesis emerged during a collaboration grant where the GPU-Parallel Repository was developed and made available on GitHub [10].

In matrix-vector multiplication, the number of computations that can be performed per data element fetched from memory is low, meaning the processor often waits for data to arrive rather than performing calculations. This is known as being *memory-bound*.

In contrast, matrix-matrix multiplication has a much higher operation-to-data ratio, allowing the processor to perform more computations per data element loaded, thus better utilizing computational resources and reducing the impact of memory bandwidth limitations.

The operation-to-data ratio is a measure of how many computations can be performed relative to the amount of data that needs to be accessed from memory. This is also often referred to as **operational intensity**. A higher operation-to-data ratio indicates that more computations are being performed for each data element accessed, which is generally more efficient and leads to better performance in computational tasks.

This chapter will analyze and compare these ratios in the context of matrix-vector and matrix-matrix operations, demonstrating the advantages of matrix-matrix computations in exploiting parallelism effectively.

2.1.1 Matrix-Vector Multiplication

$$\begin{bmatrix} a_{11} & \dots & a_{1N} \\ \vdots & \ddots & \vdots \\ a_{N1} & \dots & a_{NN} \end{bmatrix} \begin{bmatrix} b_1 \\ \vdots \\ b_N \end{bmatrix} = \begin{bmatrix} c_1 \\ \vdots \\ c_N \end{bmatrix} \tag{2.1}$$

In matrix-vector multiplication, each element of the resulting vector c_i is computed as:

$$c_i = \sum_{j=1}^{N} a_{ij} \cdot b_j \tag{2.2}$$

This requires N multiplications and $N-1$ additions, resulting in $2N-1$ operations per element of the resulting vector.

Since the resulting vector has N elements, the total number of operations becomes:

$$N_{ops} = N \cdot (2N-1) \approx 2N^2 \quad \text{(for large } N\text{)} \tag{2.3}$$

The data involved are: the matrix A with N^2 elements, the input vector b with N elements, and the output vector c that must be written (N elements). Therefore:

$$N_{\text{data}} = N^2 + 2N. \tag{2.4}$$

2.1.2 Matrix-Matrix Multiplication

$$\begin{bmatrix} d_{11} & \dots & d_{1N} \\ \vdots & \ddots & \vdots \\ d_{N1} & \dots & d_{NN} \end{bmatrix} \begin{bmatrix} e_{11} & \dots & e_{1N} \\ \vdots & \ddots & \vdots \\ e_{N1} & \dots & e_{NN} \end{bmatrix} = \begin{bmatrix} f_{11} & \dots & f_{1N} \\ \vdots & \ddots & \vdots \\ f_{N1} & \dots & f_{NN} \end{bmatrix} \tag{2.5}$$

In matrix-matrix multiplication, each element of the resulting matrix c_{ij} is computed as:

$$f_{ij} = \sum_{k=1}^{N} d_{ik} \cdot e_{kj} \tag{2.6}$$

This requires N multiplications and $N-1$ additions, resulting in $2N-1$ operations per element of the resulting matrix.

Since the resulting matrix has N^2 elements, the total number of operations becomes:

$$N_{ops} = N^2 \cdot (2N-1) \approx 2N^3 \quad \text{(for large } N\text{)} \tag{2.7}$$

With the same *ideal* assumption on data movement, the matrix inputs D and E must be read and the matrix output F must be written:

$$N_{\text{data}} = N^2 + N^2 + N^2 = 3N^2. \tag{2.8}$$

2.1.3 Comparison of Operation to Data Ratio

The comparison shows that matrix–matrix multiplication achieves a much higher operation-to-data ratio than matrix–vector multiplication. Under ideal reuse,

$$\text{MxV:} \quad \frac{N_{\text{ops}}}{N_{\text{data}}} = \frac{2N^2}{N^2 + 2N} \xrightarrow[N\to\infty]{} 2, \qquad \text{MxM:} \quad \frac{N_{\text{ops}}}{N_{\text{data}}} = \frac{2N^3}{3N^2} = \tfrac{2}{3}N.$$

Thus MxM's intensity grows linearly with N, whereas MxV's intensity saturates near 2.

	Matrix–Vector (MxV)	**Matrix–Matrix (MxM)**
Total Operations (N_{ops})	$2N^2$	$2N^3$
Total Data Elements (ideal N_{data})	$N^2 + 2N$	$3N^2$
Operation-to-Data Ratio	**2**	$\boldsymbol{\frac{2}{3}N}$

Table 2.1: Operation-to-data ratio under ideal reuse.

The operation-to-data ratio, or operational intensity, has profound implications for the computational performance of matrix-vector (MxV) and matrix-matrix (MxM) multiplications in real-world scenarios. This ratio directly influences whether a computation is *memory-bound* or *compute-bound*, which in turn affects overall efficiency, scalability, and hardware utilization.

In MxV multiplication, the low operational intensity (approaching 2 as N increases) means that the algorithm is typically memory-bound. The processor spends a significant portion of time waiting for data to be fetched from memory rather than performing arithmetic operations. This bottleneck is exacerbated in systems with limited memory bandwidth, leading to underutilization of computational cores. For large N, performance scales poorly with additional compute resources because the limiting factor is data transfer rates, not floating-point operations per second (FLOPS). In practice, this results in MxV achieving only a fraction of the peak theoretical performance on modern hardware.

On the other hand, MxM multiplication benefits from a high operational intensity that scales linearly with N ($\frac{2}{3}N$). This allows for better data reuse within caches or registers, reducing memory accesses and enabling the computation to become compute-bound. With sufficient problem size, MxM can fully exploit the arithmetic capabilities of the processor, leading to higher throughput and efficiency. Optimized libraries like BLAS (Basic Linear Algebra Subprograms) leverage this by implementing blocked algorithms that maximize cache locality, further amplifying the performance advantages.

In general, achieving a high operation-to-data ratio is crucial for efficiency because it minimizes time spent on data movement and maximizes computational throughput [5].

In the following sections, we will explore these impacts in greater detail across different hardware architectures.

Chapter 3

CPU Architecture and Performance

3.1 How a CPU Works

The **Central Processing Unit (CPU)** executes instructions following a fundamental cycle: *fetch, decode, and execute*. In the **fetch** stage, the CPU retrieves instructions and data from memory (preferably from its internal cache, a high-speed intermediary between the CPU cores and the main RAM). In the **decode** stage, these instructions are translated into low-level micro-operations, and in the **execute** stage, they are processed by the CPU's functional units using **registers**, extremely fast and small memory units located within each core.

Modern CPUs are composed of multiple **physical cores**, each capable of running one or more **threads** through *Simultaneous Multithreading* (SMT). In performance modelling, the number of physical cores will be denoted as C_{CPU}. While threads appear independent, they share the same execution resources inside a core, which means that doubling the number of threads does not double performance.

CPU performance depends not only on clock speed and core count but also on how efficiently data moves through the memory hierarchy (from registers, through L1/L2/L3 cache, down to RAM). The CPU clock speed, CPU_{GHz}, determines how many cycles per second each core can execute. Ignoring memory access patterns can cause severe performance drops, especially for workloads that cannot reuse data efficiently in cache.

3.1.1 Instruction-Level Parallelism: Micro-ops and Vectorization

High-level instructions are broken down into simpler internal steps called **micro-operations**, which are scheduled to different execution units. At a high level, a matrix multiplication involves N^3 scalar multiplications and additions. However, the CPU cannot handle such a high-level operation in one go. Instead, the compiler (or even a specialized library like BLAS) decomposes it into nested loops with scalar operations. This specialized library does it better than a naive implementation [15].

Modern CPUs can also process multiple data elements per instruction using **vectorization**, which follows the SIMD (*Single Instruction, Multiple Data*) paradigm. Here, one instruction applies the same operation to an entire block of data (a **vector register**) instead of just a single value.

The number of elements that can be processed in parallel is referred to as the **vector width in elements**, V_{vec}. For example:

- AVX–512: 512-bit registers, $V_{vec} = 16$ for single precision (32-bit) or $V_{vec} = 8$ for double precision (64-bit).
- AVX2: 256-bit registers, $V_{vec} = 8$ for single precision or $V_{vec} = 4$ for double precision.

Within these vector instructions, **Fused Multiply–Add (FMA)** operations are particularly important for numerical workloads. An FMA combines a multiplication and an addition into a single instruction:

$$r = (a \times b) + c$$

This counts as **two floating–point operations** (one multiplication and one addition) in one CPU cycle.

3.1.2 Memory Hierarchy: Registers, Cache, RAM

At the core of the CPU lies the fastest and smallest form of memory: the **registers**. These are tiny storage units, typically ranging from 32 to 256 in number, designed to hold immediate values such as operands or addresses during the execution of instructions. Registers operate at the same speed as the CPU clock, allowing near-instantaneous data access.

Due to this limitation, CPUs also rely on **cache memory**, a faster intermediary between the registers and main memory (RAM). Cache memory temporarily stores frequently accessed data and instructions to reduce the time spent retrieving information from slower memory levels. This dramatically improves performance by minimizing idle CPU cycles.

Modern processors implement a multi-level cache hierarchy [3], typically consisting of L1, L2, and L3 caches, each with different sizes and speeds.

The following figure shows the typical hierarchy of memory, organized as a pyramid:

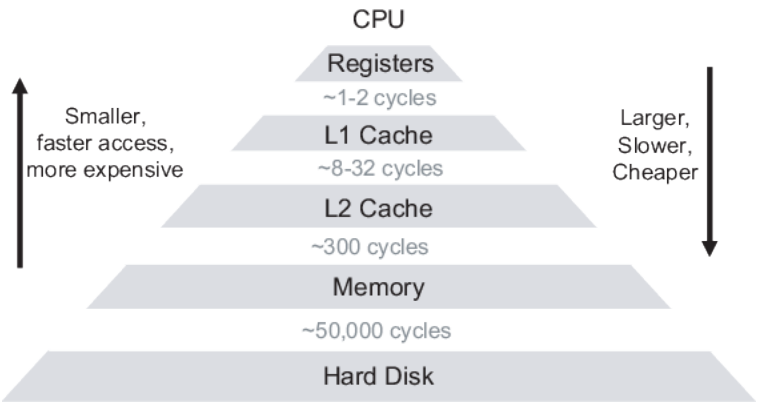

Figure 3.1: Hierarchy of memory types based on speed, cost, and capacity.

Cache Memory Operation

Cache memory operates on the principle of **locality**, which refers to the tendency of programs to access data and instructions that are close to each other in memory (spatial locality) or accessed repeatedly within a short time frame (temporal locality). The cache stores a subset of the main memory's contents in small, fast memory blocks called **cache lines**, typically 64 bytes in size. When the CPU requests data, the cache controller checks if the data resides in the cache. If it does (a **cache hit**), the data is retrieved quickly without accessing RAM. If it does not (a **cache miss**), the controller fetches the data from RAM or a lower-level cache, loading it into the cache for future access.

The cache is organized into **sets** and **ways**, forming an associative structure. For example, in a 4-way set-associative cache, each set can store four cache lines, and the cache controller uses a mapping function to determine which set a memory address corresponds to. If the target set is already full, the controller must evict a line (typically the least recently used) and repeated conflicts that keep evicting lines from the same set are known as **cache thrashing** [17].

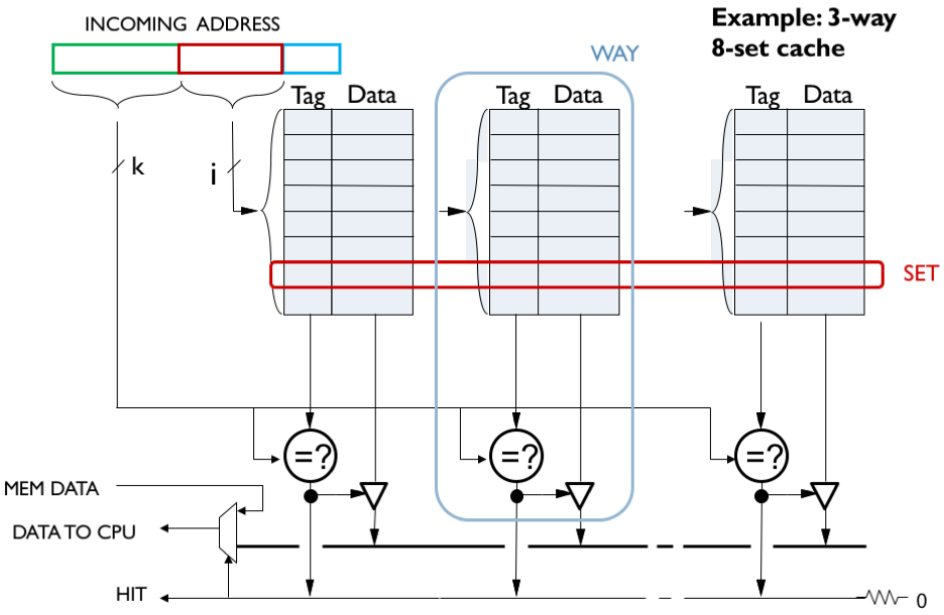

Figure 3.2: Set-associative cache (sets and ways). Memory is cached in fixed-size lines (typically 64 B). When many cache addresses map to the same set and exceed the set's capacity, thrashing occurs. This organization explains why strided access patterns (e.g., row/column scans in matrices) can perform very differently depending on stride and problem size.

For now, we just need to understand the basic concepts of cache organization, including the roles of sets and ways, as well as the implications of cache hits and misses. But as we can see in the figure, this can be quite complex. Later sections will illustrate these concepts with practical examples.

RAM (Random Access Memory)

RAM serves as the primary workspace for the CPU, storing data and program instructions that are actively in use. Unlike registers and cache, RAM offers significantly larger capacity (typically from 8GB to 64GB) but at the cost of much slower access speeds.

In an ideal scenario, the CPU retrieves the data it needs directly from its registers or from the cache, which are both extremely fast. However, if the required data is not present in any level of the cache hierarchy, a **cache miss** occurs. In this case, the CPU is forced to fetch the data from RAM, introducing a considerable delay in execution due to the higher memory latency.

This delay becomes particularly problematic in operations with low operation-to-data ratios, such as matrix-vector multiplication. As discussed in Chapter 1, Section2.1.3, MxV operations require frequent access to large portions of memory for relatively few computations, making them highly prone to cache misses. On the other hand, matrix-matrix multiplication (MxM) has a much higher operation-to-data ratio, allowing for better data reuse and more efficient use of the cache.

The performance of RAM is commonly described by two metrics: its **frequency** (in MHz), which affects memory bandwidth, and its **CAS latency** [6] (Column Access Strobe, or CL), which impacts memory access delay. CAS latency represents the number of clock cycles that elapse between the moment a memory controller sends a read command to a RAM module and when the data becomes available. However, since frequency determines how long each clock cycle lasts, the effective latency in nanoseconds is given by:

$$\text{Latency (ns)} = \frac{\text{CL} \times 1000}{\text{Frequency (MHz)}}$$

Hence, a higher frequency does not necessarily imply faster access unless CL is also considered. For instance, CL16 at 3200MHz results in a similar real latency to CL19 at 4000MHz:

Memory Speed	CAS Latency (CL)	Approx. Latency (ns)
3200 MHz	CL16	$\frac{16\times1000}{3200} \approx 5$ ns
4000 MHz	CL19	$\frac{19\times1000}{4000} \approx 4.75$ ns
2400 MHz	CL17	$\frac{17\times1000}{2400} \approx 7.08$ ns

Table 3.1: Estimated memory latency for different RAM configurations.

Even newer memory technologies such as DDR5 aim to increase bandwidth and reduce latency through architectural improvements, such as dual or quad independent memory channels and higher burst lengths. However, the fundamental limitation of main memory remains: it is still significantly slower than L1, L2, or L3 cache, and orders of magnitude slower than registers.

3.1.3 Cache Locality and Blocking

Cache locality refers to the efficient use of data elements that are either close together in memory (**spatial locality**) or accessed repeatedly in a short time frame (**temporal locality**). This aligns with how caches store data in 64-byte cache lines to minimize slower main memory access.

Blocking or tiling improves cache locality by dividing computations into smaller blocks that fit within cache, reducing cache misses and optimizing data reuse, especially beneficial for matrix multiplication. [20]

In **Naive matrix multiplication**, each element $C(i, j)$ accesses entire row $A(i, *)$ and column $B(*, j)$. For large N, strided accesses (jumps of $N \times$ element size bytes) cause cache thrashing as data doesn't fit in cache and repeated accesses evict previously loaded cache lines.

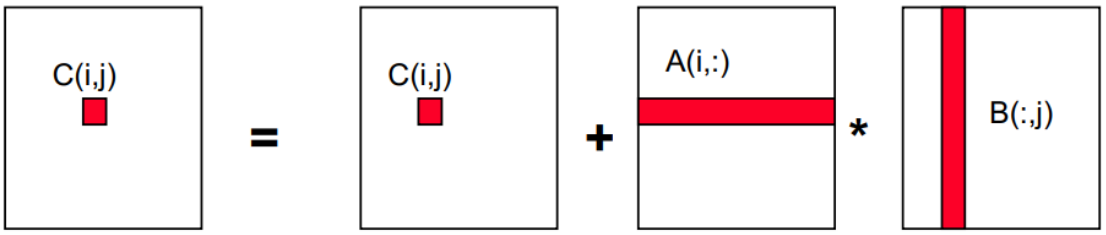

Figure 3.3: Naive matrix multiplication algorithm.

Blocked multiplication partitions A, B, and C into $b \times b$ blocks where b fits in cache. For a 32KB L1 cache with FP32 elements, since three blocks must reside simultaneously:

$$3b^2 \leq 8192 \quad \Rightarrow \quad b \leq \sqrt{\tfrac{8192}{3}} \approx 52$$

The algorithm uses outer loops over blocks and inner loops for standard multiplication on each block, ensuring all necessary data resides in cache and leveraging both spatial and temporal locality.

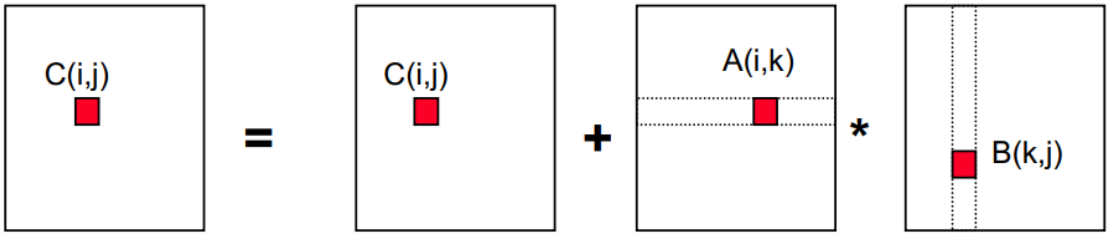

Figure 3.4: Blocked matrix multiplication algorithm.

As mentioned in Chapter 2, blocking enables aggressive data reuse: tiles are reused $O(b)$ times, raising arithmetic intensity by factor b. Good blocking approaches the ideal $\frac{2}{3}N$ intensity, while naive approaches collapse to $O(1)$ due to repeated reloading. Libraries like BLAS and MKL implement these optimizations automatically.

3.2 Benchmarking on CPU

In this section, we benchmark and compare the performance of **matrix–matrix multiplication** (MxM) and **matrix–vector multiplication** (MxV). The aim is to measure, for each operation, the sustained performance in $GFLOPS$ as the matrix size N increases, and to contrast these measurements with theoretical limits imposed by both computation and memory access.

Theoretical Compute Performance

The theoretical time required to execute a given number of floating–point operations, assuming the CPU operates at its peak throughput, can be estimated as:

$$t_{CPU} = \frac{N_{ops}}{GFLOPS \times 10^9} \times 10^6, \tag{3.1}$$

where t_{CPU} is in microseconds.

The peak performance in $GFLOPS$ is computed as:

$$GFLOPS = C_{CPU} \times CPU_{GHz} \times V_{vec} \times M_{ops}. \tag{3.2}$$

The terms in the equation are:

- C_{CPU}: number of **physical** CPU cores (logical threads are not counted here).
- CPU_{GHz}: CPU clock speed in GHz.
- V_{vec}: SIMD vector width in elements, which depends on the data type (e.g., single-precision 32-bit floats or double-precision 64-bit floats) and the instruction set architecture. The vector width V_{vec} represents the number of elements that can be processed simultaneously in a single SIMD instruction. For example:
  - Using **single-precision (F32, 32-bit)**:
    - AVX-512: $V_{vec} = 512/32 = 16$ elements per vector.
    - AVX2 (256-bit): $V_{vec} = 256/32 = 8$ elements per vector.
  - Using **double-precision (F64, 64-bit)**:
    - AVX-512: $V_{vec} = 512/64 = 8$ elements per vector.
    - AVX2 (256-bit): $V_{vec} = 256/64 = 4$ elements per vector.
- M_{ops}: floating–point operations per cycle per vector unit. For Fused Multiply–Add (FMA) [8] instructions, each operation performs one multiplication and one addition in a single step, counting as two floating–point operations. For example, AVX2 [9] on the AMD Ryzen 5 5600X (Zen 3) can execute 2 FMAs per cycle, resulting in $M_{ops} = 2 \times 2 = 4$.

Theoretical Memory Access Performance

Even if the compute units can, in theory, reach a certain $GFLOPS$ peak, performance may be limited by memory bandwidth if it relies on it. This limitation arises because the rate at which data can be transferred to and from RAM often becomes a bottleneck, especially for memory-intensive operations such as matrix multiplications or large-scale vector computations. To estimate the time required to read or write the necessary data from/to RAM, we use a simplified model:

$$t_{mem} = \frac{N_{data}}{RAM_{Hz}} \times 10^6, \tag{3.3}$$

where the terms are defined as follows:

- N_{data}: the number of data elements that must be loaded from or stored into memory. This value depends on the specific operation being performed, such as the total number of elements in a matrix for matrix-matrix multiplication ($2 \cdot N^2$ for reading and writing) or the combined elements of a matrix and vector for matrix-vector multiplication ($N^2 + N$).
- RAM_{Hz}: the effective memory frequency in Hertz, which represents the clock rate at which the RAM operates. This frequency determines the maximum theoretical data transfer rate, assuming ideal conditions without overheads or inefficiencies.

This simplified model assumes that each clock cycle of the RAM can transfer one data element, and the time is converted to microseconds by multiplying by 10^6.

However, it is important to note that this is a theoretical estimate and does not account for factors such as memory controller latency, bus contention, or the caches. For a more accurate estimation, one might consider the effective bandwidth and data size per element, but this model provides a baseline for understanding memory access costs.

For matrix-matrix multiplication (MxM), we estimate the real total memory access time by summing the individual measured components: the time to access a matrix by rows, the time to access a matrix by columns, and the time to allocate a matrix.

For matrix-vector multiplication (MxV), we sum the time to access a matrix by rows, the time to access a vector, and the time to allocate a vector.

3.2.1 Benchmark Methodology

The **GFLOPS benchmarks** are run using the `GFLOPS.jl` script where we do the following:

1. Allocate:
   - A: random $N \times N$ with `Float32` or `Float64` matrix.
   - B_{mat}: random $N \times N$ with `Float32` or `Float64` matrix (for MxM).
   - B_{vec}: random $N \times 1$ with `Float32` or `Float64` vector (for MxV).
2. Perform a **single multiplication** using `@benchmark` (`evals=1, samples=300`).
3. Use the **minimum measured time** to approximate the sustained peak performance, reducing variability from noise and cache effects.
4. Compute the sustained performance:

$$GFLOPS_{\text{MxM}} = \frac{2N^3}{t_{MxM}} \times 10^{-9}, \quad GFLOPS_{\text{MxV}} = \frac{2N^2}{t_{MxV}} \times 10^{-9}.$$

The **theoreticals memory access time t_{mem} and compute time t_{CPU} benchmarks** are run using the `tcpu_tmemory.jl` and `tmemory.jl` scripts where we do the following:

1. Compute the **theoretical times**:
   - theoretical t_{CPU}: using the total number of floating-point operations for MxM or MxV, the peak $GFLOPS$ reported by `cpu_info()` and the number of physical cores. To provide a realistic estimate, we utilize the base clock frequency rather than the turbo boost frequency.
   - theoretical t_{mem}: using the number of data elements and the RAM frequency.
2. Measure the **measured execution times** for:
   - measured t_{CPU}: using `@benchmark` on the matrix-matrix multiplication of A and B_{mat} and on the matrix-vector multiplication of A and B_{vec}.
   - measured t_{mem}: using `@benchmark` on the memory access patterns of the matrices and vectors. Includes row-wise and column-wise accesses, as well as vector accesses.

Libraries and System (CPU, RAM)

All benchmarks and plots are using MKL [12] (BLAS/LAPACK) library. OpenBLAS was also tested but generally performed worse, especially for small matrices with MxM multi-core.

The host system is an AMD Ryzen 5 5600X [18] (6C/12T) with a 3.7 GHz base frequency (this base clock is used for the theoretical GFLOPS estimate) and 32 GB DDR4 memory running at 3200 MHz.

3.2.2 Benchmark results: GFLOPS

This section reports CPU GFLOPS for dense matrix kernels and organizes the discussion around five ideas: (A) single-core MxM vs. MxV, (B) multi-core MxM vs. MxV, (C) single & multi MxM vs. theoretical peaks, (D) low-N behavior (overheads vs. asymptotics), and (E) an overview for Float64.

In brief: on a single core, MxM consistently outperforms MxV; with multiple cores, MxM benefits markedly from parallelism while MxV remains bandwidth-bound; the single-core MxM curve tracks its theoretical limit closely, whereas the multi-core curve remains below its ideal due to shared-resource contention; at small sizes, parallel overheads can outweigh benefits; and Float64 sustains lower throughput but preserves the same qualitative behavior.

(A) FP32: Single-core MxM vs. MxV

On a single core, matrix–matrix multiplication (MxM) delivers substantially higher GFLOPS than matrix–vector (MxV) across sizes. MxM is compute-bound with high arithmetic intensity ($O(N^3)$ work over $O(N^2)$ data), letting BLAS microkernels keep FMA pipelines busy and reuse cache-blocked tiles effectively. By contrast, MxV performs much less work per byte moved ($O(N^2)$ work over $O(N^2)$ data), so performance is limited by memory bandwidth and remains relatively flat with N.

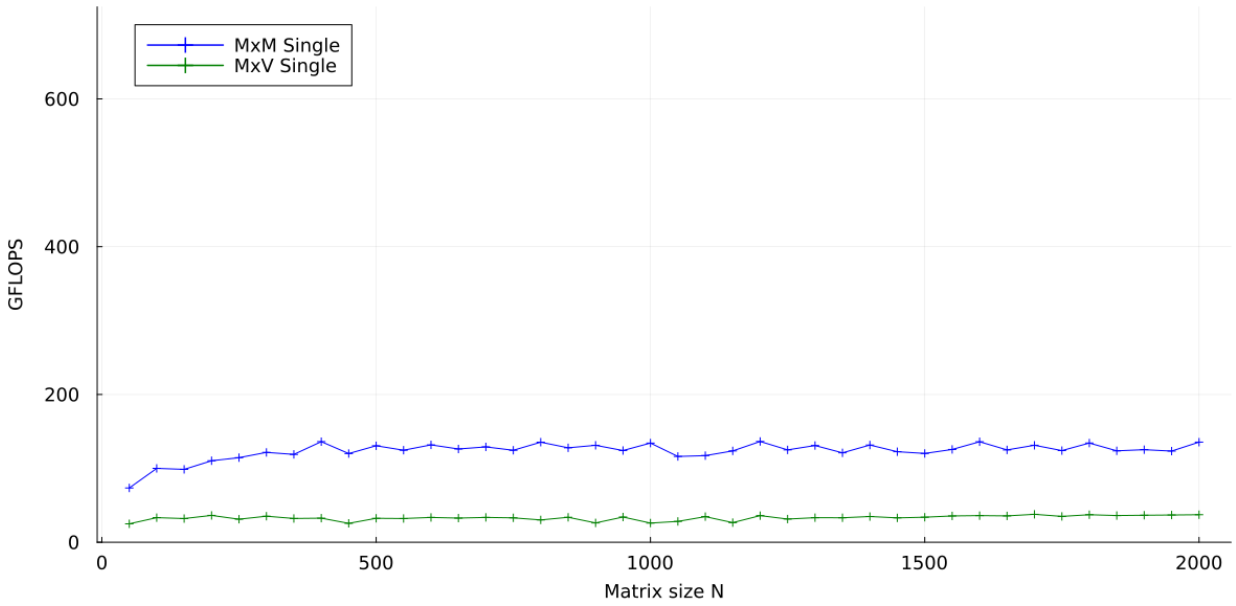

Figure 3.5: FP32, single-core: MxM vs. MxV. MxM clearly outperforms MxV across sizes. The reason is arithmetic intensity: MxM amortizes memory traffic via cache-blocked tiles and sustains high utilization of FMA units (compute-bound), whereas MxV quickly becomes limited by memory bandwidth (memory-bound), yielding a flatter, lower curve.

(B) FP32: Multi-core MxM vs. MxV

With multiple cores, MxM benefits strongly once N is large enough for effective parallelism: the workload exposes ample independent blocks, and threading overheads become negligible relative to the total compute.

In contrast, MxV exhibits little separation between single-core and multi-core curves because additional cores share the same memory subsystem, so bandwidth remains the bottleneck.

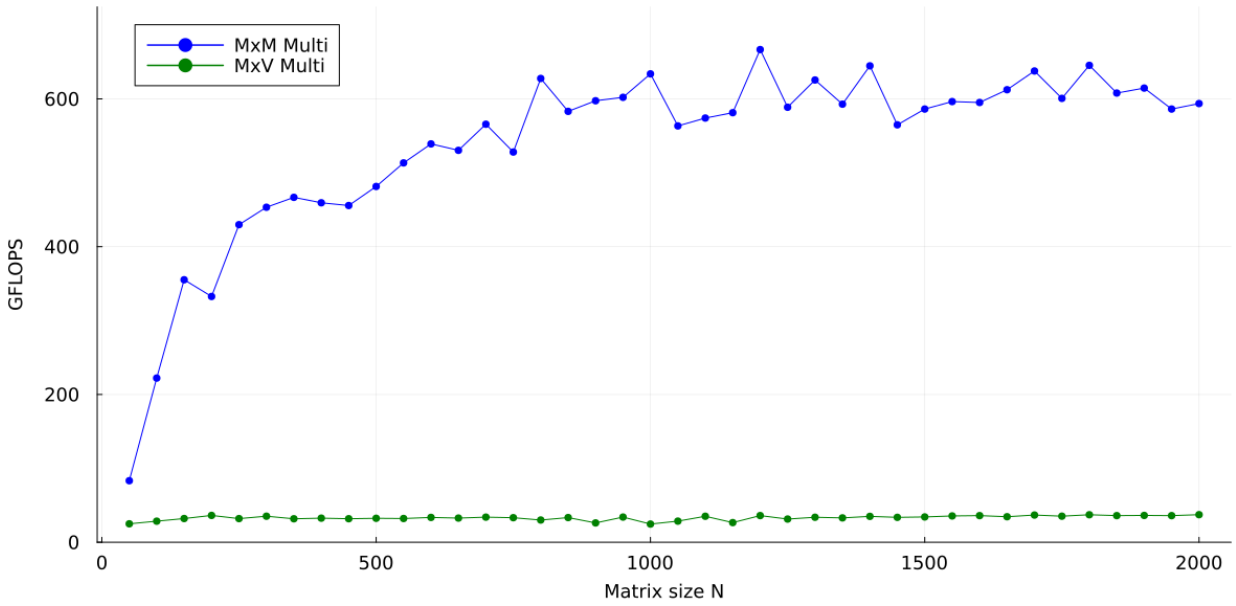

Figure 3.6: FP32, multi-core: MxM vs. MxV. MxM gains markedly from parallelism once N is sufficient to amortize threading overheads and feed all cores; MxV barely scales because RAM bandwidth is shared and already constraining performance, so extra cores cannot increase sustained FLOPS meaningfully.

Something to mention is that we see a slight drop for the largest N near the end of the benchmark in MxM.

This behavior is typical under sustained all-core load: (i) power/thermal limits reduce all-core turbo frequencies (thermal or power throttling), and (ii) the working set exceeds the last-level cache, pushing more traffic to RAM because of cache misses.

(C) FP32: MxM (single & multi) vs. theoretical

Comparing measured MxM to theoretical ceilings clarifies limits. The single-core curve typically tracks its peak closely because there is no inter-core contention and BLAS microkernels keep pipelines saturated with regular access patterns.

The multi-core curve, however, stays below its ideal due to lower all-core frequencies mostly because of power/thermal constraints, shared last-level cache and memory bandwidth.

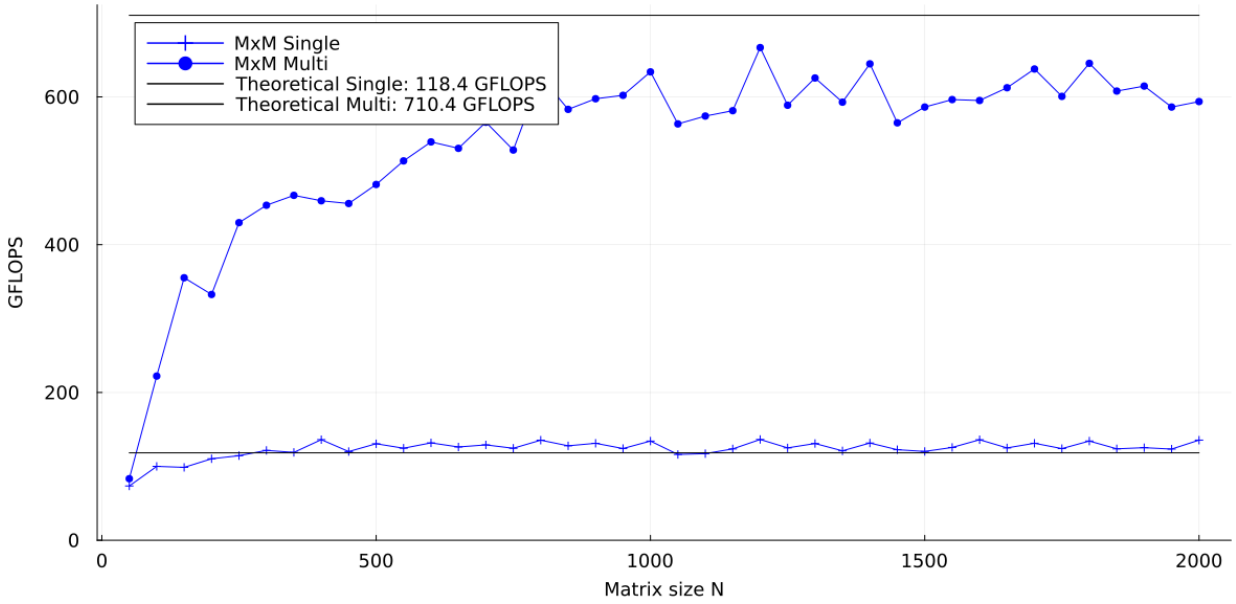

Figure 3.7: FP32: MxM (single & multi) vs. theoretical. Single-core MxM exceeds the theoretical line thanks to efficient microkernel execution and no resource contention. Multi-core MxM rises with N but remains below its theoretical bound because all-core turbo is lower than single-core boost and shared resources (shared L3 cache, memory bandwidth) and coordination overheads limit scaling.

It is important to note that the theoretical GFLOPS have been calculated by the script `system_info.jl`, for the Ryzen 5 5600X, using the **base clock speed** and the number of cores. This is why the theoretical values are lower than the measured values in single core as they do not account for the boost clock speeds that are utilized during benchmarking when only a single core is active.

(D) FP32: Low-N behavior (zoom)

At small sizes, parallelization costs dominate: thread creation, synchronization, and less effective cache blocking can make multi-core performance match or even trail single-core. Those constant-time costs that are not negligible as at large N.

As N grows, the asymptotic $O(N^3)$ compute of MxM quickly overwhelms overheads, tiles fit better into the blocking strategy, and multi-core scaling emerges toward its plateau.

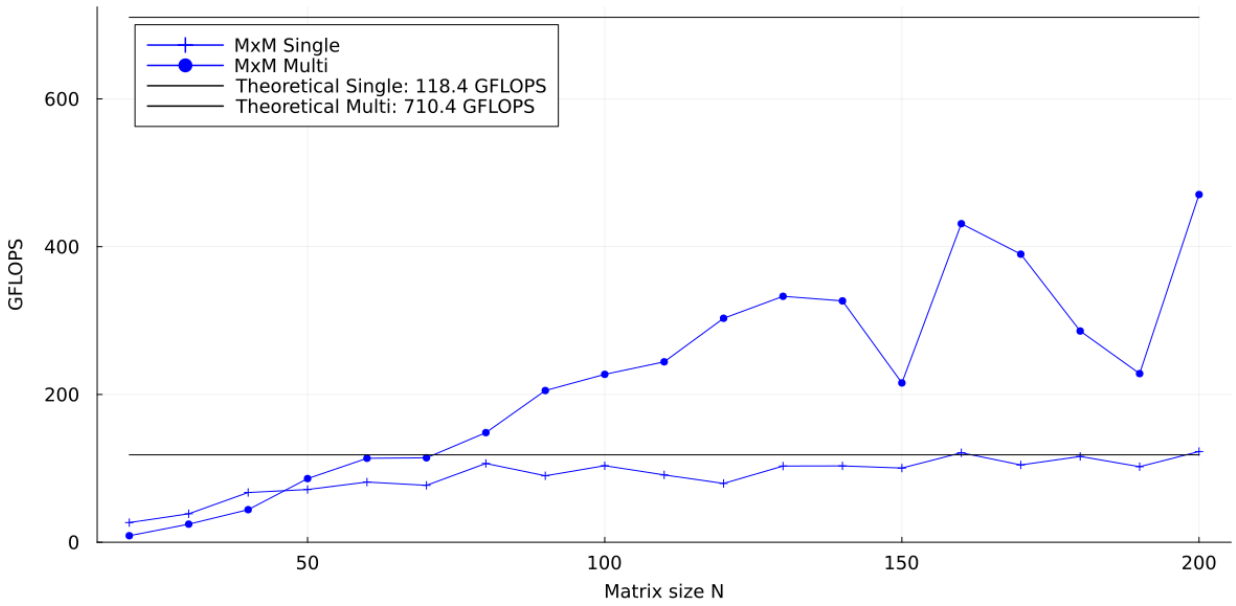

Figure 3.8: FP32 (low-N zoom): single vs. multi vs. theoretical for MxM. For tiny matrices, multi-core can underperform due to overheads and limited reuse; single-core often benefits from excellent locality. Increasing N restores the expected order: multi-core overtakes single-core as blocking becomes effective and parallel work dwarfs coordination costs.

In practice, optimized libraries like MKL often employ dynamic threading (using few threads for small N, then increasing) precisely to avoid oversubscription in this regime, but this would not be as effective as using a single thread for very small N.

(E) Float64 (overview)

With Float64, throughput drops relative to Float32 while preserving the same qualitative picture: MxM remains compute-bound and benefits from larger N and multi-core parallelism, whereas MxV stays bandwidth-bound with minimal scaling.

The gap to multi-core theoretical peaks tends to widen because doubling the data size increases cache pressure and effective bandwidth demands per operation, accentuating shared-resource limits.

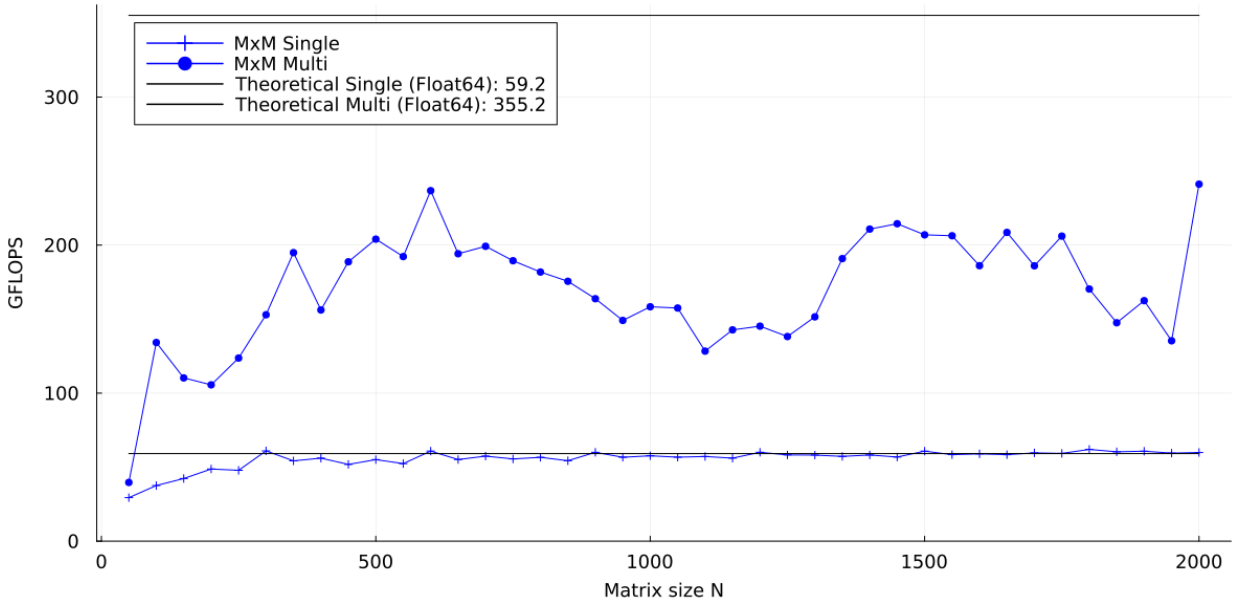

Figure 3.9: FP64: MxM (single & multi) vs. theoretical. Same trends as FP32 with lower absolute throughput and a larger gap to the multi-core theoretical due to SIMD/FMA processing half as many elements per cycle and doubles consume twice the bytes and due to higher memory pressure and shared-resource contention

y-axis scale warning: When comparing FP64 and FP32 plots, be careful with the y-axis scale: FP64 theoretical peaks are roughly half of FP32 because a fixed-width SIMD register (e.g., AVX2 at 256 bits) holds half as many elements (4 doubles vs. 8 floats), halving FMA throughput per cycle. Additionally, as we already said, each element is twice the size, increasing memory pressure for bandwidth-bound kernels.

3.2.3 Benchmark results: theoretical vs measured time

This section compares theoretical compute time (t_{CPU}) and theoretical memory time (t_{mem}) against measured execution times for MxM and MxV operations in a multi-core context.

The theoretical curves act as lower bounds: t_{CPU} assumes perfect peak FLOPS utilization without stalls, while t_{mem} assumes RAM-limited access without cache hierarchy benefits or overheads. Measured curves include those real effects.

(A) FP32: t_{CPU} vs. t_{mem} in MxM

For MxM, the operation-to-data ratio dominates the shape: the useful work grows as $O(N^3)$ while the moved data is $O(N^2)$. Consequently, t_{CPU} grows faster with N than t_{mem}. Optimized BLAS kernels (blocking/tiling) maximize reuse so the measured time follows the compute-bound trend and stays well above t_{mem} for moderate and large N.

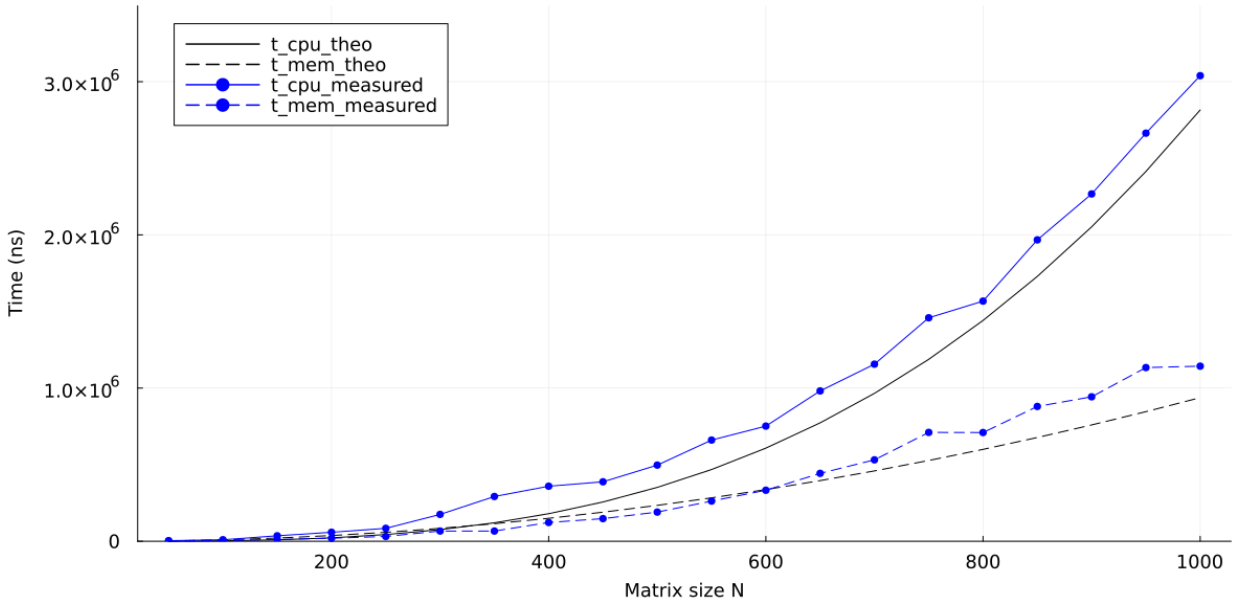

Figure 3.10: FP32 MxM: theoretical vs. measured times. t_{CPU} scales as $O(N^3)$ and dominates for moderate/large N; measured time tracks the compute-bound model thanks to cache-blocked reuse. t_{mem} is a lower bound assuming RAM-limited traffic; real runs reuse data from caches and remain above it.

Measured t_{mem} is below the theoretical line across small and mid sizes (consistent with L1/L2/L3 reuse). A gentle crossover where measured t_{mem} overtakes the RAM-only bound appears only near the upper end of the tested N range, matching the expectation that cache capacity and controller saturation effects surface late in MxM due to strong reuse.

(B) FP32: $t_{\mathbf{CPU}}$ vs. $t_{\mathbf{mem}}$ in MxV

MxV has low arithmetic intensity ($O(N^2)$ FLOPs over $O(N^2)$ data) and quickly becomes bandwidth-bound. Here the theoretical t_{mem} tends to dominate t_{CPU}.

For large N, the memory time exceeds the theoretical bound due to cache misses, row/column conflict patterns and contention in the memory controller, none of which are modeled in the simple t_{mem} bound.

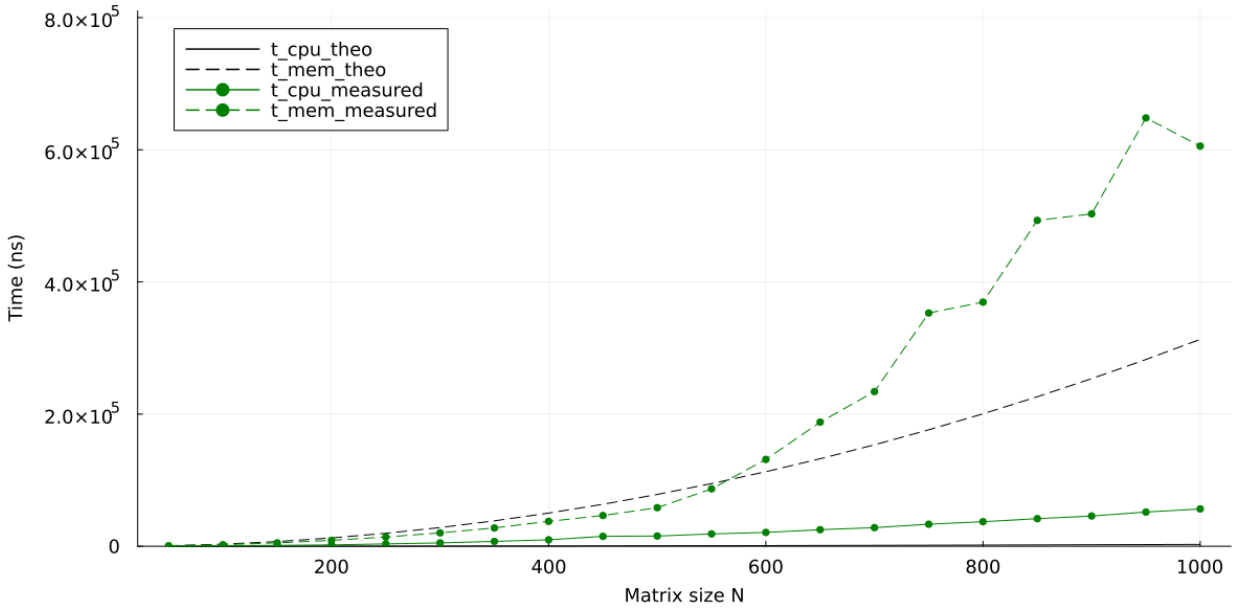

Figure 3.11: FP32 MxV: theoretical vs. measured times. Bandwidth-bound across sizes: t_{mem} dominates t_{CPU}. At large N, the measured memory time rises above the theoretical curve due to cache misses and controller saturation not captured by the simple RAM-bound model.

The divergence (measured t_{mem} climbing above the RAM-only line) appears already at mid-to-large sizes, earlier than in MxM and climbing faster due to the lower reuse in GEMV, which leads to increased memory pressure and contention. Keep in mind that the RAM-only bound is a simplified model that doesn't account for these overheads, causing the measured performance to be worse than predicted when cache efficiency degrades and memory subsystem bottlenecks emerge.

(C) FP32 (low-N zoom): t_{CPU} vs. t_{mem} in MxM

With low-N, we see that the measured t_{CPU} curve sits goes above what we expect, this is because of coordination costs multi-core overheads and short-run effects.

This can place the measured t_{CPU} curve above the measured t_{mem} initially, something that is maybe counterintuitive if we think that in these low-N regimes, the impact of cache usage would help where all data fits in cache.

Those overheads can also be spotted in very low N in the measured t_{mem} curve, which can exceed the theoretical t_{mem}, because cache advantage does not overcome the increased coordination costs and multi-core overheads.

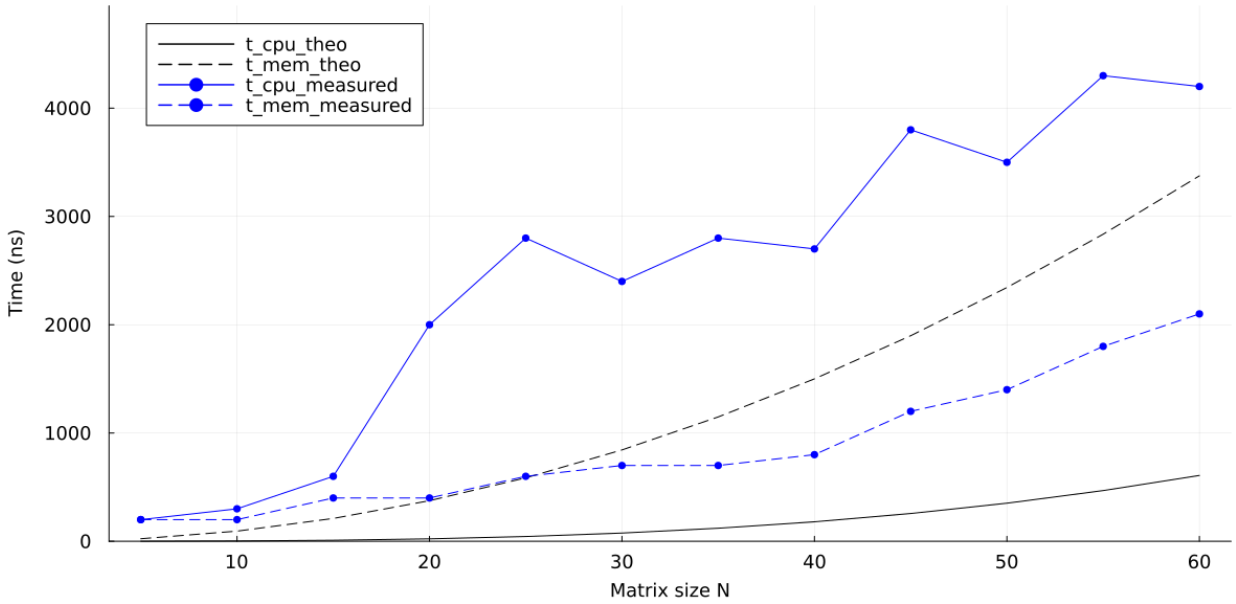

Figure 3.12: FP32 MxM (low-N zoom): In the low-N regime, overheads can make measured t_{mem} exceed the simple theoretical RAM-bound. The t_{CPU} curve remains dominant as N increases, reflecting the strong compute-bound nature of the operation.

Also, do not be misled if the theoretical t_{mem} curve appears above the theoretical t_{CPU} curve at tiny N as the models scale differently. The theoretical t_{mem} is quadratic in N (e.g., $\sim 3N^2$ terms) while the theoretical t_{CPU} is cubic (e.g., $\sim 2N^3$). For very small N, the quadratic with a larger constant sits above the cubic but as N grows, the N^3 term inevitably dominates.

(D) & (E) FP64: $t_{\mathbf{CPU}}$ vs. $t_{\mathbf{mem}}$ in MxM and MxV

In Float64, trends mirror FP32 but with larger data per element. Doubled bytes increase cache pressure and bandwidth demand, reflected in the N at which measured t_{mem} rises above the theoretical bound shifts to smaller sizes, reflecting earlier cache overflow.

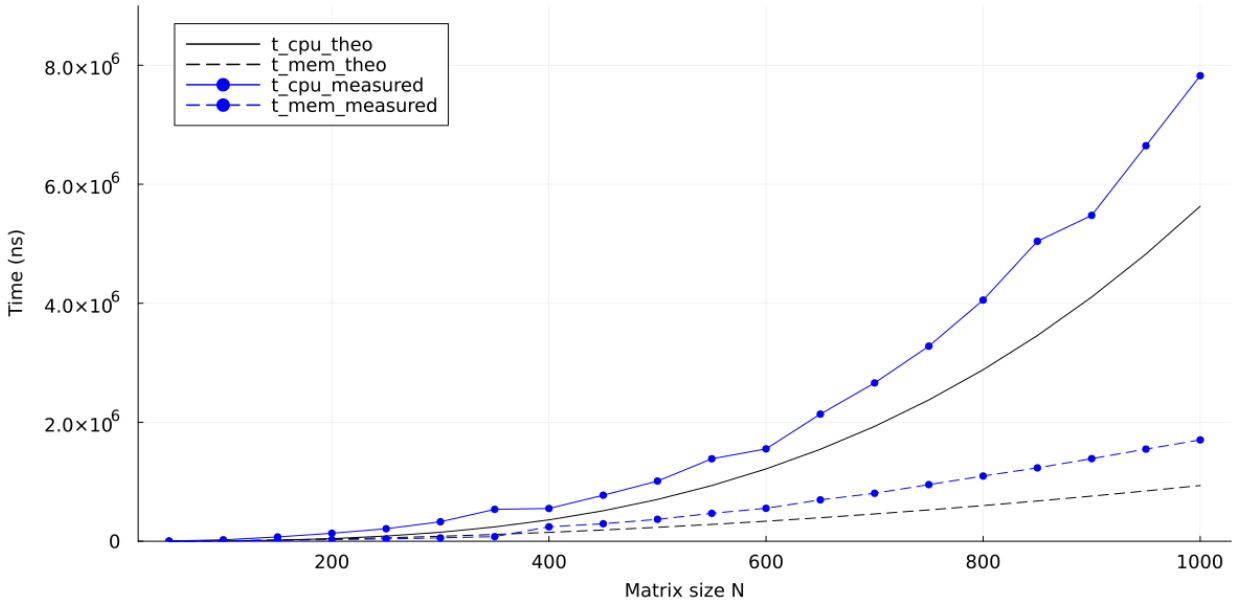

Figure 3.13: FP64 MxM: theoretical vs. measured times. Same qualitative picture as FP32: compute-bound with measured times tracking t_{CPU}.

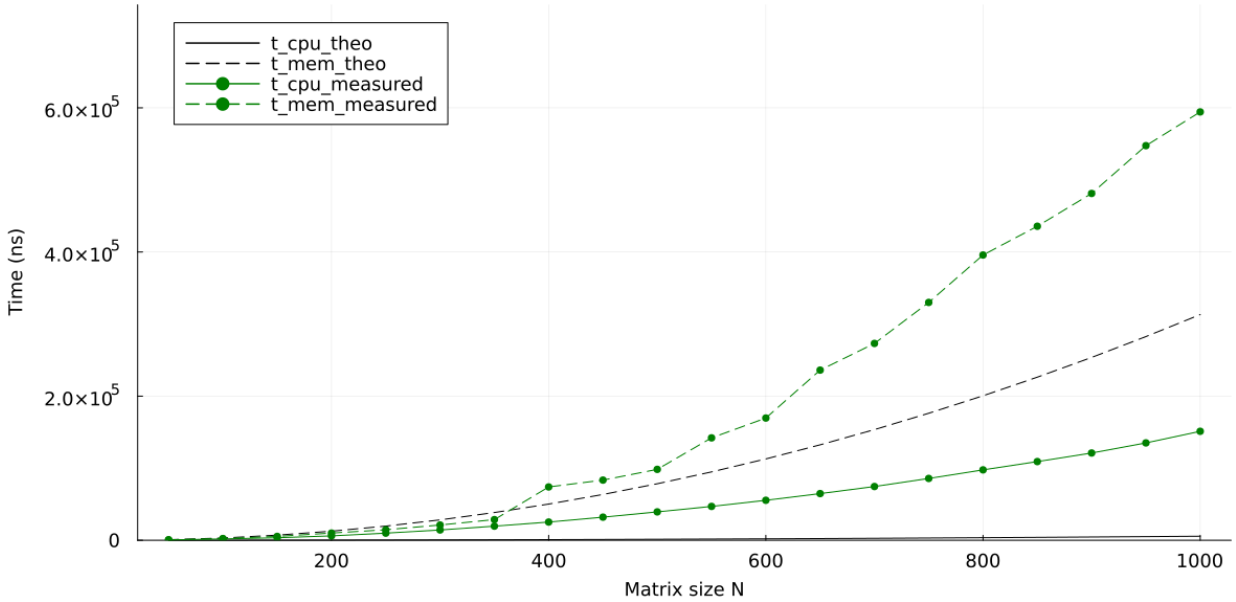

Figure 3.14: FP64 MxV: theoretical vs. measured times. Even more bandwidth-bound than FP32: the crossover to measured t_{mem} exceeding the theoretical bound occurs at smaller N due to doubled element size and earlier cache overflow.

3.2.4 Benchmark results: cache conflict spikes

To expose cache–set conflicts explicitly, we rerun the memory-time experiment for **MxM** with a fine-grained grid of sizes N (step 8). We plot again the measured memory time for the kernel: the goal is to reveal conflict-driven spikes that can be masked by coarse sampling.

(A) FP32: conflict spikes with fine N

In FP32, the memory-time curve shows pronounced spikes at regularly spaced sizes when scanning rows. In column-major layout, row-wise traversal advances by a stride of $s = N \cdot \text{sizeof}(\text{Float32}) = 4N$ bytes. With an L1 data cache of 32 KB, 64 sets, 8-way associativity and 64-byte lines, the set index advances by

$$k = \frac{s}{64} = \frac{N}{16}.$$

When k shares a large divisor with the 64 sets (e.g., N a multiple of 128), accesses concentrate on few sets and saturate the 8-way associativity, causing conflict misses and sharp latency spikes. Using a finer N-grid makes these resonances stand out clearly.

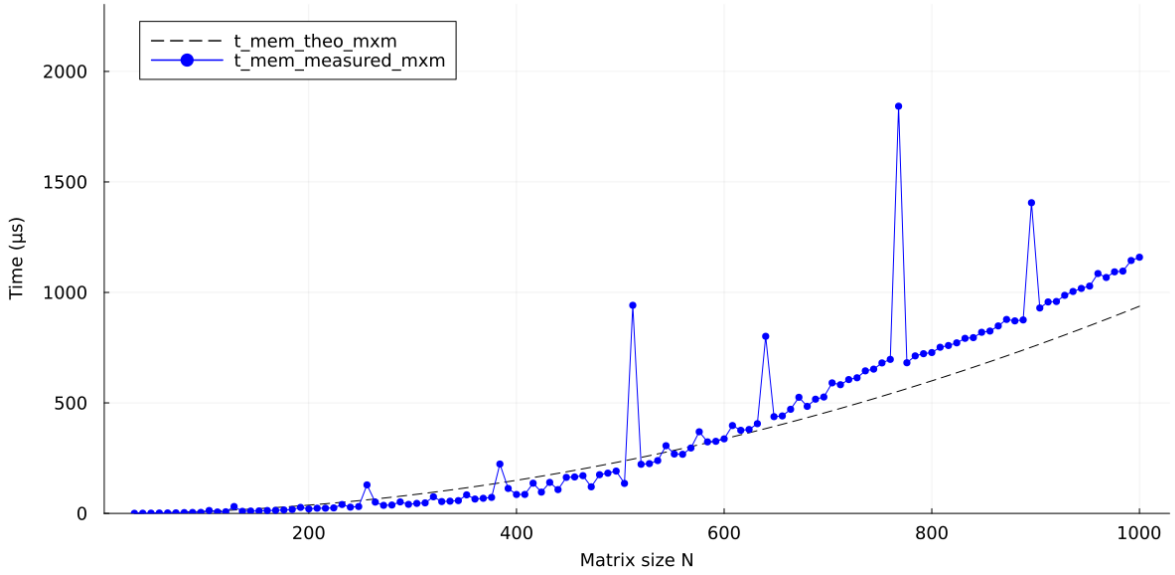

Figure 3.15: FP32 MxM (memory time, fine N). Distinct spikes appear at conflict sizes where the row stride $4N$ bytes aliases onto a small subset of the 64 L1 sets (32 KB, 8-way, 64B lines). In those cases the set footprint exceeds the 8-way capacity and lines evict each other aggressively, inflating the measured memory time.

To understand the conflict spikes in an easier way, consider the following:

At $N = 128$, $4N = 512 = 64 \times 8$: each column hop moves the cache set by exactly $+8$, so you keep accessing the same 8 of 64 sets and overflow the 8-way associativity → spike.

At $N = 129$, $4N = 516 = 64 \times 8 + 4$: that extra 4 bytes accumulates, so over time you visit all 64 sets and spread the pressure → no spike.

(B) FP64: same spikes plus interleaved ones

In FP64, the same conflict sizes reappear and additional spikes emerge between them. The stride doubles to $s = N \cdot \text{sizeof(Float64)} = 8N$ bytes, so the set-step is

$$k = \frac{s}{64} = \frac{N}{8},$$

which makes many more N satisfy the same aliasing condition (large $\gcd(k, 64)$). As a result, conflict resonances occur more frequently (e.g., at multiples of 64, not only of 128), producing interleaved spikes between the FP32 peaks. Moreover, each 64B line holds half as many elements (8 vs. 16), increasing pressure and amplifying spike height.

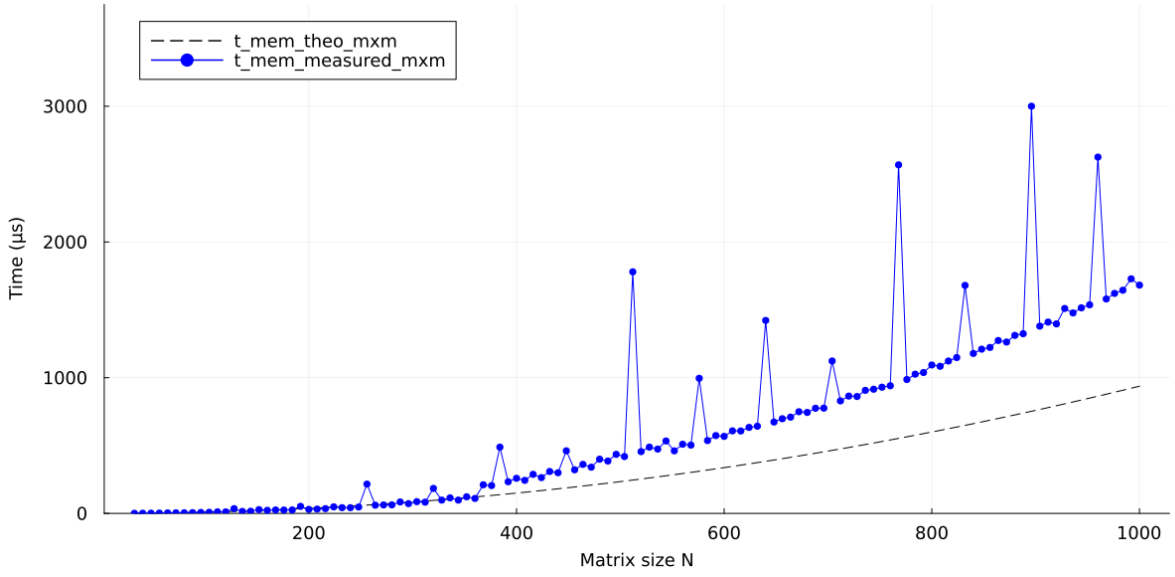

Figure 3.16: FP64 MxM (memory time, fine N). The FP32 resonance sizes persist, and new interleaved spikes appear because the larger stride $8N$ bytes increases the number of N that map onto few L1 sets (32 KB, 64 sets, 8-way, 64B lines). With fewer doubles per line, conflict pressure is stronger and peaks are typically more pronounced.

Chapter 4

GPU Architecture and Performance

4.1 How a GPU Works

Context and Relevance

Having explored the significant capabilities of CPUs, particularly in handling complex sequential tasks, it is evident that they excel in low-latency operations and intricate control flows. However, when it comes to massive parallelization, GPUs emerge as the superior choice. Their architecture, specifically designed to execute thousands of simultaneous operations, makes them indispensable for tasks requiring extensive parallel processing.

In the last decade, Graphics Processing Units (GPUs) have evolved from being exclusive components for rendering graphics to becoming key tools in High-Performance Computing (HPC) [21]. Technologies such as artificial intelligence, physical simulations, data analysis, and scientific computing leverage the massive parallel architecture of GPUs, offering significant acceleration compared to Central Processing Units (CPUs).

Differences between CPUs and GPUs

CPUs and GPUs have architectural designs optimized for different tasks:

- **CPU (Central Processing Unit):**
  - Designed to handle complex and sequential tasks.
  - Contains few cores (typically on the order of 10-100) highly optimized for single-thread performance.
  - Especially efficient for operations requiring low latency and complex control flows.
- **GPU (Graphics Processing Unit):**
  - Designed to execute a large number of operations in parallel.
  - Contains hundreds or thousands of simpler cores organized in massively parallel processing blocks.
  - Ideal for tasks with high parallelization, such as matrix multiplication and vector processing.

4.1.1 Host-Device Communication

GPUs operate in conjunction with a CPU (host) through a well-defined communication topology. This topology involves two primary components:

- **Host:** Refers to the CPU and its system memory (RAM).
- **Device:** Represents the GPU, including its on-board memories (global, shared, etc.).

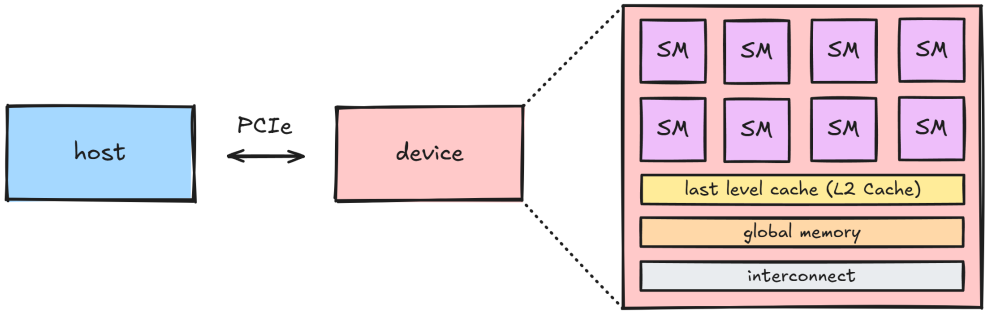

Figure 4.1: Host-Device Communication Topology

This topology division is crucial as the code for launching the kernel and the data to be processed is also divided between a host function and a device function (kernel). The host is responsible for managing the data and launching the kernel, while the device executes the kernel in each thread and performs the computations.

A **kernel** [1], [11] is a function that is executed by each thread in the GPU. The kernel must be written in a way that allows each thread to execute the same code but with different data. The host is responsible for launching the kernel to the GPU threads with the appropriate parameters and managing the data that will be processed by the kernel.

Before discussing the GPU itself, we need to understand that data transfer between the host and device occurs via the **PCI Express (PCIe) interface**.

PCIe Version	Transfer Rate	x1 BW	x8 BW	x16 BW
3.0	8 GT/s	985 MB/s	7.8 GB/s	15.8 GB/s
4.0	16 GT/s	1.97 GB/s	15.8 GB/s	31.5 GB/s
5.0	32 GT/s	3.94 GB/s	31.5 GB/s	63 GB/s

Table 4.1: PCIe Bandwidth comparison for different versions.

The NVIDIA GeForce RTX 3070 [19] uses PCIe 4.0, achieving 31.5 GB/s maximum bandwidth for x16 connections. This typically isn't a bottleneck since data is usually transferred once at computation start and results returned at the end, but frequent host-device transfers could create bottlenecks.

4.1.2 GPU Core Architecture and Memory

The architecture of a GPU is designed to handle massive parallelization, with thousands of cores working simultaneously. The architecture is divided into several components, each with its own characteristics and functions.

Figure 4.2: GA104 GPU (NVIDIA RTX 3070): Architecture and Streaming Multiprocessor

The GPU is divided into several Streaming Multiprocessors (SM), shown in the right image. AMD refers to these as Compute Units (CUs). Each SM contains several CUDA cores, which are the processing units of the GPU, responsible for executing threads in parallel. The CUDA cores are organized into warps, blocks, and grids [2], which are the basic units of parallel execution in the GPU. This organization is explained in more detail in the next section.

The GPU architecture features a memory hierarchy divided into two regions [4].

On-Chip Memories (Per Streaming Multiprocessor): These provide high-speed access but with limited capacity.

- **Registers:** The lowest-latency storage, integrated directly with the cores; organized into 32 banks (32 x 32-bit) corresponding to warp threads.
- **L1 Cache:** Serves as a buffer for register spills and frequently accessed data; physically unified with shared memory.
- **Shared Memory:** Explicitly managed by the programmer for intra-block data sharing; configurable partitioning with L1 cache.
- **Constant Caches:** Dedicated to read-only constants across threads in a warp or block.

Off-Chip Memories (Shared Across Streaming Multiprocessors): These offer greater capacity at the expense of increased latency.

- **L2 Cache:** A unified cache facilitating data transfer between global memory and PCIe.
- **Global Memory:** The primary device memory (e.g., GDDR6 on RTX 3070, bandwidth of approximately 500 GB/s); characterized by high latency (300–600 cycles).
- **Local Memory:** Utilized for register spill overflow beyond L1 and L2 caches; incurs significant performance penalties.
- **Texture and Constant Memory:** Optimized for specific access patterns, with texture data cached in L1 and constants in dedicated caches.

Memory Hierarchy

The memory hierarchy in GPUs operates on principles of locality, both temporal (reusing data recently accessed) and spatial (accessing contiguous data), to minimize latency and maximize bandwidth in parallel workloads. When a thread requests data, the GPU traverses the hierarchy starting from the fastest, lowest-capacity levels closest to the cores, falling back to slower, higher-capacity levels only on misses.

Data flow begins with registers: if the required operand is in a thread's private registers, access is immediate (sub-cycle latency). **On a miss or overflow, the system checks the L1 cache** [14] (or shared memory for explicitly managed data), which may hit if the data was recently spilled or shared within the block. **Constant caches are consulted in parallel for read-only broadcasts**, optimizing uniform accesses across warps.

If unresolved on-chip, the request propagates to the L2 cache [16], shared across SMs, which coalesces accesses from multiple warps and buffers global memory traffic. An L2 hit returns data in tens of cycles; **a miss triggers a fetch from global memory (DRAM)**.

Memory Locality

Memory accesses in GPUs must be **coalesced** for efficiency: threads in a warp should access contiguous memory locations to minimize transactions. Non-coalesced accesses lead to serialized loads, wasting bandwidth.

In shared memory, **bank conflicts** occur when multiple threads access the same bank simultaneously, serializing accesses. Shared memory is divided into 32 banks (matching warp size), and proper padding or transposition can mitigate conflicts.

For global memory, alignment to 128-byte segments (cache line equivalent) is critical. In matrix operations, row-major versus column-major access patterns can cause uncoalesced loads, similar to CPU cache thrashing.

Tiling

GPUs use **tiling** (also known as blocking in CPUs) to optimize performance by exploiting data locality. This technique involves loading small chunks of data, called tiles, from the slow global memory into the fast shared memory, where they can be reused multiple times by threads. This reduces the number of costly global memory accesses and increases **arithmetic intensity**, which measures the number of computations performed per byte of data transferred.

In a naive matrix multiplication, each thread computes one element of the result matrix C, reading an entire row from matrix A and an entire column from matrix B from global memory. This leads to high memory traffic, roughly $O(N^2)$ per thread, with low arithmetic intensity since few computations are done per data read. In contrast, tiling divides the matrices into smaller blocks, such as 32×32 tiles, that fit in shared memory. Threads within a block work together to load these tiles, perform computations, and synchronize.

Libraries like `cuBLAS` and custom kernels apply **multi-level tiling**, using different tile sizes to optimize for shared memory, L1/L2 caches, and registers, depending on the GPU's architecture. For matrix-vector multiplication (MxV), however, tiling is less effective. Since each output element requires a full row of the matrix and the input vector with minimal data reuse, the operation has low arithmetic intensity ($O(1)$) and remains **bandwidth-bound**.

4.1.3 Instruction-Level Parallelism: SIMT and Vectorization

Modern GPUs employ a **Single Instruction, Multiple Threads (SIMT)** paradigm, analogous to SIMD in CPUs but scaled to thousands of threads. In SIMT, a single instruction is executed across multiple threads simultaneously, each operating on different data elements, making it highly efficient for data-parallel workloads like matrix operations.

Threads are grouped into **warps** (typically 32 threads on NVIDIA GPUs), executing the same instruction in lockstep. If threads in a warp diverge (e.g., due to conditional branching), the GPU serializes divergent paths, masking inactive threads, which can reduce efficiency. To maximize performance, kernels should minimize branch divergence.

FMA operations are critical in numerical workloads, counting as two floating-point operations per cycle. For matrix multiplication, high-level operations like GEMM are decomposed into tiled kernels that leverage SIMT parallelism and Tensor Cores for acceleration.

4.2 Benchmarking on GPU

In this section, we benchmark and compare the performance of **matrix–matrix multiplication** (MxM) and **matrix–vector multiplication** (MxV) on the GPU. The aim is to measure, for each operation, the sustained performance in $GFLOPS$ as the matrix size N increases, and to contrast these measurements with theoretical limits imposed by both computation and memory access.

Theoretical Compute Performance

The theoretical time required to execute a given number of floating–point operations, assuming the GPU operates at its peak throughput, can be estimated as:

$$t_{GPU} = \frac{N_{ops}}{GFLOPS \times 10^9} \times 10^6, \tag{4.1}$$

where t_{GPU} is in microseconds.

The peak performance in $GFLOPS$ is computed as:

$$GFLOPS = C_{SM} \times C_{cores/SM} \times GPU_{GHz} \times M_{ops}. \tag{4.2}$$

The terms in the equation are:

- C_{SM}: number of Streaming Multiprocessors (e.g., 46 on RTX 3070).
- $C_{cores/SM}$: number of CUDA cores per SM (e.g., 128 on Ampere architecture).
- GPU_{GHz}: GPU clock speed in GHz (boost clock, e.g., 1.725 GHz on RTX 3070).
- M_{ops}: floating–point operations per cycle per core. For Fused Multiply–Add (FMA) instructions, each operation performs one multiplication and one addition in a single step, counting as two floating–point operations. On the NVIDIA RTX 3070 (Ampere), each CUDA core can execute 1 FMA per cycle, resulting in $M_{ops} = 2$.

This formula gives the peak for FP32 operations (e.g., 20.31 TFLOPS on RTX 3070). For FP64 in consumer GPUs, performance is typically limited intentionally (e.g., to 1/64 of FP32 on Ampere consumer cards, 317 GFLOPS) to encourage users needing high FP64 throughput to purchase more expensive HPC-oriented GPUs.

Theoretical Memory Access Performance

Memory-bound operations are limited by global memory bandwidth or PCIe transfers. The time to access data is:

$$t_{mem} = \frac{N_{data} \times \text{Element Size}}{BW_{eff}} \times 10^6, \tag{4.3}$$

where BW_{eff} is the effective bandwidth and it is calculated dynamically from GPU attributes:

$$BW = 2 \times (f_{mem} \times 10^3) \times \frac{w_{bus}}{8}, \tag{4.4}$$

where:

- f_{mem} is the memory clock rate in kHz.
- w_{bus} is the global memory bus width in bits.
- The factor 2 accounts for double data rate (DDR) in GDDR memory.
- The 10^3 converts kHz to Hz.
- Division by 8 converts bits to bytes.

Both f_{mem} and w_{bus} can be obtained from the CUDA.jl package by running:

- `CUDA.attribute(dev, CUDA.CU_DEVICE_ATTRIBUTE_MEMORY_CLOCK_RATE)`
- `CUDA.attribute(dev, CUDA.CU_DEVICE_ATTRIBUTE_GLOBAL_MEMORY_BUS_WIDTH)`

This yields peak theoretical BW, e.g., $\approx 448 \times 10^9$ bytes/s (448 GB/s) on RTX 3070.

For matrix-matrix multiplication (MxM), we estimate the real total memory access time by summing the individual measured components: the time to access a matrix by rows, the time to access a matrix by columns, and the time to allocate a matrix.

For matrix-vector multiplication (MxV), we sum the time to access a matrix by rows, the time to access a vector, and the time to allocate a vector.

4.2.1 Benchmark Methodology

The **GFLOPS benchmarks** are run using the `gflops_1operation_gpu.jl` script with CUDA.jl and cuBLAS. For each matrix size N, we do the following:

1. Allocate random matrices/vectors on GPU: A $(N \times N)$, B_{mat} $(N \times N)$, B_{vec} $(N \times 1)$.
2. Perform a warm-up multiplication to compile kernels.
3. Benchmark a **single multiplication** using `@benchmark` (`evals=1`, 200 samples), taking the minimum time to compute sustained GFLOPS:

$$GFLOPS_{\text{MxM}} = \frac{2N^3}{t_{MxM}} \times 10^{-9}, \quad GFLOPS_{\text{MxV}} = \frac{2N^2}{t_{MxV}} \times 10^{-9}.$$

Theoretical peaks are derived from GPU specs via `gpu_info()`. Benchmarks exclude host-device transfers to focus on pure device performance.

Theoretical memory access time t_{mem} and compute time t_{GPU} benchmarks are run using the `tgpu_tmemory_GPU.jl` script. For each matrix size N we do:

1. Compute the **theoretical times**:
   - theoretical t_{GPU}: using the total number of floating-point operations for MxM or MxV and the peak $GFLOPS$ reported by `gpu_info()` (based on SM count, clock, and FMA throughput, including the device FP32:FP64 ratio).
   - theoretical t_{mem}: using the number of data elements to load/store and the GPU global memory bandwidth.
2. Measure the **measured execution times**:
   - measured t_{GPU}: using `@benchmark` on $A \times B_{\text{mat}}$ (MxM, cuBLAS) and on $A \times B_{\text{vec}}$ (MxV, cuBLAS), device-only (excluding host–device transfers).
   - measured t_{mem}: using `@benchmark` on memory access patterns over matrices and vectors (row-wise, column-wise, and vector) and on allocations, mirroring the CPU methodology.

Libraries and System (GPU)

All benchmarks and plots are using `CUDA.jl` and cuBLAS.

The device is an NVIDIA GeForce RTX 3070 (Ampere, GA104) with 46 SMs and 8 GB GDDR6 on a 256-bit bus (with an effective bandwidth of approximately 448 GB/s).

Theoretical peaks are ∼20.3 TFLOPS (FP32) and ∼317 GFLOPS (FP64) (FP32:FP64 ≈ 64:1).

4.2.2 Benchmark results: GFLOPS

This section organizes the GPU GFLOPS discussion around three ideas: (A) FP32 MxM vs. MxV, (B) FP32 MxM vs. FP64 MxM (hardware FP64 ratio), and (C) FP64 MxM vs. MxV. Throughout, we focus on trends rather than exact values.

(A) FP32: MxM vs. MxV

On the RTX 3070, FP32 MxM ramps up quickly with N and settles into a high plateau. This is the classic compute-bound behavior where cuBLAS tiling exploits shared memory and the L2 cache, keeping SM pipelines busy with high reuse.

In contrast, FP32 MxV stays far below, almost despreciable compared to MxM, and grows only modestly as N increases. Its arithmetic intensity is tiny, so additional threads mostly wait on GDDR traffic rather than issuing more useful FMAs; performance is governed by memory bandwidth rather than by SM throughput.

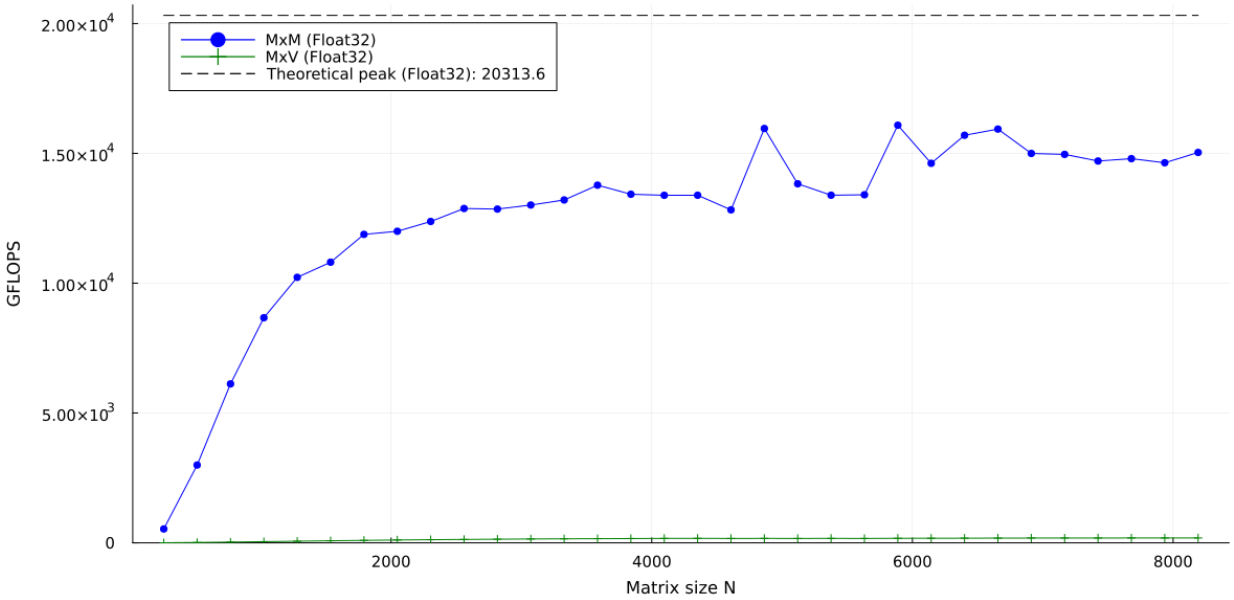

Figure 4.3: GFLOPS GPU FP32: MxM vs. MxV. MxM, being compute-bound, climbs sharply and approaches a high plateau as cuBLAS tiles reuse data in shared memory and L2 cache. On the other hand, MxV, being memory-bound, sits much lower and changes little with N: extra parallelism cannot overcome the memory wall, so GFLOPS remain well below MxM.

Minor undulations on the MxM curve at large N are consistent with tile-size/occupancy boundaries and cache/L2 effects, but the overall picture remains a rise followed by a plateau.

(B) **FP32** MxM **vs. FP64** MxM

Consumer Ampere GPUs throttle FP64 heavily (typically ∼1/64 of FP32 throughput). This is typically limited intentionally on consumer GPUs to encourage users needing high FP64 throughput to purchase more expensive HPC-oriented GPUs.

The result is that FP64 MxM reproduces the shape of the FP32 curve (tiling-driven ramp then plateau) but tops out at a much lower level, so cannot be even seen in the graph. This is fixed by a hardware ratio rather than by algorithmic limits. For large N, FP64 MxM consistently runs near its own (low) theoretical ceiling, illustrating that the bottleneck is architectural FP64 capacity, not inefficiency in the GEMM kernel.

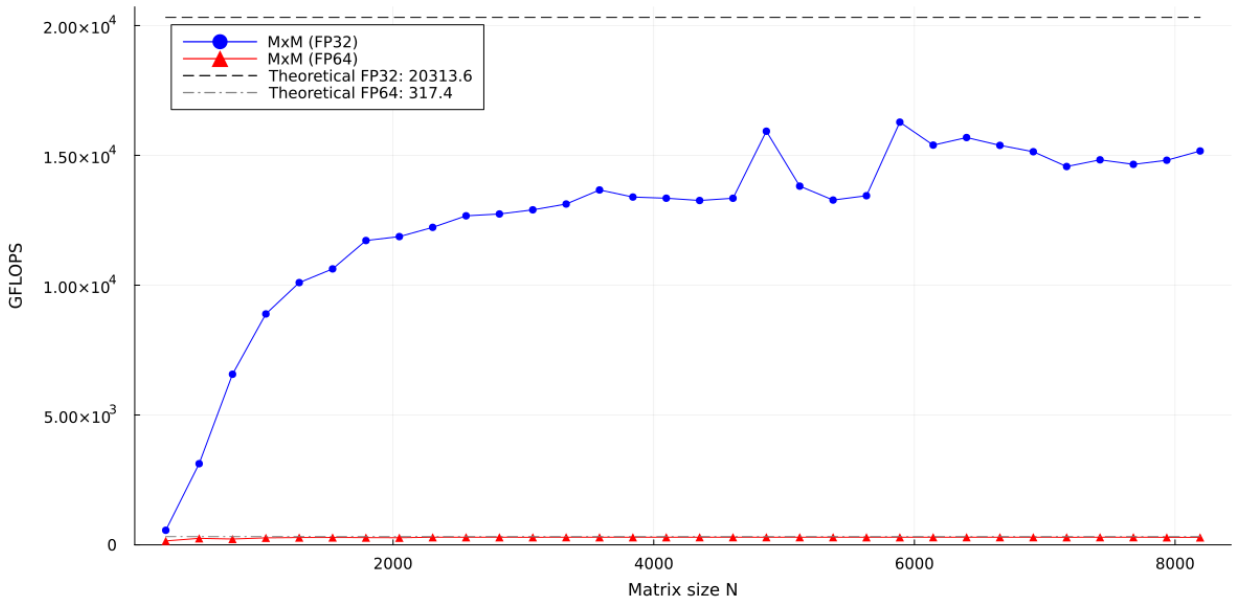

Figure 4.4: GFLOPS GPU: FP32 MxM vs. FP64 MxM. Same scaling mechanics, different ceilings: FP64 saturates well below FP32 because consumer GPUs offer drastically reduced FP64 throughput. Large-N FP64 MxM rides close to its own theoretical limit, highlighting a hardware cap rather than a software one.

The ratio between FP32 and FP64 units depends on the specific GPU architecture and its design choices.

For example, the NVIDIA RTX 3070 (Ampere consumer grade) has only 2 FP64 units per SM, while the NVIDIA A100 (a HPC grade oriented GPU) has 32 per SM.

(C) **FP64:** MxM **vs.** MxV

With FP64, the MxM curve is compute-capped by the architecture as we commented earlier, and MxV remains bandwidth-bound. However, the GPU's memory system is fast (the GPU NVIDIA RTX 3070 memory bandwidth is approximately 448 GB/s while the CPU's is around 50 GB/s on DDR4 Dual Channel 3200 MHz).

Consequently, FP64 MxV climbs to a sizable fraction of FP64 MxM at large N (much closer, in relative terms, than in FP32).

In other words, even though FP64 compute is throttled, the high GDDR bandwidth of the GPU lets a memory-bound kernel exploit a larger share of the available "budget" (the higher memory bandwidth) than it ever could on CPU RAM.

This is why FP64 MxV looks against FP32, comparatively healthy on the GPU, as the faster memory system provides a significant advantage.

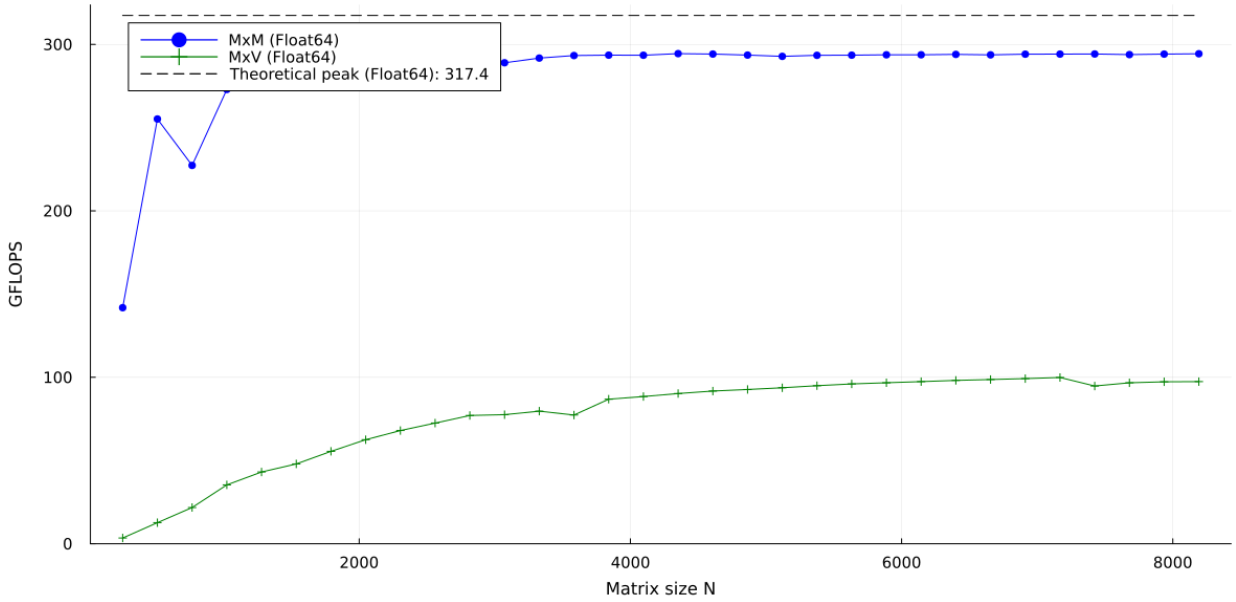

Figure 4.5: GFLOPS GPU FP64: MxM vs. MxV. FP64 compute is heavily throttled, but memory bandwidth is unchanged. As a result, FP64 **MxV** (bandwidth-bound) reaches a substantial fraction of FP64 **MxM**, shrinking the relative gap vs. FP32. This underscores that the limiter for MxV is memory, where GPUs enjoy a big advantage over CPUs.

Finally, in both precisions you may notice lower GFLOPS at very small N, where those points are dominated by fixed kernel-launch and setup overheads before the workload is large enough to amortize them.

4.2.3 Benchmark results: theoretical vs measured time

We compare theoretical GPU compute time (t_{GPU}) and theoretical memory time (t_{mem}) to measured times. As with CPUs, t_{GPU} is a compute-bound lower bound (ideal SM utilization), and t_{mem} is a bandwidth-bound lower bound (ideal coalescing, no overheads).

(A) FP32: t_{GPU} vs. t_{mem} in MxM

For FP32 **MxM**, the operation-to-data ratio dominates again: $O(N^3)$ work over $O(N^2)$ data. cuBLAS's tiling packs blocks into shared memory, driving reuse and keeping warps busy; measured times follow t_{GPU} closely and stay well above t_{mem} for moderate/large N.

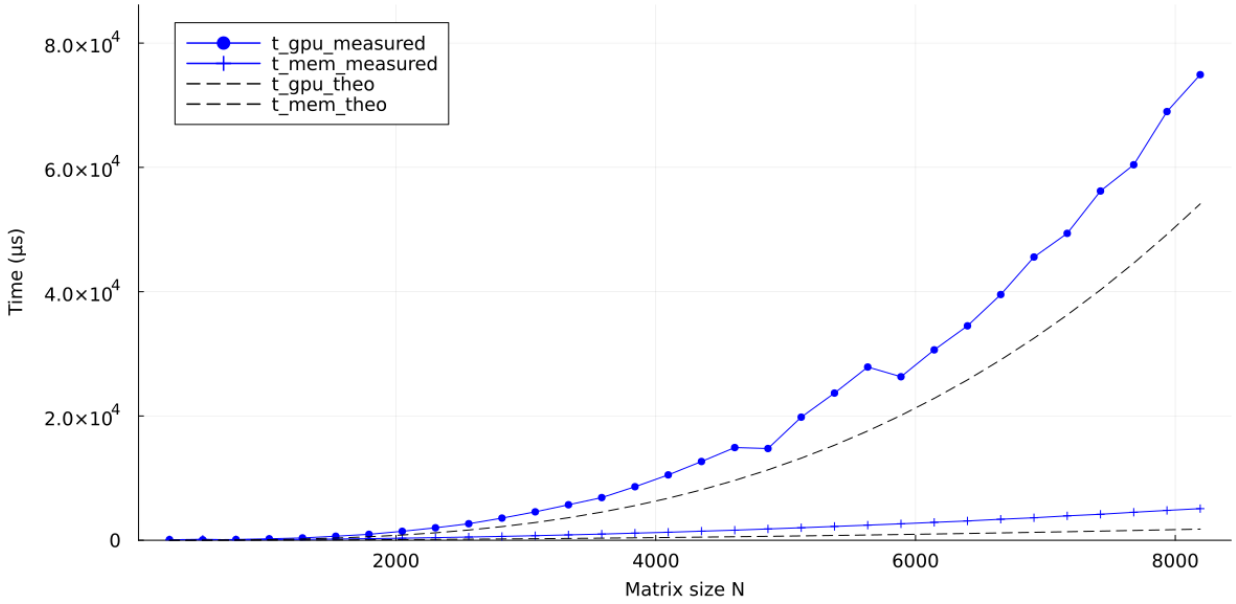

Figure 4.6: GPU FP32 MxM: theoretical vs. measured times. Compute-bound: measured t_{GPU} tracks the theoretical t_{GPU} as tiling and reuse sustain high SM occupancy; t_{mem} remains a lower bound that is not reached in practice. The $O(N^3)$ vs. $O(N^2)$ scaling widens the gap with N.

We can observe that the measured time is consistently above the theoretical compute time, which is expected due to overheads and inefficiencies. However, the gap does not widen dramatically with N because the kernel remains compute-bound, and cuBLAS optimizations keep the GPU busy.

(B) FP32 (low-N zoom): kernel-launch overheads

At small sizes, fixed costs dominate (kernel launch latency, queueing, synchronization, initializations). Both measured curves sit far above their theoretical bounds: there is simply too little work to amortize the launch.

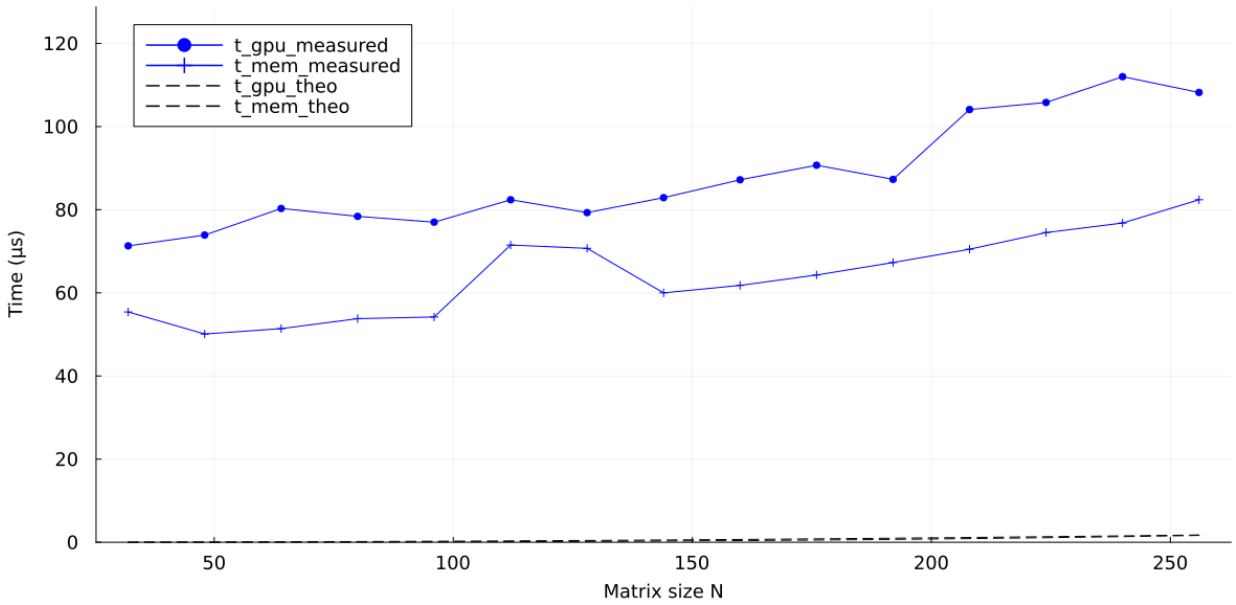

Figure 4.7: GPU FP32 MxM (low-N zoom): Measured times $\gg$ theoretical bounds due to launch/scheduling overheads and underutilization of SMs at tiny problem sizes. Past a threshold N, overheads are amortized and the curve collapses toward the compute-bound regime.

Concretely, at N=32 the theoretical times are essentially negligible ($t_{\text{GPU}} \approx 0\,\mu$s, $t_{\text{mem}} \approx 0\,\mu$s), while the measurements are around the $80\,\mu$s mark.

Even at N=256, the theoretical t_{GPU} does not match the measured $t_{\text{GPU}} \approx 100\,\mu$s. These gaps are the fixed launch/setup costs being amortized too slowly at small problem sizes.

This is due to the host-device model: the CPU must launch the kernel and manage data transfers, which adds latency that is significant when the workload is small.

(C) FP64: $t_{\mathbf{GPU}}$ vs. $t_{\mathbf{mem}}$ in MxM

In FP64 MxM the operation-to-data ratio still rules: $2N^3$ FLOPs vs. $\mathcal{O}(N^2)$ bytes.

However, as consumer GPUs throttle FP64 throughput (e.g., ∼1/32–1/64 of FP32), the kernel is even more compute-bound. As we see at the y-axis scale, the measured t_{GPU} reaches two orders above that we had in FP32.

Again, t_{mem} remains well below compute time despite doubling the element size (8 B) relative to FP32.

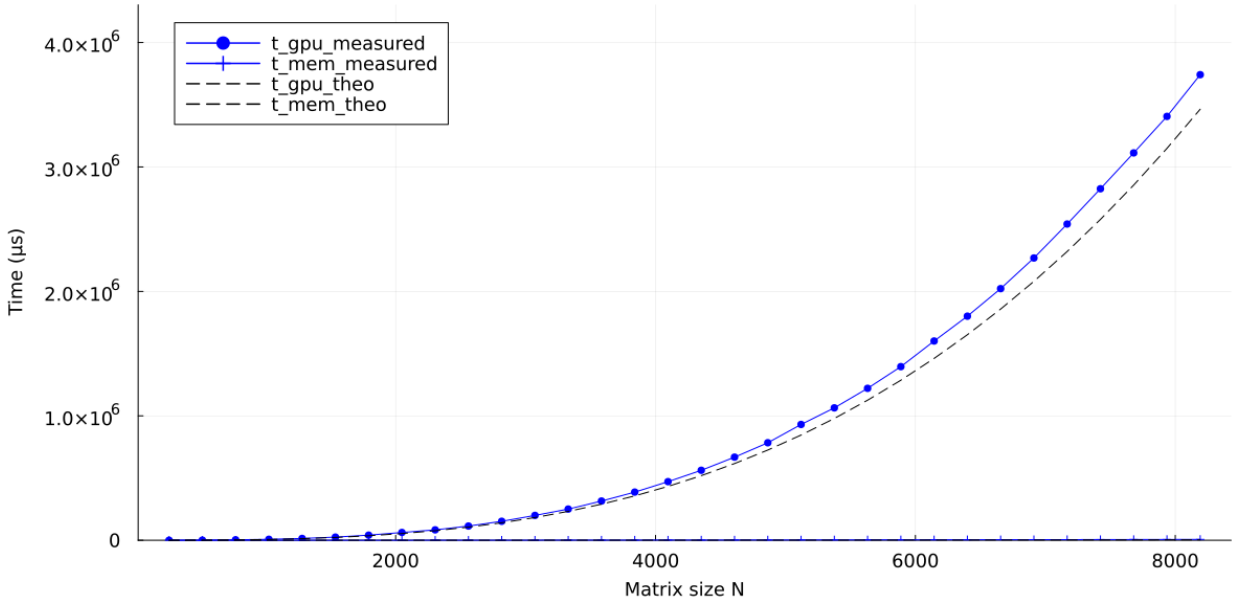

Figure 4.8: GPU FP64 MxM: theoretical vs. measured times. Same N^3 vs. N^2 scaling, but FP64 throughput is hardware-limited on consumer GPUs. Memory traffic (now 8 B per element) still scales as $\mathcal{O}(N^2)$ and stays below compute. The measured t_{GPU} sits nearer its theoretical lower bound due to reduced memory bandwidth utilization and pressure.

Compute is artificially capped, but memory does not dominate anyway, and measured t_{GPU} aligns more tightly with the ideal trend because of reduced memory bandwidth utilization. As a result, the measured t_{GPU} lies closer to its theoretical curve (smaller gap) because memory pressure is even lower than in FP32.

Chapter 5

Comparative between CPU and GPU

5.1 Speedup (GPU / CPU)

This section condenses the comparison to two core views: (A) FP32 MxM vs. MxV speedup, and (B) FP64 MxM vs. MxV speedup. Here, when we refer to speedup, we mean GPU GFLOPS divided by CPU (multi-thread) GFLOPS.

(A) FP32: MxM vs. MxV Speedup

In FP32, both MxM and MxV deliver a clear advantage for GPU over CPU. The effect is much stronger for MxM because it is compute-bound, while MxV also improves but plateaus lower due to its bandwidth-bound nature.

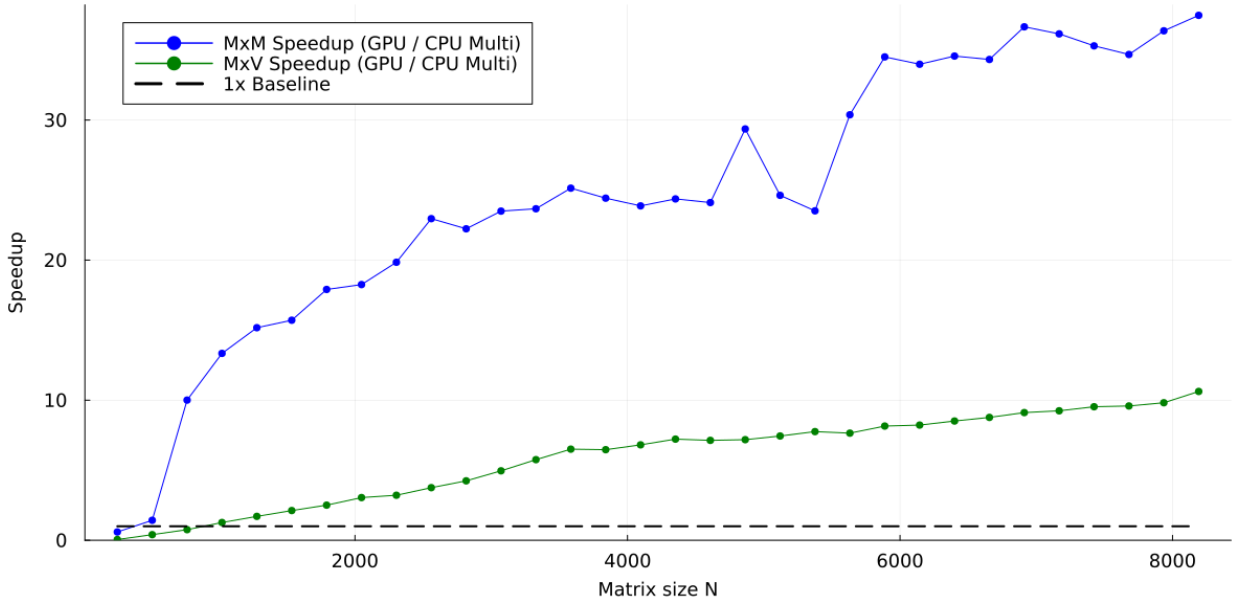

Figure 5.1: FP32 speedup (GPU/CPU): MxM vs. MxV. MxM rapidly enters a high speedup regime as the GPU saturates compute; MxV remains bandwidth-limited and achieves a smaller but noticeable, flatter speedup. We can clearly assume the superiority of GPUs against CPUs for large problem sizes, in both MxM and MxV operations.

It is important to clarify that for very large sizes, the speedup widens further mostly because the CPU side falls off (all-core turbo drops, memory pressure rises, thermal throttling and BLAS strategies become less favorable) so CPU GFLOPS decline while the GPU holds steady.

Even we cannot see where the plateau for MxM exactly begins, do not expect the speedup to keep increasing indefinitely with N as this was caused by the GPU's ability to maintain high throughput because of better handling in thermals and libraries, compared to the CPU.

(B) FP64: MxM vs. MxV Speedup

In FP64, consumer GPUs throttle floating-point throughput, so MxM speedup is much lower than in FP32 even at large N: the kernel remains compute-bound, but the architectural FP64 cap limits the attainable advantage over the CPU.

By contrast, MxV is still memory-bound and can exploit the GPU's much higher memory bandwidth. As a result, its FP64 speedup becomes larger than MxM's (the gap even flips, having a higher speedup on MxV than MxM).

This is the key takeaway for FP64: compute throttling shrinks MxM's headroom, while MxV continues to benefit from fast GDDR, yielding a comparatively healthy speedup.

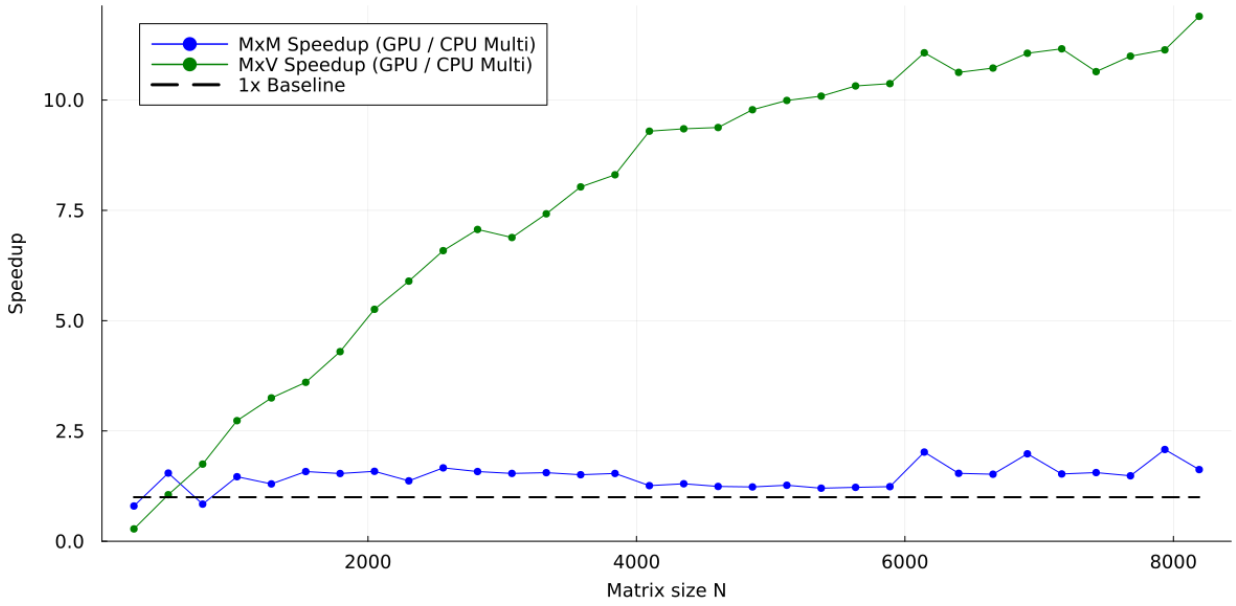

Figure 5.2: FP64 speedup (GPU/CPU): MxM vs. MxV. MxM rapidly enters a high speedup regime as the GPU saturates compute; MxV remains bandwidth-limited and achieves a smaller, flatter speedup.

Chapter 6

Wave Equation 1D with 2nd order finite differences

6.1 Introduction to 1D Wave Equation

We will illustrate this approach using the wave equation as a practical example. The wave equation describes the propagation of waves through a medium and is a fundamental PDE in physics and engineering. It can be expressed in one dimension as:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}, \qquad x \in [0, L],\ t \geq 0, \tag{6.1}$$

where $u(x, t)$ is the displacement at position x and time t, and c is the wave propagation speed.

The initial conditions (IC) are

$$u(x, 0) = A \exp\left(-\frac{(x - x_c)^2}{2\sigma^2}\right), \qquad v(x, 0) = 0, \tag{6.2}$$

and we impose homogeneous Dirichlet boundary conditions (BC) on both fields:

$$u(0, t) = u(L, t) = 0, \qquad v(0, t) = v(L, t) = 0, \qquad t \geq 0. \tag{6.3}$$

The wave equation can be reformulated as a first-order system by introducing an auxiliary variable $v(x, t)$, defined as the first time derivative of u:

$$v = \frac{\partial u}{\partial t}. \tag{6.4}$$

Differentiating u once in time, and recognizing the wave equation as a second-order ODE in time, we reformulate it as a coupled system of two first-order PDEs:

$$\begin{aligned} \frac{\partial u}{\partial t} &= v, \\ \frac{\partial v}{\partial t} &= c^2 \frac{\partial^2 u}{\partial x^2}. \end{aligned} \tag{6.5}$$

6.1.1 Spatial Discretization: 2nd Order Finite Differences

We consider a uniform grid $x_i = i\Delta x$ for $i = 0, 1, \ldots, N$, with $\Delta x = L/N$.
The second derivative is approximated using a centered stencil as

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{x=x_i} \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{\Delta x^2}, \quad i = 1, \ldots, N-1, \tag{6.6}$$

where $u_i(t) \approx u(x_i, t)$. We will now show how this approximation is derived using Lagrange interpolation. This method involves creating a simple polynomial that fits the function values at nearby points, and then calculating its second derivative to match the given formula.

We approximate $u(x)$ near x_i by the quadratic polynomial $p(x)$ that passes through the points (x_{i-1}, u_{i-1}), (x_i, u_i), and (x_{i+1}, u_{i+1}), where $x_{i-1} = x_i - \Delta x$, $x_i = i\Delta x$, and $x_{i+1} = x_i + \Delta x$.

The Lagrange polynomial is expressed as

$$p(x) = u_{i-1}\ell_{i-1}(x) + u_i\ell_i(x) + u_{i+1}\ell_{i+1}(x), \tag{6.7}$$

The Lagrange basis polynomials for points x_{i-1}, x_i, x_{i+1} are:

$$\begin{gathered}\ell_{i-1}(x) = \frac{(x - x_i)(x - x_{i+1})}{(x_{i-1} - x_i)(x_{i-1} - x_{i+1})}, \qquad \ell_i(x) = \frac{(x - x_{i-1})(x - x_{i+1})}{(x_i - x_{i-1})(x_i - x_{i+1})} \\ \ell_{i+1}(x) = \frac{(x - x_{i-1})(x - x_i)}{(x_{i+1} - x_{i-1})(x_{i+1} - x_i)}.\end{gathered} \tag{6.8}$$

Shift coordinates so $x_i = 0$, $x_{i-1} = -\Delta x$, $x_{i+1} = \Delta x$. The basis polynomials become

$$\ell_{i-1}(x) = \frac{x(x - \Delta x)}{2\Delta x^2}, \quad \ell_i(x) = \frac{\Delta x^2 - x^2}{\Delta x^2}, \quad \ell_{i+1}(x) = \frac{x(x + \Delta x)}{2\Delta x^2}. \tag{6.9}$$

The second derivative is $p''(x) = u_{i-1}\ell''_{i-1}(x) + u_i\ell''_i(x) + u_{i+1}\ell''_{i+1}(x)$. Compute

$$\begin{gathered}\ell''_{i-1}(x) = \frac{d^2}{dx^2}\left(\frac{x^2 - x\Delta x}{2\Delta x^2}\right) = \frac{1}{\Delta x^2}, \qquad \ell''_i(x) = \frac{d^2}{dx^2}\left(1 - \frac{x^2}{\Delta x^2}\right) = -\frac{2}{\Delta x^2} \\ \ell''_{i+1}(x) = \frac{d^2}{dx^2}\left(\frac{x^2 + x\Delta x}{2\Delta x^2}\right) = \frac{1}{\Delta x^2}.\end{gathered} \tag{6.10}$$

At $x = x_i$ (i.e., $x = 0$),

$$p''(0) = u_{i-1} \cdot \frac{1}{\Delta x^2} + u_i \cdot \left(-\frac{2}{\Delta x^2}\right) + u_{i+1} \cdot \frac{1}{\Delta x^2} = \frac{u_{i-1} - 2u_i + u_{i+1}}{\Delta x^2}. \tag{6.11}$$

matching the centered stencil (6.6) we shown earlier.

6.1.2 Temporal Discretization

From the coupled system (6.5), we can define the state vector $\mathbf{U}$ and the physical vector $\mathbf{F}$ as:

$$\mathbf{U} = \begin{pmatrix} u \\ v \end{pmatrix}, \qquad \mathbf{F} = \begin{pmatrix} v \\ c^2 \frac{\partial^2 u}{\partial x^2} \end{pmatrix}. \tag{6.12}$$

where u and v are the displacement and velocity fields, respectively. The update rule for the state vector can then be expressed as:

$$\mathbf{U}^{n+1} = \mathbf{U}^n + \Delta t\, \mathbf{F}^n, \tag{6.13}$$

This approximation is known as the explicit Euler method. Temporal discretization breaks time into steps (Δt) to approximate the continuous evolution of the system. This process starts by approximating the time derivative of the state vector. Specifically, we use the forward difference

$$\frac{\partial \mathbf{U}}{\partial t} \approx \frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{\Delta t}, \qquad \text{and setting it equal to } \mathbf{F}^n \tag{6.14}$$

The explicit Euler method is a first-order accurate time-stepping method. It requires a small Δt for stability and can be adapted to various PDEs with spatial discretization.

This formulation allows us to decouple the physics of the problem from the numerical method used to solve it. The state vector $\mathbf{U}$ contains all relevant information about the system, while the physical vector $\mathbf{F}$ defines the dynamics based on the underlying PDE.

The update rule can be implemented using various numerical methods, such as explicit Euler, implicit Euler, or higher-order Runge-Kutta methods. The choice of method will depend on the specific requirements of the problem, such as stability, accuracy, and computational efficiency.

6.2 Code Implementation

We evolve the solver through four versions in Julia, each building modularity while solving the same problem (explicit finite differences). Code is in `code/state_formulation`. This progression introduces the state vector concept basically, from explicit loops to abstract dynamics, preparing for global interpolation in the next chapter.

6.2.1 Explicit Implementation

This is the most basic and transparent implementation of the wave equation solver.

The update equations for u (displacement) and v (velocity) are written explicitly. No modularity or separation of logic (everything is defined inline). There is no use of state vector formulation or external functions.

Initial conditions (Gaussian displacement, zero velocity):

```
for i in 0:Nx
    x = i * dx
    u[i, 0] = A * exp(-((x - x_c)^2) / (2 * sigma^2))
    v[i, 0] = 0.0
end
```

Time integration with Dirichlet boundaries:

```
for n in 0:Nt-1
    u[0, n] = 0.0; u[Nx, n] = 0.0
    v[0, n] = 0.0; v[Nx, n] = 0.0

    for i in 1:Nx-1
        u[i, n+1] = u[i, n] + dt * v[i, n]
        v[i, n+1] = v[i, n] + dt * c^2 * (u[i+1, n] - 2u[i, n] + u[i-1, n]) / dx^2
    end
end
```

6.2.2 Modular Implementation

In this version, we introduce reusable functions to compute the discrete Laplacian, apply boundary conditions, and set the initial state.

Initial conditions:

```
function apply_initial_conditions(u, v, Nx, dx, A, sigma, x_c)
    for i in 0:Nx
        x = i * dx
        u[i, 0] = A * exp(-((x - x_c)^2) / (2 * sigma^2))
        v[i, 0] = 0.0
    end
    return u, v
end
```

Boundaries:

```
function apply_dirichlet(u, v, n, Nx)
    u[0, n] = 0.0
    u[Nx, n] = 0.0
    v[0, n] = 0.0
    v[Nx, n] = 0.0
    return u, v
end
```

Laplacian (second derivative):

```
function laplacian_1D(u, Nx, dx)
    L = OffsetArray(zeros(Nx + 1), 0:Nx)
    for i in 1:Nx-1
        L[i] = (u[i+1] - 2u[i] + u[i-1]) / dx^2
    end
    return L
end
```

First derivative (for energy):

```
function derivative_1D(u, Nx, dx)
    dudx = OffsetArray(zeros(Nx + 1), 0:Nx)
    for i in 1:Nx-1
        dudx[i] = (u[i+1] - u[i-1]) / (2 * dx)
    end
    return dudx
end
```

Time loop:

```
u, v = apply_initial_conditions(u, v, Nx, dx, A, sigma, x_c)

for n in 0:Nt-1
    u, v = apply_dirichlet(u, v, n, Nx)
    L = laplacian_1D(u[:, n], Nx, dx)

    for i in 1:Nx-1
        u[i, n+1] = u[i, n] + dt * v[i, n]
        v[i, n+1] = v[i, n] + dt * c^2 * L[i]
    end
end
```

Note: The current 3-point stencil (using u_{i-1}, u_i, u_{i+1}) approximates $\partial^2 u/\partial x^2$ with second-order accuracy. Extend to 5-point (u_{i-2} to u_{i+2}) for fourth-order, or more.

In the next chapter advances to global interpolation, building the precomputed matrices $\mathbf{D}_0$ (interpolation), $\mathbf{D}_1$ (first derivative), $\mathbf{D}_2$ (second derivative), with Lagrange polynomials. But for this chapter, we stick to the 3-point stencil.

6.2.3 State Vector Implementation

In this version, we introduce a structural change that brings us closer to general-purpose numerical solvers. Instead of working with two separate arrays—one for the displacement u and one for the velocity v, we combine both into a single object: the **state vector** U.

We define U as a two-dimensional array that stores the full state of the system at each time step. Its rows are split into two regions:

- $U[0 : Nx, :]$ contains the displacement u,
- $U[Nx + 1 : 2Nx + 1, :]$ contains the velocity v.

To keep the familiar notation and make the code readable, we create `views` over U:

```
N_total = 2 * (Nx + 1)
U = OffsetArray(zeros(N_total, Nt+1), 0:(N_total-1), 0:Nt)
u = OffsetArray(@view(U[0:Nx, :]), (0:Nx, 0:Nt))
v = OffsetArray(@view(U[Nx+1:2Nx+1, :]), (Nx+1:2Nx+1, 0:Nt))
```

From this point on, the entire system evolves through U, but we can still refer to u and v as if they were separate fields. This improves clarity without sacrificing structure.

The boundary conditions must also reflect this new layout:

```
function apply_dirichlet(u, v, n, Nx)
    u[0, n] = 0.0
    u[Nx, n] = 0.0
    v[Nx+1, n] = 0.0
    v[2Nx+1, n] = 0.0
    return u, v
end
```

Finally, the time-stepping loop is modified to operate on the combined state. Since v is now in the second half of U, we must apply an index shift of `Nx+1` when accessing its values:

```
for n in 0:Nt-1
    u, v = apply_dirichlet(u, v, n, Nx)
    L = laplacian_1D(u[:, n], Nx, dx)

    for i in 1:Nx-1
        u[i, n+1] = u[i, n] + dt * v[i + (Nx+1), n]
        v[i + (Nx+1), n+1] = v[i + (Nx+1), n] + dt * c^2 * L[i]
    end
end
```

This version maintains the same numerical method as before, but rewrites the solver using a more general and scalable data structure. By adopting a state vector, we open the door to more elegant and powerful formulations.

6.2.4 Physical Vector Implementation

In this final version, we fully adopt the dynamical system structure introduced at the beginning of the chapter. We compute an explicit physical vector F at each time step, which contains the time derivatives of all components of the state. The update rule becomes:

$$\mathbf{U}^{n+1} = \mathbf{U}^n + \Delta t\, \mathbf{F}^n$$

This approximation, as we already noted, is known as the explicit Euler method.

We use the same state vector layout and each time step, we assemble the physical vector F:

```
for n in 0:Nt-1
    u, v = apply_dirichlet(u, v, n, Nx)
    L = laplacian_1D(u[:, n], Nx, dx)

    @views F[0:Nx] = v[Nx+1:2Nx+1, n]
    @views F[Nx+1:2Nx+1] = c^2 * L

    @views U[:, n+1] = U[:, n] + dt * F
end
```

The physical vector F now explicitly represents the right-hand side of the differential equation. Its upper half stores the velocity v, while the lower half stores the acceleration $c^2 \cdot \nabla^2 u$.

This structure is clean, extensible, and fully decouples the physics (stored in F) from the time-stepping logic. It is a natural starting point for implementing more sophisticated physical models.

Chapter 7

Global Interpolation

7.1 Introduction to Global Interpolation

In numerical methods for partial differential equations (PDEs) such as the wave equation, the accurate approximation of spatial derivatives (e.g., $\partial u/\partial x$, $\partial^2 u/\partial x^2$) on discrete grids is fundamental to solution quality and computational efficiency.

In the previous chapter, we employed explicit 3-node finite difference stencils to approximate derivatives locally. While these stencils are computationally inexpensive and straightforward to implement, they impose several significant limitations like low-order accuracy and rigid grid requirements.

This chapter introduces a fundamentally different approach through **global interpolation**, implemented via differentiation matrices $\mathbf{D}_0$, $\mathbf{D}_1$, and $\mathbf{D}_2$. Rather than using fixed local stencils, these matrices are constructed using Lagrange basis polynomials that can incorporate information from all grid points simultaneously, hence the term "global."

The key computational advantage of this approach aligns directly with our thesis theme: instead of performing multiple small matrix-vector operations (one per grid point), we can compute all spatial derivatives through a single, large matrix-vector multiplication (in 1D). In 2D this becomes even more pronounced as it is a matrix-matrix multiplication.

The matrices are constructed once at initialization using the Lagrange polynomial algorithm (`lagrange.jl`), which efficiently computes basis polynomials $L_j(x)$ and their derivatives $L'_j(x)$, $L''_j(x)$ for any desired accuracy order.

We demonstrate the approach using 1D examples, with straightforward extension to multi-dimensional problems. This foundation prepares our solvers for the matrix-matrix formulations that will be crucial for achieving optimal computational performance in the subsequent wave equation implementations.

7.2 Lagrange Polynomials

Lagrange polynomials form the basis for global interpolation. For a set of nodes $\{x_k\}_{k=0}^{N_x}$, the Lagrange basis polynomial $L_j(x)$ satisfies $L_j(x_k) = \delta_{jk}$. The interpolated function is:

$$u(x) \approx \sum_{j=0}^{N_x} u(x_j) L_j(x). \tag{7.1}$$

Where the standard product form for $L_j(x)$ is:

$$L_j(x) = \prod_{k=0, k\neq j}^{N_x} \frac{x - x_k}{x_j - x_k}. \tag{7.2}$$

We use a iterative formulation that builds $L_j(x)$, $L_j'(x)$, and $L_j''(x)$ incrementally by adding one node at a time to the polynomial from the first node up to N_x.

This starts with a base case with no nodes (q=0): $L = 1$, $L' = 0$, $L'' = 0$.

For each subsequent node x_k (with $k \neq j$), we update:

$$f(x) = \frac{x - x_k}{x_j - x_k}, \quad f'(x) = \frac{1}{x_j - x_k}, \quad f''(x) = 0. \tag{7.3}$$

Then, applying the product rule to incorporate this factor into the current partial polynomial $L_m(x)$ (built from the first m nodes), we can build the polynomial $L(x)$ with $m + 1$ nodes:

$$L(x) = L_m(x) \cdot f(x), \tag{7.4}$$

$$L'(x) = L_m'(x) \cdot f(x) + L_m(x) \cdot f'(x), \tag{7.5}$$

$$L''(x) = L_m''(x) \cdot f(x) + 2L_m'(x) \cdot f'(x) + L_m(x) \cdot f''(x). \tag{7.6}$$

This process is repeated for each node $x_k \neq x_j$, building up to the full polynomial. When $k = j$, the factor is skipped (effectively $f = 1$, $f' = 0$, $f'' = 0$), ensuring $L_j(x_j) = 1$.

Mathematically, let $L^{(q)}(x)$ be the partial Lagrange polynomial after including q nodes (excluding x_j itself). The recursion is:

$$L^{(0)}(x) = 1, \quad L^{(0)'}(x) = 0, \quad L^{(0)''}(x) = 0 \tag{7.7}$$

$$L^{(q)}(x) = L^{(q-1)}(x) \cdot f_q(x) \tag{7.8}$$

$$L^{(q)'}(x) = L^{(q-1)'}(x) \cdot f_q(x) + L^{(q-1)}(x) \cdot f_q'(x) \tag{7.9}$$

$$L^{(q)''}(x) = L^{(q-1)''}(x) \cdot f_q(x) + 2L^{(q-1)'}(x) \cdot f_q'(x) + L^{(q-1)}(x) \cdot f_q''(x) \tag{7.10}$$

where $f_q(x) = \frac{x - x_{idx_q}}{x_j - x_{idx_q}}$ for the q-th node index $idx_q \neq j$, $\quad f_q'(x) = \frac{1}{x_j - x_{idx_q}}$, $\quad$ and $f_q''(x) = 0$.

This avoids computing large products directly by accumulating derivatives step-by-step.

Code implementation

The following Julia code implements an iterative method to compute the Lagrange basis polynomial $L_j(x)$ and its first and second derivatives ($L'_j(x)$, $L''_j(x)$) at a given point x, using a set of nodes provided in an `OffsetVector` for zero-based indexing.

```
using OffsetArrays

function lagrange_derivatives(xnodes::OffsetVector{T,Vector{T}}, j::Int, x::T) where T<:Real
    return iterative_lagrange(xnodes, j, x, length(xnodes))
end

function iterative_lagrange(xnodes::OffsetVector{T,Vector{T}}, j::Int, x::T, q::Int) where T<:Real
    L, dL, ddL = T(1.0), T(0.0), T(0.0)
    keys = collect(axes(xnodes, 1))
    for k in 1:q
        idx = keys[k]
        if idx == j
            continue
        end
        denom = xnodes[j] - xnodes[idx]
        factor = (x - xnodes[idx]) / denom
        dfactor = T(1.0) / denom
        ddfactor = T(0.0)

        new_L = L * factor
        new_dL = dL * factor + L * dfactor
        new_ddL = ddL * factor + 2 * dL * dfactor + L * ddfactor
        L, dL, ddL = new_L, new_dL, new_ddL
    end
    return (L, dL, ddL)
end

function evalL(xnodes::OffsetVector{T,Vector{T}}, j::Int, x::T; order::Int=0) where T<:Real
    L, dL, ddL = lagrange_derivatives(xnodes, j, x)
    return order == 0 ? L : order == 1 ? dL : ddL
end
```

The function `iterative_lagrange` iteratively constructs the Lagrange basis polynomial $L_j(x)$, along with its first and second derivatives.

The iteration begins with an initial product of 1 (with derivatives set to 0) and processes each node sequentially, applying the product rule to update the polynomial value and its derivatives. By using an iterative approach, the implementation avoids the overhead of recursive function calls, improving performance for large node sets.

The function `evalL` provides a convenient interface to evaluate $L_j(x)$ or its derivatives at a specific point x, based on the specified order.

7.3 Interpolation Matrix $\mathbf{D}_0$

Considering $\{x_j\}_{j=0}^{M-1}$ as the original (computational) nodes and $\{x_k^{\text{interp}}\}_{k=0}^{N-1}$ as the new evaluation points (e.g., a finer grid for visualization). The interpolation matrix $\mathbf{D}_0 \in \mathbb{R}^{N\times M}$ is defined so that the interpolated vector $\mathbf{u}_{\text{interp}} \in \mathbb{R}^N$ (values of u at the x_k^{interp}) is obtained as

$$\mathbf{u}_{\text{interp}} \;=\; \mathbf{D}_0\,\mathbf{u}, \qquad \mathbf{u} = \Big(u(x_0), \ldots, u(x_{M-1})\Big)^\top. \tag{7.11}$$

Mathematical derivation

For each x_k^{interp} choose a local stencil of q nodes

$$S_k \subset \{0, \ldots, M-1\}, \qquad |S_k| = q,$$

e.g., the q contiguous nodes centered at the node nearest to x_k^{interp} (clamped at the boundaries). On that stencil, define the Lagrange polynomials

$$\ell_j^{(S_k)}(x) = \prod_{\substack{m\in S_k \\ m\neq j}} \frac{x - x_m}{x_j - x_m}, \qquad j \in S_k, \qquad \ell_j^{(S_k)}(x) = 0 \text{ if } j \notin S_k. \tag{7.12}$$

The k-th row of $\mathbf{D}_0$ consists of the interpolation coefficients evaluated at $x = x_k^{\text{interp}}$:

$$\boxed{\mathbf{D}_0[k, j] \;=\; \ell_j^{(S_k)}(x_k^{\text{interp}}), \qquad j = 0, \ldots, M-1} \tag{7.13}$$

Then, for any nodal vector $\mathbf{u}$,

$$(\mathbf{D}_0\mathbf{u})_k \;=\; \sum_{j=0}^{M-1} \mathbf{D}_0[k, j]\, u(x_j) \;=\; \sum_{j\in S_k} \ell_j^{(S_k)}(x_k^{\text{interp}})\, u(x_j), \tag{7.14}$$

which is exactly the $(q-1)$th degree Lagrange interpolant at x_k^{interp} built on stencil S_k.

Global interpolation case

In the global case we use the same stencil for every row:

$$S_k \equiv \{0, \ldots, M-1\} \qquad (\text{i.e., } q = M, \text{ independent of } k).$$

Let $L_j(x)$ be the global Lagrange basis (note that we used $\ell_j^{(S_k)}(x)$ when discussing local stencils and now referring to the global case with $L_j(x)$) on all M nodes,

$$L_j(x) \;=\; \prod_{\substack{m=0 \\ m\neq j}}^{M-1} \frac{x - x_m}{x_j - x_m}, \qquad j = 0, \ldots, M-1, \tag{7.15}$$

Then every row of $\mathbf{D}_0$ is just the evaluation of these global basis polynomials at x_k^{interp}:

$$\boxed{\mathbf{D}_0[k, j] \;=\; L_j(x_k^{\text{interp}}), \quad k = 0, \ldots, N-1, \quad j = 0, \ldots, M-1} \tag{7.16}$$

This $L_j(x)$ can be computed using the iterative Lagrange formulation we explained in (7.8).

Code implementation

In the math explanation above we use $X = \{x_j\}_{j=0}^{M-1}$ (original/computational nodes) and $X^{\text{interp}} = \{x_k^{\text{interp}}\}_{k=0}^{N-1}$ (evaluation points). In the code, these appear as:

$$x_j \leftrightarrow \texttt{xnodes[j]}, \qquad x_k^{\text{interp}} \leftrightarrow \texttt{xinterp[i+1]}.$$

We choose a stencil S_k of size $\texttt{stencil\_size} = q$ with `select_stencil`, and we fill the row k of $\mathbf{D}_0$ with the values $\ell_j^{(S_k)}(x_k^{\text{interp}})$ returned by `iterative_lagrange` explained above:

$$\mathbf{D}_0[k, j] \;=\; \ell_j^{(S_k)}(x_k^{\text{interp}}) \qquad \text{(Eq. (7.13))}$$

```
function select_stencil(i::Int, M::Int, q::Int)
    half = div(q, 2)
    imin = clamp(i - half, 0, M - q)
    imax = imin + q - 1
    return imin:imax
end
# xinterp ≠xnodes
function build_D0_matrix(xnodes::OffsetVector{T,Vector{T}},
                         stencil_size::Int,
                         xinterp::Vector{T}) where T<:Real
    M = length(xnodes) # Number of nodes.
    N = length(xinterp) # Number of evaluation points.
    D0 = OffsetArray(zeros(T, N, M), 0:N-1, 0:M-1) # Initialize matrix.

    for i in 0:N-1
        x = xinterp[i + 1] # xinterp is 1-based.
        idx_nearest = findmin(abs.(xnodes .- x))[2]
        stencil_range = select_stencil(idx_nearest, M, stencil_size)
        xstencil = OffsetArray(collect(xnodes[stencil_range]), stencil_range)

        for j in stencil_range
            q = length(xstencil)
            lj, _, _ = iterative_lagrange(xstencil, j, x, q)
            D0[i, j] = lj # Set the basis weight.
        end
    end
    return D0
end
# xinterp = xnodes
function build_D0_matrix(xnodes::OffsetVector{T,Vector{T}},
                         stencil_size::Int) where T<:Real
    build_D0_matrix(xnodes, stencil_size, collect(xnodes))
end
```

Each loop over k builds the k-th row by evaluating the local Lagrange basis on the stencil S_k at x_k^{interp}, exactly implementing $\mathbf{D}_0[k, j] = \ell_j^{(S_k)}(x_k^{\text{interp}})$. Thus $\mathbf{D}_0\mathbf{u}$ produces the $(q-1)$th degree interpolant at all x_k^{interp}.

With $\texttt{stencil\_size} = M$, it becomes the global $(M-1)$th degree interpolant.

7.4 First Derivative Matrix $\mathbf{D}_1$

Let $X = \{x_j\}_{j=0}^{M-1}$ be the computational nodes. The first derivative matrix $\mathbf{D}_1 \in \mathbb{R}^{M\times M}$ maps nodal samples $\mathbf{u} = (u(x_0), \ldots, u(x_{M-1}))^\top$ to an approximation of the spatial derivative u_x evaluated at the nodes:

$$(\mathbf{D}_1\mathbf{u})_i \;\approx\; \left.\frac{\partial u}{\partial x}\right|_{x=x_i}, \qquad i = 0, \ldots, M-1. \tag{7.17}$$

Mathematical derivation

For each target node x_i choose a local stencil of q nodes

$$S_i \subset \{0, \ldots, M-1\}, \qquad |S_i| = q,$$

e.g., the q contiguous indices centered at i (clamped at the boundaries).

On that stencil, define the local Lagrange basis $\{\ell_j^{(S_i)}(x)\}_{j\in S_i}$ by

$$\ell_j^{(S_i)}(x) = \prod_{\substack{m\in S_i \\ m\neq j}} \frac{x - x_m}{x_j - x_m}, \qquad j \in S_i, \qquad \ell_j^{(S_i)}(x) = 0 \text{ if } j \notin S_i, \tag{7.18}$$

The derivative at x_i is approximated by differentiating the local interpolant on S_i and evaluating at x_i:

$$\left.\frac{\partial u}{\partial x}\right|_{x=x_i} \;\approx\; \sum_{j\in S_i} \ell_j^{(S_i)\prime}(x_i)\, u(x_j). \tag{7.19}$$

Therefore, the entries of $\mathbf{D}_1$ are

$$\boxed{\mathbf{D}_1[i,j] \;=\; \ell_j^{(S_i)\prime}(x_i), \qquad i = 0, \ldots, M-1, \;\; j = 0, \ldots, M-1} \tag{7.20}$$

with $\mathbf{D}_1[i,j] = 0$ whenever $j \notin S_i$. With this definition, $(\mathbf{D}_1\mathbf{u})_i$ is exactly the derivative of the $(q-1)$th degree Lagrange interpolant on S_i evaluated at x_i.

Global interpolation case

If $S_i = \{0, \ldots, M-1\}$ for every i (i.e., $q = M$), we obtain the global first-derivative matrix:

$$\boxed{\mathbf{D}_1[i,j] \;=\; L_j'(x_i)} \tag{7.21}$$

where $L_j(x)$ are the global Lagrange polynomials on all M nodes. This polynomials derivatives can be obtained using the same iterative Lagrange formulation we explained in (7.9).

In this case of global interpolation, the $\mathbf{D}_1$ matrix is dense.

Code implementation

In the math above we use $X = \{x_j\}_{j=0}^{M-1}$ as the node set and we evaluate the first derivative at the nodes. In the code, these appear as:

$$x_j \leftrightarrow \texttt{xnodes[j]}, \qquad x_i \text{ (target node)} \leftrightarrow \texttt{xnodes[i]}.$$

For each target node x_i we choose a stencil S_i of size $\texttt{stencil\_size} = q$ with `select_stencil(i, M, q)`, build the local node array, and fill row i of $\mathbf{D}_1$ with the first derivatives of the local Lagrange basis at x_i, returned by `iterative_lagrange`.

$$\mathbf{D}_1[i, j] \;=\; \ell_j^{(S_i)\prime}(x_i) \qquad \text{(Eq. (7.20))}$$

```
function select_stencil(i::Int, M::Int, q::Int)
    half = div(q, 2)
    imin = clamp(i - half, 0, M - q)
    imax = imin + q - 1
    return imin:imax
end

function build_D1_matrix(xnodes::OffsetVector{<:Real}, stencil_size::Int, ::Type{T}=
    Float64) where T<:Real
    M = length(xnodes) # Number of nodes.
    D1 = OffsetArray(zeros(T, M, M), 0:M-1, 0:M-1) # Initialize matrix

    for i in 0:M-1
        stencil_range = select_stencil(i, M, stencil_size)
        xstencil = OffsetArray(convert.(T, collect(xnodes[stencil_range])), stencil_range)

        for j in stencil_range
            q = length(xstencil)
            _, lp, _ = iterative_lagrange(xstencil, j, T(xnodes[i]), q)
            D1[i, j] = lp # Set the differentiation weight.
        end
    end

    return D1
end
```

Setting $\texttt{stencil\_size} = M$ yields the global case $S_i = \{0, \ldots, M-1\}$ for all i.

Multiplying $\mathbf{D}_1$ by the nodal vector $\mathbf{u}$ returns the first derivative approximations at all nodes. The same selection logic for `stencil_size` controls whether the construction is local (sparse/banded) or global (dense).

7.5 Second Derivative Matrix $\mathbf{D}_2$

Let $X = \{x_j\}_{j=0}^{M-1}$ be the computational nodes. The second derivative matrix $\mathbf{D}_2 \in \mathbb{R}^{M\times M}$ maps nodal samples $\mathbf{u} = (u(x_0), \dots, u(x_{M-1}))^\top$ to an approximation of the spatial second derivative u_{xx} evaluated at the nodes:

$$(\mathbf{D}_2\mathbf{u})_i \;\approx\; \left.\frac{\partial^2 u}{\partial x^2}\right|_{x=x_i}, \qquad i = 0, \dots, M-1. \tag{7.22}$$

Mathematical derivation

For each target node x_i choose a local stencil of q nodes

$$S_i \subset \{0, \dots, M-1\}, \qquad |S_i| = q,$$

e.g., the q contiguous indices centered at i (clamped at the boundaries).

On that stencil, define the local Lagrange basis $\{\ell_j^{(S_i)}(x)\}_{j\in S_i}$ by

$$\ell_j^{(S_i)}(x) = \prod_{\substack{m\in S_i \\ m\neq j}} \frac{x - x_m}{x_j - x_m}, \qquad j \in S_i, \qquad \ell_j^{(S_i)}(x) = 0 \text{ if } j \notin S_i. \tag{7.23}$$

Differentiate the local interpolant twice and evaluate at x_i:

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{x=x_i} \;\approx\; \sum_{j\in S_i} \ell_j^{(S_i)\prime\prime}(x_i)\, u(x_j). \tag{7.24}$$

Therefore, the entries of $\mathbf{D}_2$ are

$$\boxed{\mathbf{D}_2[i,j] \;=\; \ell_j^{(S_i)\prime\prime}(x_i), \qquad i = 0, \dots, M-1, \quad j = 0, \dots, M-1} \tag{7.25}$$

with $\mathbf{D}_2[i,j] = 0$ whenever $j \notin S_i$. With this definition, $(\mathbf{D}_2\mathbf{u})_i$ is exactly the second derivative of the degree-$(q-1)$ Lagrange interpolant on S_i evaluated at x_i.

Global interpolation case

If $S_i = \{0, \dots, M-1\}$ for every i (i.e., $q = M$), we obtain the global second-derivative matrix:

$$\boxed{\mathbf{D}_2[i,j] \;=\; L_j''(x_i)} \tag{7.26}$$

where $L_j(x)$ are the global Lagrange polynomials on all M nodes. The values $L_j''(x_i)$ can be computed with the iterative Lagrange formulation in (7.10).

In this case of global interpolation, $\mathbf{D}_2$ is a dense matrix, as it was for $\mathbf{D}_1$.

Code implementation

Implementation bridge. We evaluate at the nodes themselves (target x_i and source x_j are entries of `xnodes`). For each row i, we choose $S_i = \texttt{select\_stencil}(i, M, q)$, form the local node array, and fill $\mathbf{D}_2[i, j]$ with the *second derivatives* returned by `iterative_lagrange` at x_i:

$$\mathbf{D}_2[i, j] \;=\; \ell_j^{(S_i)''}(x_i) \qquad \text{(Eq. (7.25))}$$

```julia
# File: code/interpolants/D2_matrix.jl

using OffsetArrays

include("iterative_lagrange.jl") # Your module with lagrange_derivatives

function build_D2_matrix(xnodes::OffsetVector{<:Real}, stencil_size::Int, ::Type{T}=
    Float64) where T<:Real
    M = length(xnodes) # Number of nodes.
    D2 = OffsetArray(zeros(T, M, M), 0:M-1, 0:M-1) # Initialize matrix with type T.

    for i in 0:M-1
        # Select stencil centered at i.
        stencil_range = select_stencil(i, M, stencil_size)
        # Create a sub-OffsetArray for the stencil nodes, converting to T.
        xstencil = OffsetArray(convert.(T, collect(xnodes[stencil_range])), stencil_range)

        for j in stencil_range
            q = length(xstencil) # Actual stencil size.
            _, _, lpp = iterative_lagrange(xstencil, j, T(xnodes[i]), q)
            D2[i, j] = lpp # Set the differentiation weight.
        end
    end

    return D2
end
```

Setting `stencil_size` $= M$ yields the global case $S_i = \{0, \ldots, M-1\}$ for all i.

Multiplying $\mathbf{D}_2$ by the nodal vector $\mathbf{u}$ returns the second derivative approximations at all nodes. The same selection logic for the local stencils applies here, ensuring that each node's second derivative is computed using only its neighbors. The choice of `stencil_size` controls again whether the construction is local or global.

Chapter 8

Wave Equation in 1D and 2D with Global Interpolation

8.1 Wave Equation 1D (Matrix–Vector)

8.1.1 Mathematical model

The one-dimensional wave equation can be expressed as:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}, \qquad x \in [0, L],\ t \geq 0, \tag{8.1}$$

where $u(x,t)$ is the displacement at position x and time t, and c is the wave propagation speed. The initial condition and boundary conditions (homogeneous Dirichlet) are specified as:

$$u(x,0) = A\ \exp\left(-\frac{(x-x_c)^2}{2\sigma^2}\right), \qquad v(x,0) = 0, \tag{8.2}$$

$$u(0,t) = u(L,t) = 0, \qquad v(0,t) = v(L,t) = 0, \qquad t \geq 0. \tag{8.3}$$

where A is the amplitude, x_c is the center position, and σ is the width of the initial Gaussian pulse, and $v(x,t) = \frac{\partial u}{\partial t}$ is the velocity field.

The wave equation can be reformulated as a first-order system by introducing the auxiliary variable $v(x,t) = \frac{\partial u}{\partial t}$:

$$\begin{aligned} \frac{\partial u}{\partial t} &= v, \\ \frac{\partial v}{\partial t} &= c^2 \frac{\partial^2 u}{\partial x^2}. \end{aligned} \tag{8.4}$$

Spatial discretization (global interpolation)

We discretize the spatial domain by selecting a set of collocation nodes $X = \{x_i\}_{i=0}^{N_x} \subset [0, L]$. The nodes may be uniform or nonuniform (e.g., Chebyshev–Lobatto).

The second derivative at the nodes is represented by the dense matrix $\mathbf{D}_2^x \in \mathbb{R}^{(N_x+1)\times(N_x+1)}$ whose entries are the second derivatives of the basis functions evaluated at the nodes:

$$\mathbf{D}_2^x[i, j] \;=\; L_j''(x_i), \qquad i, j = 0, \ldots, N_x \tag{8.5}$$

i.e., the global case of Eq. (7.26).

Given the nodal vector $\mathbf{u}(t) = \Big(u(x_0, t), \ldots, u(x_{N_x}, t)\Big)^\top$, the second derivative is approximated at the nodes by the matrix–vector product

$$u_{xx}(\cdot, t)\Big|_X \;\approx\; \mathbf{D}_2^x\, \mathbf{u}(t). \tag{8.6}$$

Temporal discretization (explicit Euler)

Writing the first–order system $u_t = v, \quad v_t = c^2\, u_{xx}$ and inserting the semidiscrete operator (8.6) gives the ODE system for the nodal unknowns:

$$\frac{d\mathbf{u}}{dt} \;=\; \mathbf{v}, \qquad \frac{d\mathbf{v}}{dt} \;=\; c^2\, \mathbf{D}_2^x\, \mathbf{u}. \tag{8.7}$$

Collect the state at time level t^n as the stacked vector

$$\mathbf{U}^n = \begin{pmatrix} \mathbf{u}^n \\ \mathbf{v}^n \end{pmatrix} \in \mathbb{R}^{2(N_x+1)}, \qquad \mathbf{F}^n = \begin{pmatrix} \mathbf{v}^n \\ c^2\, \mathbf{D}_2^x\, \mathbf{u}^n \end{pmatrix},$$

so that a forward Euler step with time step Δt reads

$$\mathbf{U}^{n+1} \;=\; \mathbf{U}^n \;+\; \Delta t\, \mathbf{F}^n. \tag{8.8}$$

This is followed immediately by the strong enforcement of the Dirichlet constraints at the two boundary nodes for both u and v.

The time step Δt must satisfy a CFL–like restriction to ensure stability of the explicit advance. This will be enforced by adjusting the time step based on the spatial grid size and wave speed.

8.1.2 Code implementation

The 1D wave solver stores the state in a rank-3 tensor `U` with shape $(N_x+1,\ 2,\ N_t+1)$:

- First dimension: spatial nodes x_i, $i = 0, \ldots, N_x$.
- Second dimension: fields (index $0 \to$ displacement u, index $1 \to$ velocity v).
- Third dimension: time levels $n = 0, \ldots, N_t$.

The Laplacian in 1D is applied by a single matrix–vector product with the global second-derivative matrix $\mathbf{D}_2^x$ from Eq. (8.5):

```
function laplacian_1D(U::OffsetArray{T}, D2x::OffsetArray{T}) where {T}
    return D2x * U[:, 0]
end
```

Here, `U[:, 0]` extracts the displacement vector $\mathbf{u}^n \in \mathbb{R}^{N_x+1}$ at the current time step, and `D2x * U[:, 0]` returns an approximation of $\partial^2 u/\partial x^2$ at all nodes (cf. Eq. (8.5)).

The explicit Euler advance follows the block form in Eq. (8.8):

```
apply_dirichlet!(U, n, Nx) # enforce homogeneous BCs at t^n
L = laplacian_1D(U[:, :, n], D2x)

@views begin
    F[:, 0] .= U[:, 1, n] # RHS for u_t = v
    F[:, 1] .= c^2 .* L # RHS for v_t = c^2 u_xx
    U[:, :, n+1] .= U[:, :, n] .+ dt .* F
end

apply_dirichlet!(U, n+1, Nx) # enforce BCs at t^(n+1)
```

The working array `F` has shape $(N_x+1, 2)$ and stores the right-hand side $\left[\mathbf{v}^n,\ c^2\mathbf{D}_2^x\mathbf{u}^n\right]$. Boundary conditions are imposed strongly by overwriting the boundary entries of u and v at each step.

The differentiation matrix $\mathbf{D}_2^x$ is built once, at startup, via the global Lagrange construction, choosing a stencil size of $N_x + 1$:

```
D2x = OffsetArray(build_D2_matrix(xnodes, stencil, T), 0:Nx, 0:Nx)
```

Here `xnodes` holds the collocation nodes (uniform or nonuniform, e.g., Chebyshev–Lobatto). With `stencil_size = Nx+1` the operator is global and therefore dense; smaller stencils would yield a banded $\mathbf{D}_2^x$. The dominant kernel per step is the BLAS `gemv` $\mathbf{D}_2^x\mathbf{u}^n$.

8.1.3 Results

Laplacian Operator FP32 Performance

The isolated 1D Laplacian operator performance demonstrates characteristic memory-bound behavior, consistent with the matrix-vector benchmarks analyzed in Chapter 3.

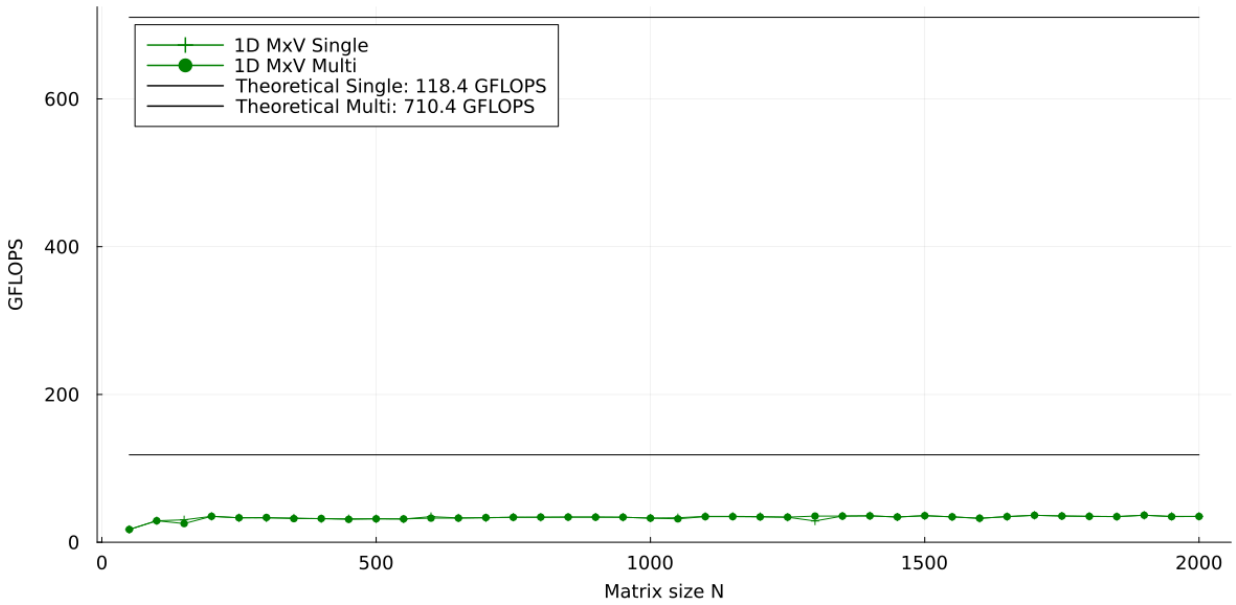

Figure 8.1: Float32 1D Laplacian (MxV): single vs. multi-threaded performance. Single-threaded performance shows steady, memory-bound behavior with gradual improvement. Multi-threaded performance closely matches single-threaded initially but exhibits dramatic degradation around mid-range sizes before recovering at larger scales.

Memory-bound plateau behavior: Both curves plateau well below compute-bound performance levels, reflecting the low arithmetic intensity inherent to matrix-vector operations. This behavior mirrors the synthetic MxV benchmarks from Chapter 3, where memory bandwidth becomes the fundamental bottleneck regardless of computational resources.

Consistent single-threaded efficiency: The stable single-threaded curve across all sizes confirms that memory bandwidth limitation dominates performance, avoiding the complications of inter-thread resource competition.

This 1D implementation validates the theoretical predictions from Chapter 3 regarding matrix-vector limitations, demonstrating that real-world PDE operators exhibit the same fundamental memory-bound characteristics as synthetic benchmarks.

1D Wave Equation (MxV) FP64 Performance

We evaluated the complete 1D wave equation solver across a range of grid sizes using **Float64 precision** and 1.0 second simulation time.

Each time step involves matrix-vector multiplication, plus additional operations for temporal integration and boundary condition enforcement. The total floating-point operations per time step can be approximated as $2 \times N_x^2$ (dominated by the MxV operation):

$$\text{Total Operations} = \text{Time Steps} \times \text{Operations per Step} \approx N_t \times 2 \times N_x^2 \tag{8.9}$$

We used an adaptive time step of $dt = 10^{-4} \times \frac{1}{N_x}$ seconds for stability control. This formulation ensures that finer grids use proportionally smaller time steps.

Grid Size	**Time (s)**	**Total Operations**	**GFLOPS**
26 nodes	0.10	3.52×10^8	3.52
51 nodes	0.35	2.65×10^9	7.58
76 nodes	0.90	8.78×10^9	9.76
101 nodes	1.95	2.06×10^{10}	10.57
126 nodes	3.66	4.00×10^{10}	10.93
151 nodes	6.05	6.89×10^{10}	11.38
176 nodes	8.91	1.09×10^{11}	12.24
201 nodes	13.47	1.62×10^{11}	12.06

The performance progression shows consistent scaling behavior across all grid sizes. For small grids (26-76 nodes), we observe steady efficiency improvements as matrix operations begin to dominate computational overhead. The medium grids (101-151 nodes) show continued linear scaling reaching peak efficiency around 11.3 GFLOPS. Finally, large grids (176-201 nodes) exhibit a performance plateau near 12.1 GFLOPS, indicating optimal throughput for this precision.

These results align closely with the Float64 matrix-vector multiplication performance characteristics observed in Chapter 3, confirming that the wave equation solver achieves the expected computational efficiency for dense linear algebra operations. The final performance of approximately 12 GFLOPS demonstrates that the full solver maintains the same computational characteristics as the isolated matrix-vector benchmarks.

Physical 1D Wave Evolution (MxV) - Original Nodes Representation

The solver captures correct wave physics across all stable configurations. Using N=25 nodes with 26-node stencil demonstrates complete wave propagation behavior:

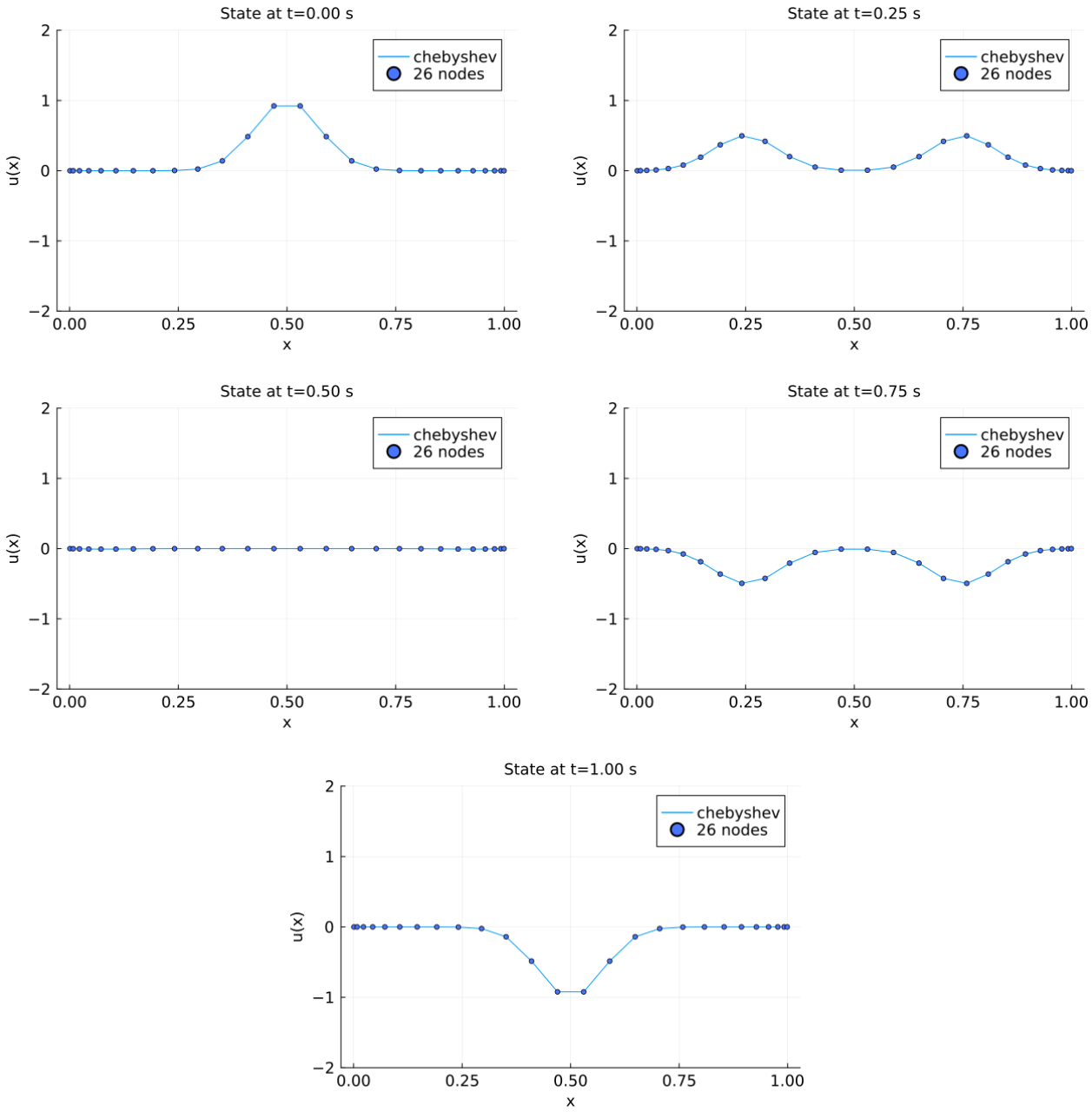

Figure 8.2: 1D Wave Equation Evolution. The temporal sequence shows: (top left) initial Gaussian pulse centered at x = 0.5, (top right) wave splitting into counter-propagating components, (middle left) reflection at Dirichlet boundaries, (middle right) wave interference during convergence, and (bottom) final standing wave configuration. Blue circles indicate the 26 Chebyshev node positions where the discrete solution is computed.

Physical 1D Wave Evolution (MxV) - Interpolated Representation

The Chebyshev node distribution enables high-quality Lagrange interpolation to reconstruct the continuous wave field. The solution is interpolated from 26 Chebyshev nodes to 100 equispaced points:

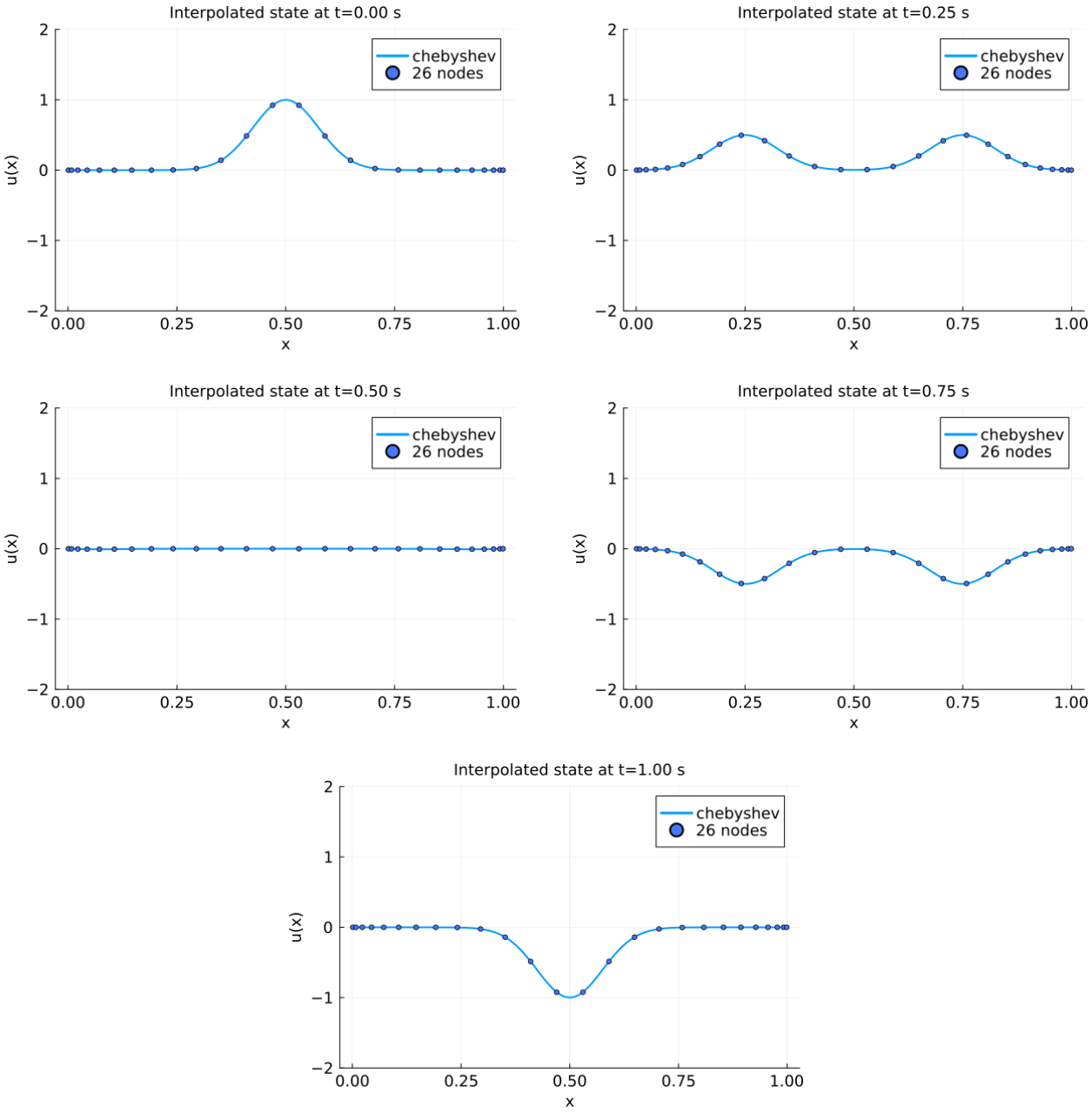

Figure 8.3: 1D Wave Equation Evolution (Fine Grid). The continuous curves demonstrate the high accuracy of Lagrange interpolation from the optimal Chebyshev node distribution (blue circles) to a fine equispaced grid of 100 points.

8.2 Wave Equation 2D (Matrix–Matrix)

8.2.1 Mathematical model

The two-dimensional wave equation can be expressed as:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \qquad (x, y) \in [0, L_x] \times [0, L_y],\ t \geq 0, \tag{8.10}$$

where $u(x, y, t)$ is the displacement at position (x, y) and time t, and c is the wave propagation speed. The initial condition and boundary conditions (homogeneous Dirichlet) are specified as:

$$u(x, y, 0) = A \exp\left(-\frac{(x - x_c)^2 + (y - y_c)^2}{2\sigma^2} \right), \qquad v(x, y, 0) = 0, \tag{8.11}$$

$$u(0, y, t) = u(L_x, y, t) = u(x, 0, t) = u(x, L_y, t) = 0, \tag{8.12}$$

$$v(0, y, t) = v(L_x, y, t) = v(x, 0, t) = v(x, L_y, t) = 0, \qquad t \geq 0. \tag{8.13}$$

where A is the amplitude, (x_c, y_c) is the center position, and σ is the width of the initial Gaussian pulse, and $v(x, y, t) = \frac{\partial u}{\partial t}$ is the velocity field.

The wave equation can be reformulated as a first-order system by introducing the auxiliary variable $v(x, y, t) = \frac{\partial u}{\partial t}$:

$$\begin{aligned} \frac{\partial u}{\partial t} &= v, \\ \frac{\partial v}{\partial t} &= c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right). \end{aligned} \tag{8.14}$$

Spatial discretization (global interpolation)

We discretize the spatial domain using a tensor-product grid by selecting collocation nodes $X^{(x)} = \{x_i\}_{i=0}^{N_x} \subset [0, L_x]$ and $X^{(y)} = \{y_j\}_{j=0}^{N_y} \subset [0, L_y]$. The nodes may be uniform or nonuniform (e.g., Chebyshev–Lobatto) in each direction.

The 2D Laplacian at grid point (x_i, y_j) splits additively: $\nabla^2 u(x_i, y_j) = u_{xx}(x_i, y_j) + u_{yy}(x_i, y_j)$.

The directional second derivatives are represented by dense matrices $\mathbf{D}_2^x \in \mathbb{R}^{(N_x+1)\times(N_x+1)}$ and $\mathbf{D}_2^y \in \mathbb{R}^{(N_y+1)\times(N_y+1)}$ whose entries are the second derivatives of the basis functions evaluated at the nodes:

$$\mathbf{D}_2^x[i, k] = L_k^{(x)\prime\prime}(x_i), \qquad i, k = 0, \ldots, N_x \tag{8.15}$$

$$\mathbf{D}_2^y[j, \ell] = L_\ell^{(y)\prime\prime}(y_j), \qquad j, \ell = 0, \ldots, N_y \tag{8.16}$$

i.e., the global case of Eq. (7.26) applied in each direction.

Given the displacement field as a matrix $\mathbf{U}(t) \in \mathbb{R}^{(N_x+1)\times(N_y+1)}$ with $\mathbf{U}[i,j](t) = u(x_i, y_j, t)$, where rows correspond to different x_i values and columns to different y_j values, the directional derivatives are computed as:

$$u_{xx}(\cdot,\cdot,t) \approx \mathbf{D}_2^x\,\mathbf{U}(t) \qquad \text{(acts on each column, fixed } y_j\text{)} \tag{8.17}$$

$$u_{yy}(\cdot,\cdot,t) \approx \mathbf{U}(t)\,(\mathbf{D}_2^y)^\top \qquad \text{(acts on each row, fixed } x_i\text{)} \tag{8.18}$$

The transpose in the y-derivative appears because $\mathbf{D}_2^y$ is designed to act on column vectors of y-direction values, but we need to apply it to each row of $\mathbf{U}$ (corresponding to fixed x_i). The matrix multiplication $\mathbf{U}\,(\mathbf{D}_2^y)^\top$ is equivalent to applying $\mathbf{D}_2^y$ to each row vector $\mathbf{U}[i,:]$ and can be understood as:

$$\Big(\mathbf{U}\,(\mathbf{D}_2^y)^\top\Big)[i,j] = \sum_{\ell=0}^{N_y} \mathbf{U}[i,\ell]\,(\mathbf{D}_2^y)^\top[\ell,j] = \sum_{\ell=0}^{N_y} \mathbf{D}_2^y[j,\ell]\,u(x_i,y_\ell)$$

which correctly computes the y-derivative at (x_i, y_j) using values along the fixed x_i line.

The 2D Laplacian is then approximated by:

$$\nabla^2 u(\cdot,\cdot,t)\Big|_{X^{(x)}\times X^{(y)}} \;\approx\; \mathbf{D}_2^x\,\mathbf{U}(t) + \mathbf{U}(t)\,(\mathbf{D}_2^y)^\top. \tag{8.19}$$

Temporal discretization (explicit Euler)

Writing the first–order system $u_t = v$, $v_t = c^2\,(u_{xx} + u_{yy})$ and inserting the semidiscrete operator (8.19) gives the ODE system for the nodal unknowns:

$$\frac{d\mathbf{U}}{dt} \;=\; \mathbf{V}, \qquad \frac{d\mathbf{V}}{dt} \;=\; c^2\,\Big(\mathbf{D}_2^x\,\mathbf{U} + \mathbf{U}\,(\mathbf{D}_2^y)^\top\Big). \tag{8.20}$$

Collect the state at time level t^n as the stacked tensor

$$\mathbf{U}^n = \begin{pmatrix} \mathbf{u}^n \\ \mathbf{v}^n \end{pmatrix} \in \mathbb{R}^{(N_x+1)\times(N_y+1)\times 2}, \qquad \mathbf{F}^n = \begin{pmatrix} \mathbf{v}^n \\ c^2\,\Big(\mathbf{D}_2^x\,\mathbf{u}^n + \mathbf{u}^n\,(\mathbf{D}_2^y)^\top\Big) \end{pmatrix},$$

so that a forward Euler step with time step Δt reads

$$\mathbf{U}^{n+1} \;=\; \mathbf{U}^n \;+\; \Delta t\,\mathbf{F}^n. \tag{8.21}$$

This is followed immediately by the strong enforcement of the Dirichlet constraints at all four boundary edges for both u and v.

In 2D, the choice of Δt must satisfy a CFL–like restriction governed by the smallest grid spacing in both directions, which ensures stability of the explicit advance.

8.2.2 Code implementation

The 2D wave solver stores the state in a rank-4 tensor `U` with shape $(N_x{+}1,\ N_y{+}1,\ 2,\ N_t{+}1)$:

- First dimension: spatial nodes x_i, $i = 0, \ldots, N_x$.
- Second dimension: spatial nodes y_j, $j = 0, \ldots, N_y$.
- Third dimension: fields (index $0 \to$ displacement u, index $1 \to$ velocity v).
- Fourth dimension: time levels $n = 0, \ldots, N_t$.

The Laplacian in 2D is applied by two matrix–matrix products with the global second-derivative matrices $\mathbf{D}_2^x$ and $\mathbf{D}_2^y$ from Eq. (8.16):

```
function laplacian_2D(U::OffsetArray{T}, D2x::OffsetArray{T}, D2yT::OffsetArray{T}) where
    {T}
    return D2x * U[:, :, 0] .+ U[:, :, 0] * D2yT
end
```

Here, `U[:, :, 0]` extracts the displacement matrix $\mathbf{u}^n \in \mathbb{R}^{(N_x+1)\times(N_y+1)}$ at the current time step. The operation `D2x * U[:, :, 0]` applies the x-direction second derivative to each column (fixed y_j), while `U[:, :, 0] * D2yT` applies the y-direction second derivative to each row (fixed x_i), returning approximations of $\partial^2 u/\partial x^2$ and $\partial^2 u/\partial y^2$ respectively (cf. Eq. (8.19)).

The explicit Euler advance follows the block form in Eq. (8.21):

```
apply_dirichlet!(U, n, Nx, Ny) # enforce homogeneous BCs at t^n
L = laplacian_2D(U[:, :, :, n], D2x, D2yT)

@views begin
    F[:, :, 0] .= U[:, :, 1, n] # RHS for u_t = v
    F[:, :, 1] .= c^2 .* L # RHS for v_t = c^2 (u_xx + u_yy)
    U[:, :, :, n+1] .= U[:, :, :, n] .+ dt .* F
end

apply_dirichlet!(U, n+1, Nx, Ny) # enforce BCs at t^(n+1)
```

The working array `F` has shape $(N_x+1, N_y+1, 2)$ and stores $\left[\mathbf{v}^n,\ c^2(\mathbf{D}_2^x\mathbf{u}^n + \mathbf{u}^n(\mathbf{D}_2^y)^\top)\right]$. Boundary conditions are imposed strongly by overwriting the boundary entries of u and v at each step on all four edges.

The differentiation matrices $\mathbf{D}_2^x$ and $\mathbf{D}_2^y$ are built once, at startup, via the global Lagrange construction, choosing stencil sizes of $N_x + 1$ and $N_y + 1$ respectively:

```
D2x = OffsetArray(build_D2_matrix(xnodes, Nx+1, T), 0:Nx, 0:Nx)
D2y = OffsetArray(build_D2_matrix(ynodes, Ny+1, T), 0:Ny, 0:Ny)
D2yT = transpose(D2y)
```

Here `xnodes` and `ynodes` hold the collocation nodes in each direction (uniform or nonuniform, e.g., Chebyshev–Lobatto). With `stencil_size = Nx+1` and `Ny+1` the operators are global and therefore dense; smaller stencils would yield banded matrices. The dominant kernels per step are the two BLAS `gemm` operations $\mathbf{D}_2^x\mathbf{u}^n$ and $\mathbf{u}^n(\mathbf{D}_2^y)^\top$.

8.2.3 Results

Laplacian Operator FP32 Performance

The 2D matrix-matrix Laplacian operator demonstrates exceptional computational efficiency, exhibiting behavior consistent with the synthetic benchmarks from Chapter 3.

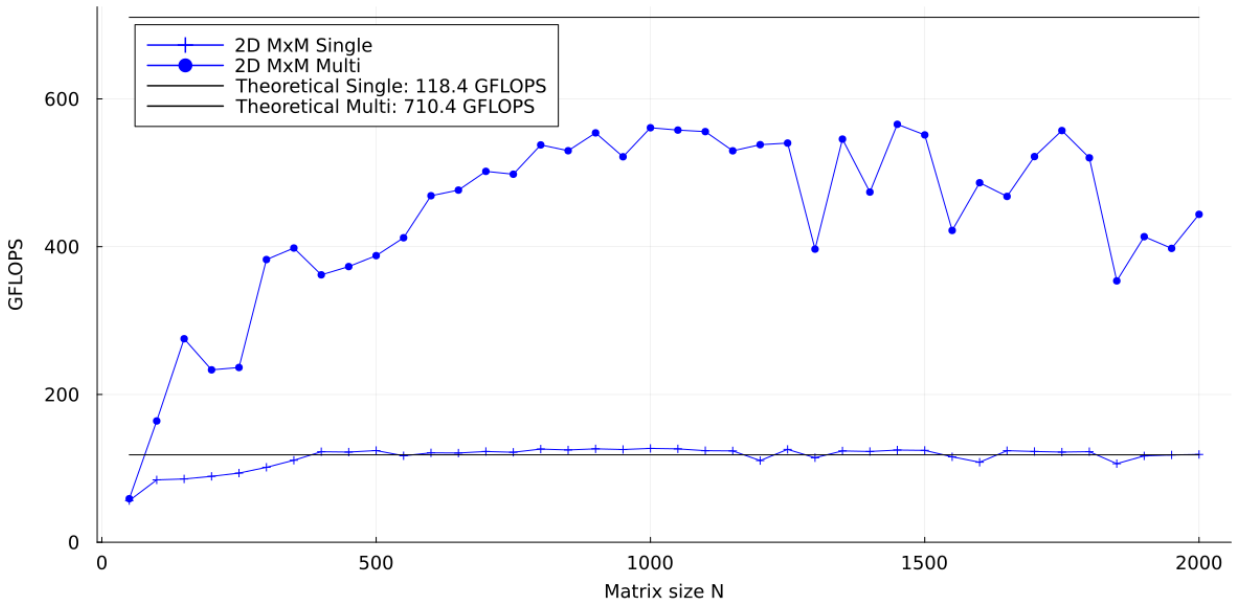

Figure 8.4: Float32 2D Laplacian (MxM): single vs. multi-threaded performance. Single-threaded performance steadily improves with matrix size, achieving theoretical performance due to turbo boost effects. Multi-threaded performance shows dramatic scaling with a sharp transition, though remaining below theoretical peaks due to shared resource contention.

Single-threaded theoretical convergence: The single-threaded curve reaches and slightly exceeds theoretical performance, mirroring Chapter 3 results. This reflects turbo boost effects where single-core workloads benefit from higher frequencies than base clock calculations assume.

Multi-threaded scaling limitations: Multi-threaded performance exhibits the characteristic sharp transition seen in synthetic benchmarks but remains below theoretical peaks. This gap reflects shared resource contention (L3 cache, memory bandwidth) and lower all-core turbo frequencies, consistent with our earlier analysis.

BLAS optimization effectiveness: The two matrix-matrix products ($\mathbf{D}_2^x\mathbf{u}$ and $\mathbf{u}(\mathbf{D}_2^y)^\top$) achieve compute-bound performance through optimized `gemm` kernels, demonstrating superior arithmetic intensity compared to matrix-vector alternatives.

The close correspondence with Chapter 3 synthetic results confirms that real-world PDE solvers can achieve predicted computational efficiency when algorithmic structure aligns with modern CPU architectures.

2D Wave Equation (MxM) FP64 Performance

We evaluated the complete 2D wave equation solver across a range of grid sizes using **Float64 precision** and 1.0 second simulation time.

Each time step involves matrix-matrix multiplication operations, plus additional operations for temporal integration and boundary condition enforcement. The total floating-point operations per time step can be approximated as $2 \times 2 \times N^3$ (dominated by the MxM operations):

$$\text{Total Operations} = \text{Time Steps} \times \text{Operations per Step} \approx N_t \times 2 \times 2 \times N^3 \tag{8.22}$$

We used an adaptive time step of $dt = 10^{-4} \times \frac{1}{N_x}$ seconds for stability control. This formulation ensures that finer grids use proportionally smaller time steps.

Grid Size	**Time (s)**	**Total Operations**	**GFLOPS**
26 nodes	6.50	1.83×10^{10}	2.81
51 nodes	22.12	2.71×10^{11}	12.23
76 nodes	58.44	1.33×10^{12}	22.84
101 nodes	118.34	4.16×10^{12}	35.17
126 nodes	207.86	1.01×10^{13}	48.50
151 nodes	398.64	2.08×10^{13}	52.17
176 nodes	546.00	3.84×10^{13}	70.29
201 nodes	919.12	6.53×10^{13}	71.03

The performance progression demonstrates a scaling behavior across all grid sizes. For small grids (26-76 nodes), we observe rapid efficiency improvements as the cubic scaling of matrix operations begins to dominate computational overhead. The medium grids (101-151 nodes) show continued strong scaling with performance reaching over 50 GFLOPS. Finally, large grids (176-201 nodes) exhibit peak performance near 70 GFLOPS.

This demonstrates the superior computational characteristics of 2D matrix-matrix operations compared to 1D matrix-vector operations. The cubic scaling of operations with grid size (N^3) enables higher arithmetic intensity, leading to significantly better GFLOPS performance.

We observe that the performance is still below of what was achieved in the synthetic benchmarks in Chapter 3, likely due to the additional overhead of boundary condition enforcement and temporal integration steps that are not present in isolated matrix-matrix multiplication tests.

Physical 2D Wave Evolution (MxM) - Original Nodes Representation

The 2D solver captures complete wave physics on Chebyshev node grids. Using 25×25 nodes with 26-node stencils demonstrates radial wave propagation and boundary interactions:

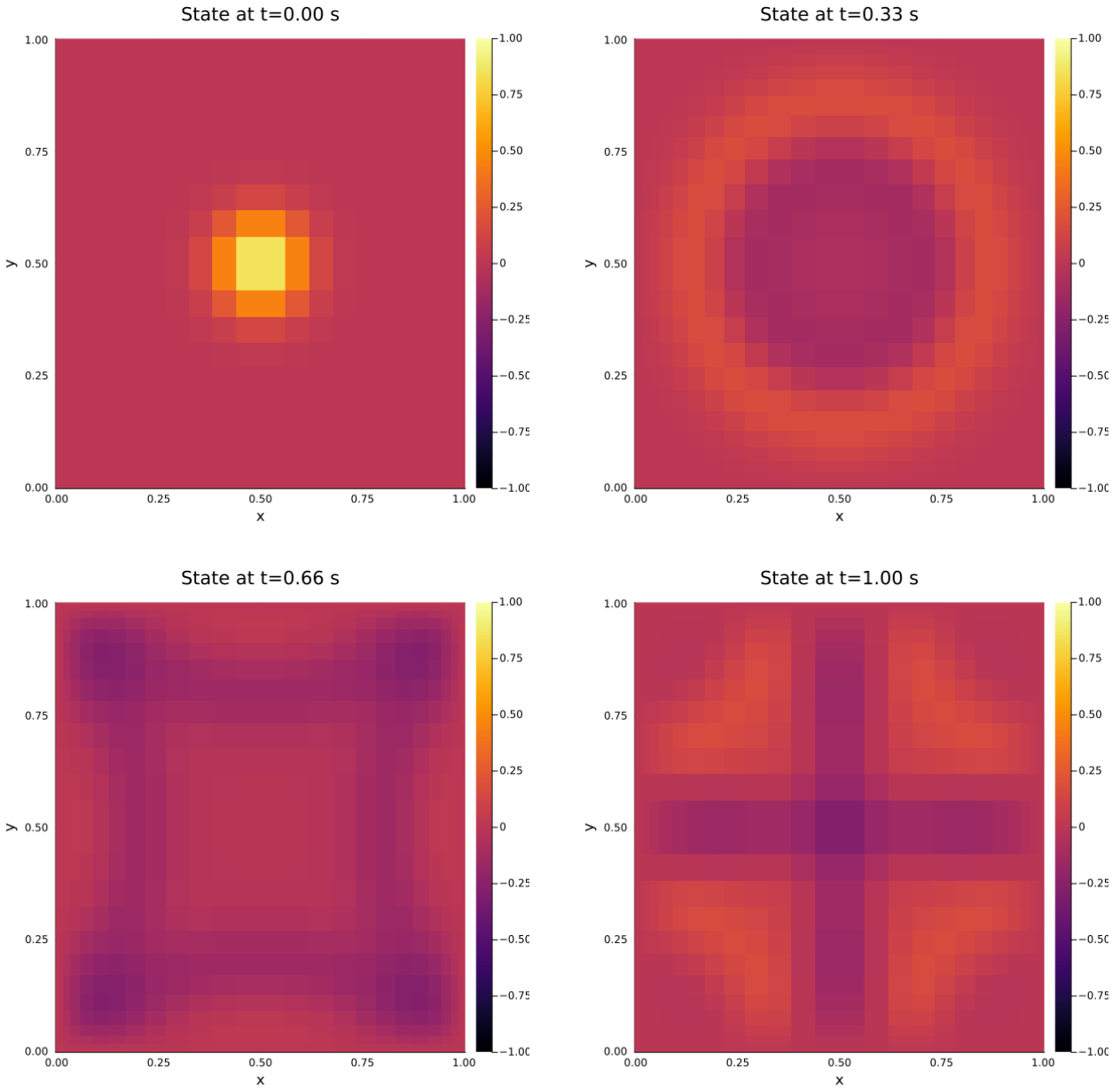

Figure 8.5: 2D Wave Equation Evolution. The temporal sequence shows: (top left) initial Gaussian pulse centered at domain center, (top right) radial wave expansion at t=0.33s, (bottom left) complex interference patterns at t=0.66s, and (bottom right) final standing wave configuration with multiple nodes at t=1.0s. The discrete solution is computed on a 26×26 Chebyshev node grid.

Physical 2D Wave Evolution (MxM) - Interpolated Representation

The 2D Chebyshev node distribution enables high-quality tensor-product Lagrange interpolation to reconstruct the continuous wave field. The solution is interpolated from 26×26 Chebyshev nodes to a 100×100 equispaced grid:

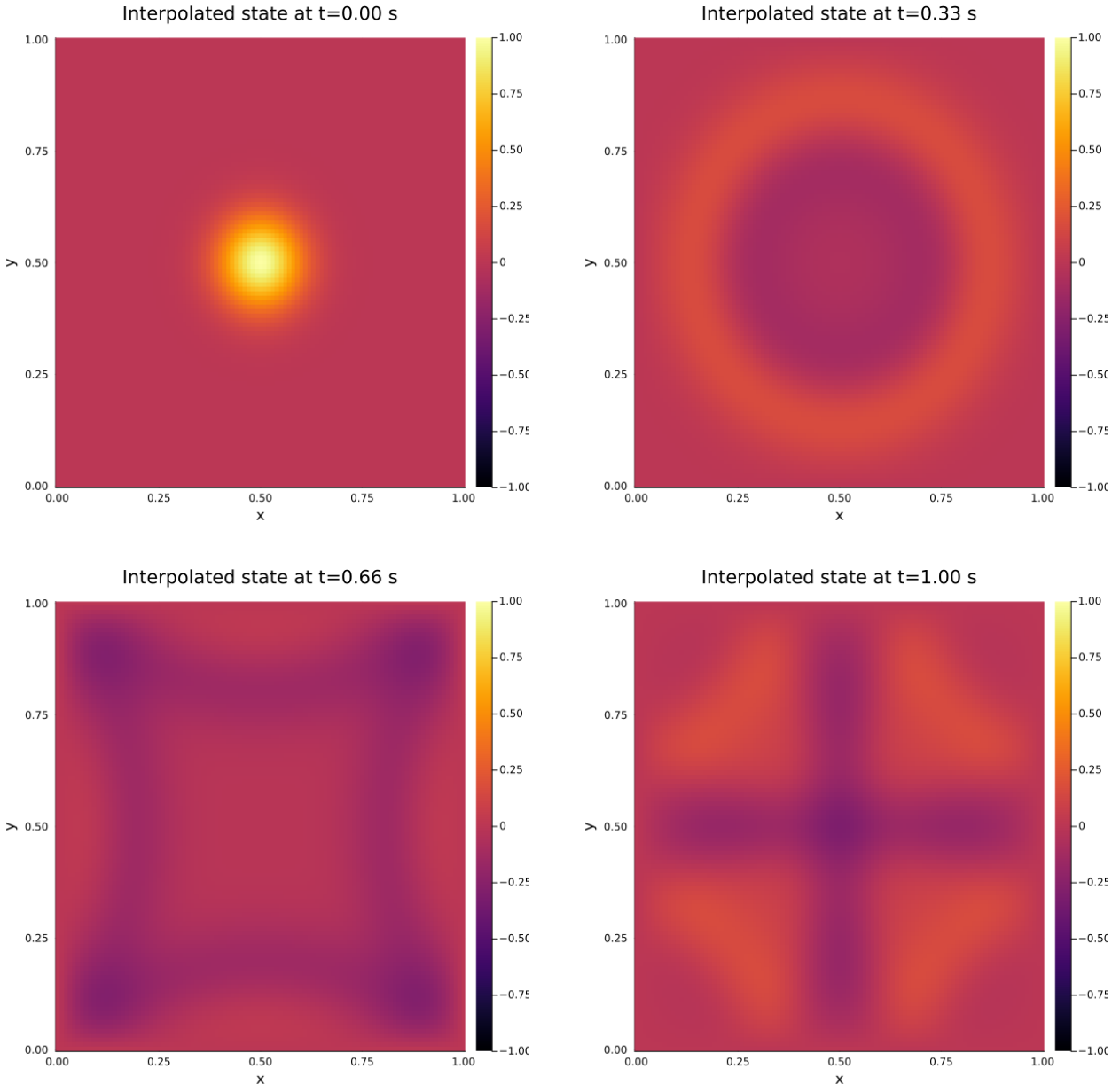

Figure 8.6: 2D Wave Equation Evolution. (Fine Grid) The smooth contours demonstrate excellent interpolation accuracy from the optimal 26×26 Chebyshev node distribution to a fine 100×100 equispaced visualization grid. The tensor-product interpolation preserves wave symmetry and captures fine-scale interference patterns throughout the temporal evolution.

8.3 Wave Equation 2D (Looped Matrix–Vector)

8.3.1 Mathematical model

The mathematical formulation is identical to Section 8.2, solving the 2D wave equation:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \qquad (x, y) \in [0, L_x] \times [0, L_y],\ t \geq 0 \tag{8.23}$$

with identical initial and boundary conditions.

The key difference lies in the computational approach for evaluating the Laplacian operator.

Alternative Laplacian computation (explicit loops)

Instead of using the efficient matrix-matrix formulation $\mathbf{D}_2^x \, \mathbf{U} + \mathbf{U} \, (\mathbf{D}_2^y)^\top$, this implementation computes the Laplacian through explicit loops that perform matrix-vector operations for each spatial direction:

$$\nabla^2 u(x_i, y_j) = \sum_{k=0}^{N_x} \mathbf{D}_2^x[i, k] \, u(x_k, y_j) + \sum_{\ell=0}^{N_y} \mathbf{D}_2^y[j, \ell] \, u(x_i, y_\ell) \tag{8.24}$$

This approach mimics applying 1D differentiation operations repeatedly:

- For each fixed y_j, apply $\mathbf{D}_2^x$ to the vector $\mathbf{u}[:, j]$ (x-direction derivative)
- For each fixed x_i, apply $\mathbf{D}_2^y$ to the vector $\mathbf{u}[i, :]$ (y-direction derivative)

Computational complexity

The looped approach performs $(N_y + 1)$ matrix-vector products for x-direction derivatives, each costing $2(N_x+1)^2$ FLOPs and (N_x+1) matrix-vector products for y-direction derivatives, each costing $2(N_y + 1)^2$ FLOPs.

This results in a total of $2(N_x + 1)^2(N_y + 1) + 2(N_x + 1)(N_y + 1)^2 \approx 4M^3$ FLOPs.

While theoretically equivalent to the matrix-matrix approach in FLOP count, the explicit loops result in poorer cache utilization, less efficient use of BLAS routines (`gemv` vs `gemm`), and additional loop overhead.

8.3.2 Code implementation

The looped matrix-vector implementation uses the same 4D tensor structure as the matrix-matrix version but computes the Laplacian through explicit nested loops:

```
function laplacian_2D(U::OffsetArray{T}, n::Int, D2x::OffsetArray{T},
                      D2y::OffsetArray{T}, Nx::Int, Ny::Int) where {T}
    L = OffsetArray(zeros(T, Nx+1, Ny+1), 0:Nx, 0:Ny)
    @inbounds @threads for j in 0:Ny
        for i in 0:Nx
            # Compute x-direction second derivative
            d2x = zero(T)
            for ii in 0:Nx
                d2x += D2x[i, ii] * U[ii, j, 0, n]
            end
            # Compute y-direction second derivative
            d2y = zero(T)
            for jj in 0:Ny
                d2y += D2y[j, jj] * U[i, jj, 0, n]
            end
            L[i, j] = d2x + d2y
        end
    end
    return L
end
```

Key implementation details:

- `@threads`: Parallelizes the outer loop over j (y-direction)
- Inner loops manually compute matrix-vector products using explicit summation
- No transpose operation needed for $\mathbf{D}_2^y$ due to explicit indexing structure

The time-stepping loop remains identical to the matrix-matrix version:

```
apply_dirichlet!(U, n, Nx, Ny) # enforce homogeneous BCs at t^n
L = laplacian_2D(U, n, D2x, D2y, Nx, Ny)

@views begin
    F[:, :, 0] .= U[:, :, 1, n] # RHS for u_t = v
    F[:, :, 1] .= c^2 .* L # RHS for v_t = c^2 (u_xx + u_yy)
    U[:, :, :, n+1] .= U[:, :, :, n] .+ dt .* F
end

apply_dirichlet!(U, n+1, Nx, Ny) # enforce BCs at t^(n+1)
```

The differentiation matrices are built identically to the matrix-matrix version:

```
D2x = OffsetArray(build_D2_matrix(xnodes, Nx+1, T), 0:Nx, 0:Nx)
D2y = OffsetArray(build_D2_matrix(ynodes, Ny+1, T), 0:Ny, 0:Ny)
```

8.3.3 Results

Laplacian Operator FP32 Performance

The 2D looped matrix-vector Laplacian operator consistently demonstrates the poorest performance among all tested approaches, highlighting fundamental algorithmic limitations.

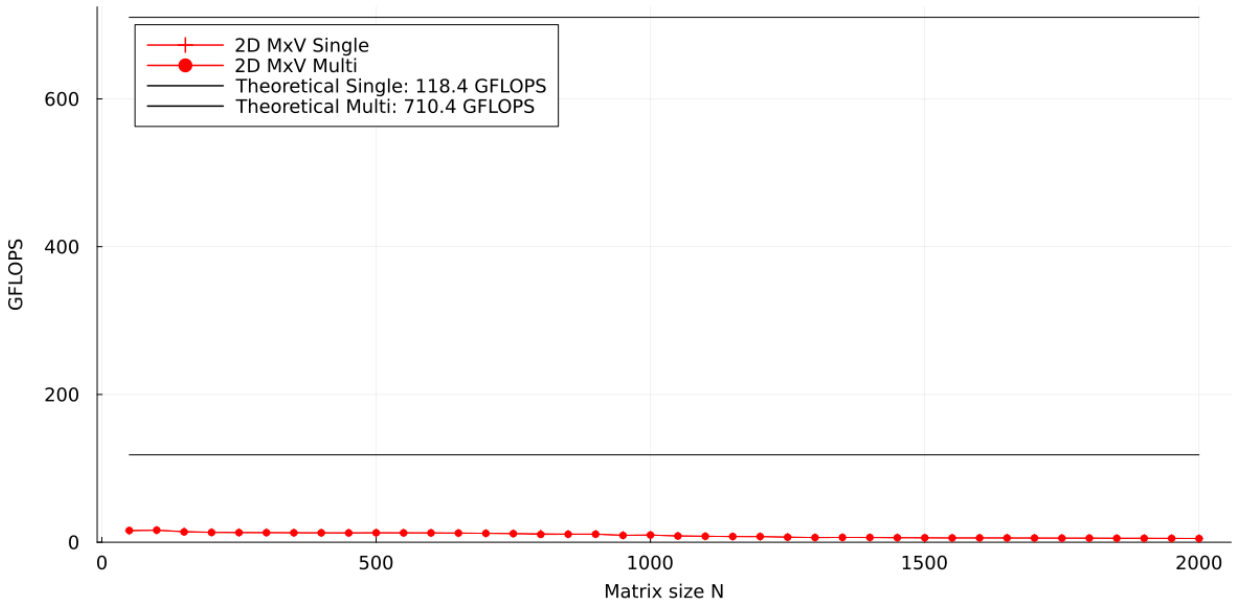

Figure 8.7: Float32 2D Laplacian (looped MxV): single vs. multi-threaded performance. Both curves remain consistently low across all matrix sizes, with performance trapped in the 4-16 GFLOPS range. Single-threaded performance shows a declining trend with increasing size, while multi-threaded performance exhibits minimal improvement and similar degradation patterns.

Consistently poor performance plateau: Both curves remain trapped well below theoretical and practical performance levels achieved by other approaches, with performance ranging from 4-16 GFLOPS across all tested sizes. Unlike other implementations that show improvement or stability with increasing matrix size, both curves exhibit a general downward trend. Performance drops from peak values around 15-16 GFLOPS at small sizes to 4-6 GFLOPS for larger matrices.

Threading ineffectiveness: The `@threads` directive provides negligible performance improvements, with multi-threaded results closely tracking single-threaded performance throughout the tested range. This indicates that the explicit loop structure fundamentally limits effective parallelization, preventing the runtime from achieving meaningful speedups despite attempting thread-level parallelism.

This implementation validates the critical importance of algorithmic choice and library optimization, demonstrating that theoretical FLOP equivalence does not guarantee computational efficiency when implementation approaches differ fundamentally in their utilization of modern CPU architectures and optimized libraries.

2D Wave Equation (MxV Loop) FP64 Performance

We evaluated the complete 2D wave equation solver across a range of grid sizes using **Float64 precision** and 1.0 second simulation time.

Each time step involves matrix-vector multiplication operations, plus additional operations for temporal integration and boundary condition enforcement. The total floating-point operations per time step can be approximated as $2 \times 2 \times N^3$ (dominated by the MxV operations):

$$\text{Total Operations} = \text{Time Steps} \times \text{Operations per Step} \approx N_t \times 2 \times 2 \times N^3 \tag{8.25}$$

We used an adaptive time step of $dt = 10^{-4} \times \frac{1}{N_x}$ seconds for stability control. This formulation ensures that finer grids use proportionally smaller time steps.

Grid Size	**Time (s)**	**Total Operations**	**GFLOPS**
26 nodes	4.84	1.83×10^{10}	3.78
51 nodes	30.64	2.71×10^{11}	8.83
76 nodes	119.77	1.33×10^{12}	11.14
101 nodes	350.85	4.16×10^{12}	11.86
126 nodes	943.62	1.01×10^{13}	10.68

The performance progression demonstrates a different scaling behavior compared to the MxM implementation across all grid sizes. For small grids (26-51 nodes), we observe steady efficiency improvements as computational overhead is amortized. The medium to large grids (76-126 nodes) show peak performance around 10-11 GFLOPS, with a slight decline for the largest grid size.

This demonstrates the computational characteristics of 2D matrix-vector operations compared to matrix-matrix operations. While the total number of operations remains the same, the MxV implementation shows different performance characteristics due to memory access patterns and computational structure. The peak performance of approximately 11 GFLOPS is significantly lower than the 70 GFLOPS achieved in the MxM benchmark.

The performance plateau and slight decline for larger grids suggests that memory bandwidth limitations become more prominent in MxV operations, where the lower arithmetic intensity compared to MxM operations results in reduced computational throughput.

We observe that the MxV performance is substantially lower than what was achieved in the MxM benchmark, highlighting the importance of algorithmic choice and computational structure in high-performance computing applications, even when the total operation count remains constant.

Physical 2D Wave Evolution (MxV) - Original Nodes Representation

We see exactly the same physical wave evolution as in the matrix-matrix version.

The 2D solver captures complete wave physics on Chebyshev node grids. Using 25×25 nodes with 26-node stencils demonstrates radial wave propagation and boundary interactions:

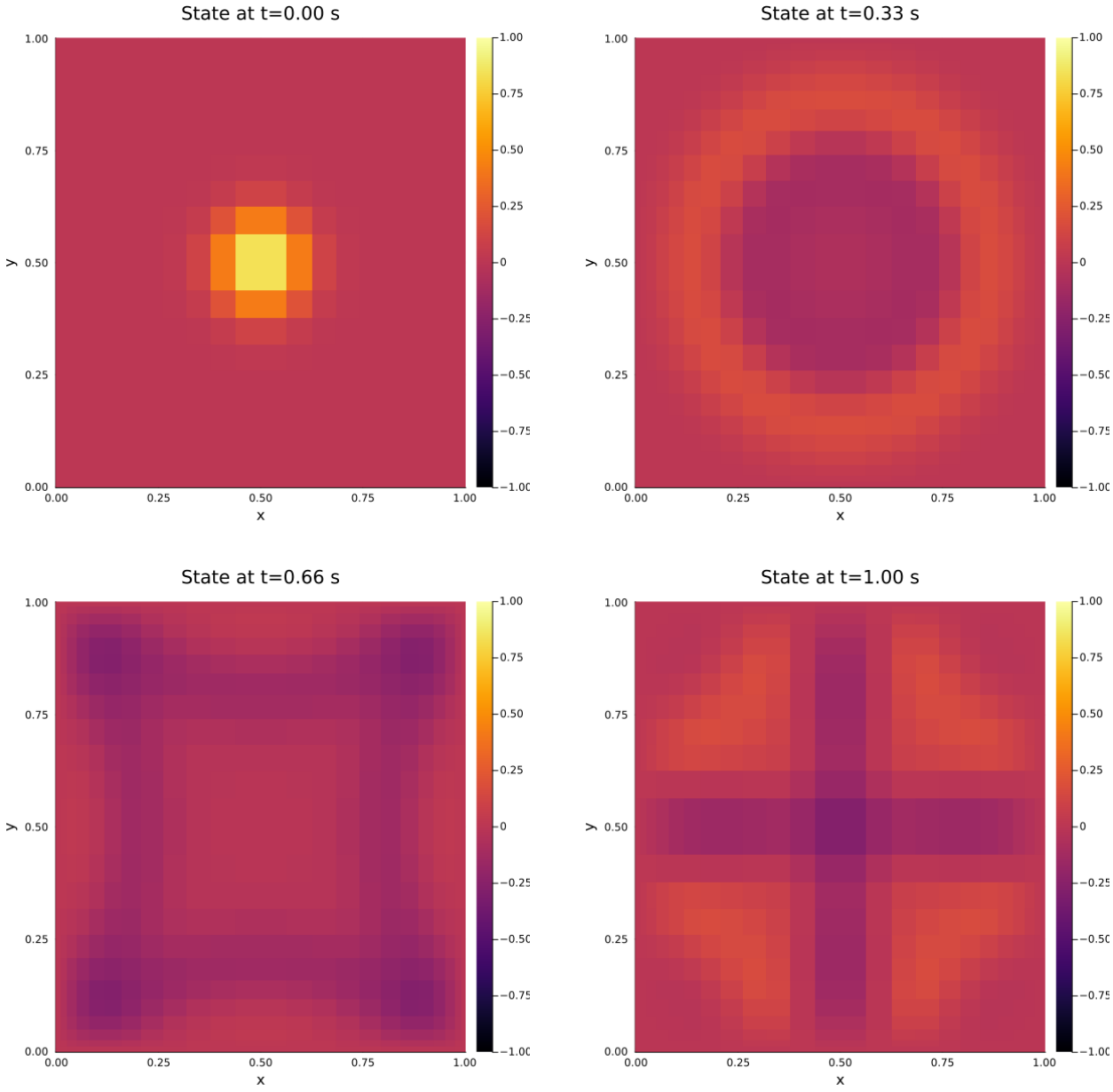

Figure 8.8: 2D Wave Equation Evolution. The temporal sequence shows: (top left) initial Gaussian pulse centered at domain center, (top right) radial wave expansion at t=0.33s, (bottom left) complex interference patterns at t=0.66s, and (bottom right) final standing wave configuration with multiple nodes at t=1.0s. The discrete solution is computed on a 26×26 Chebyshev node grid.

Physical 2D Wave Evolution (MxV) - Interpolated Representation

The 2D Chebyshev node distribution enables high-quality tensor-product Lagrange interpolation to reconstruct the continuous wave field. The solution is interpolated from 26×26 Chebyshev nodes to a 100×100 equispaced grid:

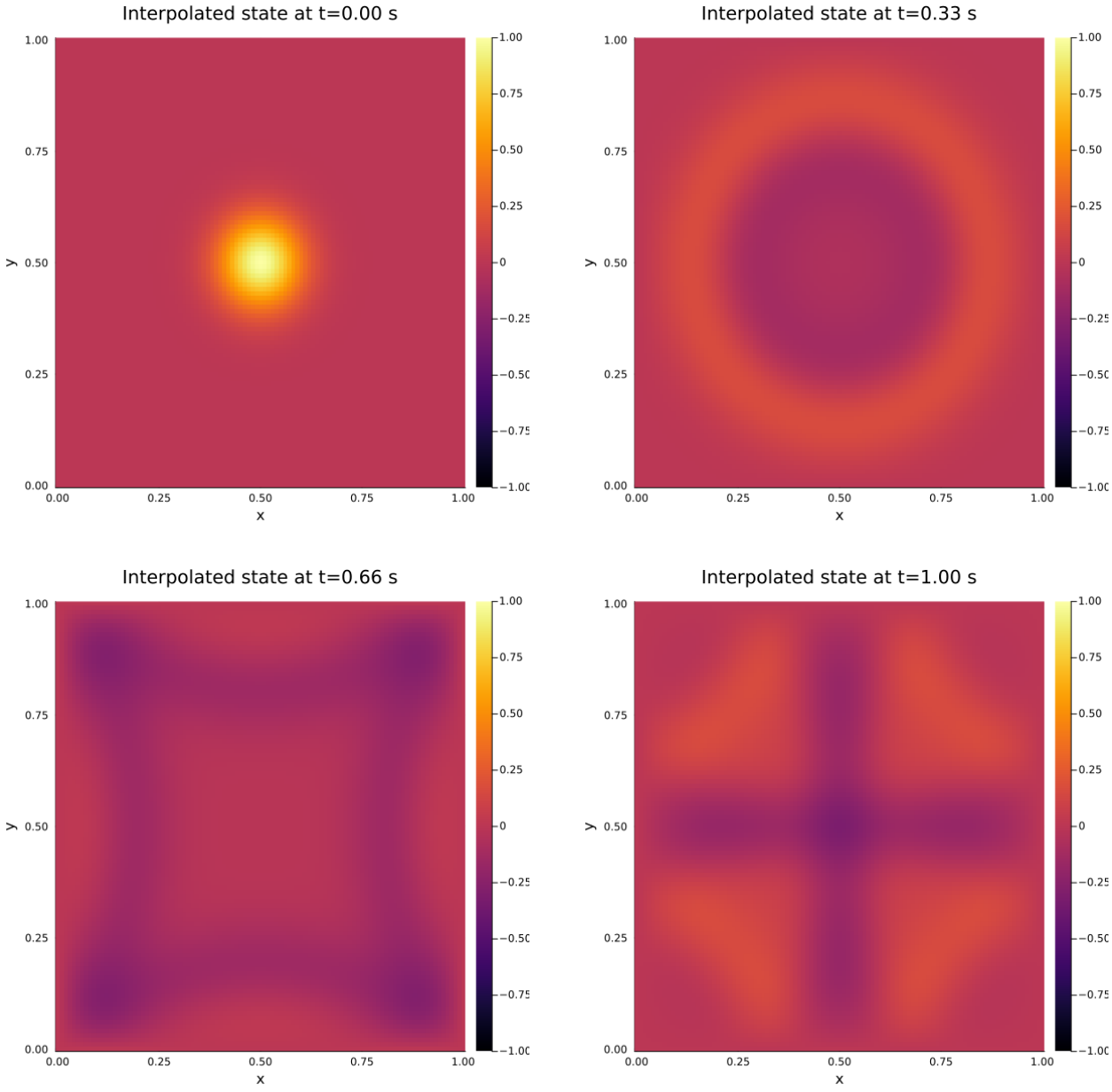

Figure 8.9: 2D Wave Equation Evolution (Fine Grid). The smooth contours demonstrate excellent interpolation accuracy from the optimal 26×26 Chebyshev node distribution to a fine 100×100 equispaced visualization grid. The tensor-product interpolation preserves wave symmetry and captures fine-scale interference patterns throughout the temporal evolution.

8.4 Wave Equation 2D in NumericalHub (Fortran)

2D Wave Equation NumericalHub (Fortran) Performance

We evaluated the NumericalHub Fortran implementation of the complete 2D wave equation solver across a range of grid sizes using **Float64 precision** and 1.0 second simulation time. This serves as a benchmark comparison against our custom Julia implementations to assess the relative performance achieved by our developed codes.

The NumericalHub library uses established numerical methods with optimized Fortran implementations. Each time step involves the same fundamental operations as our implementations: spatial differentiation, temporal integration, and boundary condition enforcement. The total floating-point operations per time step follow the same approximation as $2 \times 2 \times N^3$:

$$\text{Total Operations} = \text{Time Steps} \times \text{Operations per Step} \approx N_t \times 2 \times 2 \times N^3 \tag{8.26}$$

The same adaptive time step formulation of $dt = 10^{-4} \times \frac{1}{N_x}$ seconds is used for stability control, ensuring consistent comparison conditions with our Julia implementations.

Grid Size	Time (s)	Total Operations	GFLOPS
26 nodes	12.06	1.56×10^{10}	1.29
51 nodes	228.03	2.50×10^{11}	1.10
76 nodes	792.80	1.27×10^{12}	1.60
101 nodes	1812.31	4.16×10^{12}	2.30

The NumericalHub Fortran implementation demonstrates consistent but modest performance across all grid sizes. Notably, this performance is significantly lower than both our Julia MxM implementation (which achieved up to 70 GFLOPS) and our Julia MxV implementation (which achieved up to 11 GFLOPS).

The comparison reveals several important insights about computational performance. While NumericalHub provides a robust, well-tested library solution, the performance characteristics suggest either more conservative algorithmic choices, additional computational overhead from generalized library functions, or different optimization strategies compared to our specialized implementations.

This performance comparison validates the effectiveness of our custom Julia implementations, demonstrating that targeted optimization and algorithmic design can achieve substantial performance improvements over general-purpose library solutions.

The consistent wave physics results across all implementations confirm that our optimized codes maintain numerical accuracy while delivering superior computational throughput.

Physical 2D Wave Evolution (Fortran) Representation

The 2D wave equation solver demonstrates complete wave physics on a 26×26 Chebyshev node grid. The implementation captures radial wave propagation from a central source and subsequent boundary interactions across the computational domain.

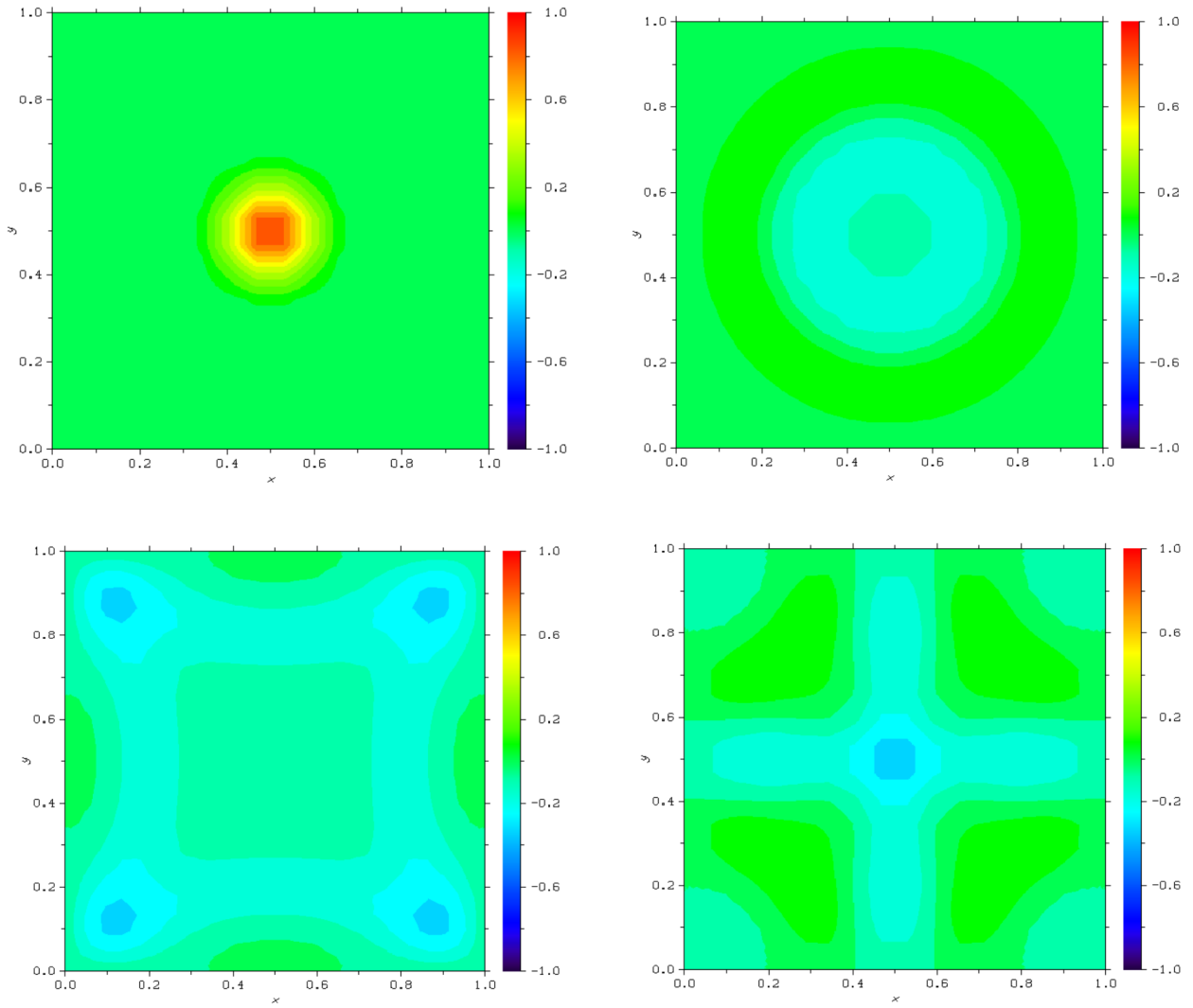

Figure 8.10: 2D Wave Equation Evolution. Numerical solution of the 2D wave equation implemented in NumericalHub and visualized using the plot_contour function.

This visualization represents the same numerical approach implemented in the Julia versions, but executed through the NumericalHub framework with Dislin-based contour plotting for scientific visualization.

Chapter 9

Conclusions

9.1 Key Findings and Contributions

This thesis has conducted a comprehensive investigation into the computational performance characteristics of matrix-vector (MxV) and matrix-matrix (MxM) operations within the context of numerical partial differential equation solvers, with particular emphasis on wave equation implementations across CPU and GPU architectures.

Fundamental Performance Insights

The central hypothesis of this work (that matrix-matrix formulations achieve superior computational efficiency due to higher arithmetic intensity) has been rigorously validated through both synthetic benchmarks and real-world PDE applications. The theoretical analysis presented in Chapter 2 established that MxM operations achieve operational intensity scaling as $\mathcal{O}(N)$, while MxV operations saturate at approximately 2 FLOPS per data element. This fundamental difference manifests consistently across all architectural platforms investigated.

On CPU architectures, our benchmarks demonstrate that MxM operations achieve sustained performance approaching theoretical peaks, while MxV operations remain constrained by memory bandwidth limitations, typically plateauing regardless of problem size.

The GPU analysis reveals even more pronounced performance differentials, with MxM operations achieving up to a 30x speedup for large matrices, while MxV performance remains severely bandwidth-bound at substantially lower levels, but still benefits from the higher memory bandwidth of the GPU architecture, allowing up to 10x speedups.

Practical Implications for PDE Solvers

The transition from 1D to 2D wave equation implementations provides a compelling demonstration of these performance principles in practice. The 1D solver, inherently dominated by matrix-vector operations for spatial differentiation, exhibits characteristic memory-bound behavior with modest GFLOPS rates that plateau as grid resolution increases. In contrast, the 2D matrix-matrix implementation leverages optimized BLAS routines to achieve significant performance improvements, validating the theoretical predictions in a realistic computational physics context.

Particularly noteworthy is the comparison with the reference Fortran implementation from NumericalHub. Despite Fortran's reputation for computational efficiency, our Julia matrix-matrix solver consistently outperforms the reference implementation, demonstrating that algorithmic formulation choices can overcome language-level performance differences.

Architectural Performance Characteristics

The cross-platform analysis reveals distinct performance profiles across architectures.

CPU implementations benefit significantly from cache-friendly data access patterns in MxM operations, while suffering from cache misses effects in MxV kernels.

GPU architectures amplify these trends: the massive parallelism inherent in GPUs can only be effectively utilized when sufficient arithmetic intensity allows compute units to remain occupied while memory operations proceed in parallel.

The investigation of memory hierarchy effects, including detailed analysis of cache set conflicts and bandwidth limitations, provides valuable insights for performance optimization. The identification of specific grid sizes that trigger cache thrashing (e.g., multiples of 128 for FP32 operations) offers practical guidance for computational grid design.

Global Interpolation as a Unifying Framework

The development of the global interpolation framework in Chapters 7 and 8 represents a significant methodological contribution. By pre-computing differentiation matrices using Lagrange basis polynomials, the framework transforms the traditional node-by-node stencil applications into highly optimized matrix operations. This approach not only improves computational efficiency but also provides a flexible foundation for achieving arbitrary-order accuracy and handling non-uniform grids through Chebyshev node distributions.

The iterative Lagrange polynomial implementation enables efficient computation of high-order differentiation matrices while maintaining numerical stability. The resulting solver architecture clearly separates the physics (encoded in the state tensor $\mathbf{U}$ and physical vector $\mathbf{F}$) from the numerical implementation, facilitating code modularity and extensibility.

9.2 Limitations and Scalability Challenges

The primary constraint lies in the explicit time integration scheme employed throughout this work. The CFL stability condition necessitates time step reductions proportional to spatial grid refinement, resulting in computational cost that scales as $\mathcal{O}(N^4)$ for 2D problems when both spatial and temporal resolutions are considered.

This scaling limitation becomes prohibitive for large-scale simulations (N $>$ 500), where the number of required time steps grows exponentially with grid refinement. Consequently, while MxM operations provide significant per-timestep performance improvements, the overall computational advantage can be offset by stability-imposed time step restrictions in explicit schemes. Additionally, the performance benefits of MxM formulations require sufficiently large matrix dimensions to amortize computational overhead.

9.3 Future Work

Advanced Temporal Integration Schemes

The most significant limitation of our current approach stems from the use of the explicit Euler method for temporal integration. This simple first-order scheme, while straightforward to implement and understand, imposes severe stability restrictions through the CFL (Courant-Friedrichs-Lewy) condition. When the spatial grid becomes finer (larger N), the time step Δt must be reduced proportionally to maintain numerical stability.

The path forward requires investigation of temporal integration schemes with significantly larger stability regions. Unlike explicit methods that restrict time steps based on the finest spatial scale, more advanced schemes could allow much larger time steps while maintaining numerical stability.

Full GPU Implementation

The natural extension of this work involves full GPU implementation of the wave equation solvers using CUDA.jl and cuBLAS. Preliminary analysis suggests that GPU memory bandwidth (448 GB/s on RTX 3070) could provide substantial improvements for memory-bound operations, while the massive parallelism would dramatically accelerate compute-bound MxM kernels.

Appendix

A.1 Julia programming language initial setup

This study will be implemented using the Julia programming language. This section provides an overview of the initial setup required to start programming in Julia.

The Julia programming language is a high-level, high-performance language designed for numerical and scientific computing. It is known for its speed and ease of use, making it an ideal choice for developing computational applications. This section provides an overview of the initial setup required to start programming in Julia.

A.1.1 Installation

The first step in setting up Julia is to download and install the latest version of the language from the official website (https://julialang.org/downloads/). The installation process is straightforward and involves running the installer and following the on-screen instructions.

This will enable you to run Julia code from the command line but to take full advantage, we will use Visual Studio Code and the Julia extension for Visual Studio Code.

A.1.2 Package Management in Julia

Managing packages efficiently is essential when working on different Julia projects. Julia provides a built-in package manager, `Pkg`, which allows users to install, update, and manage dependencies. However, to ensure reproducibility and avoid conflicts between projects, it is best to use project-specific environments.

Each Julia project organizes its dependencies using two key files:

- **`Project.toml`**: Specifies the direct dependencies of the project.
- **`Manifest.toml`**: Stores the full list of installed packages, including exact versions.

Working with project environments ensures that each project has its own set of dependencies, preventing conflicts between different projects. You will just need the **`Project.toml`** file to recreate the environment.

Automating Package Management

To automate package management tasks such as activating environments, resolving dependencies, and installing required packages, we can use the `setup.jl` script.

Listing A.1: setup.jl

```
# File: 1_annexes/setup.jl

using Pkg
project_path = pwd()

println("Activating the project environment...")
Pkg.activate(project_path)

println("Checking for inconsistencies in the project...")
try
    Pkg.resolve()
    println("Inconsistencies resolved.")
catch e
    println("Error while resolving dependencies: ", e)
    println("Trying to fix dependencies...")
    try
        Pkg.instantiate()
        println("Dependencies fixed.")
    catch e_inner
        println("Error while fixing dependencies: ", e_inner)
        println("Consider manually inspecting Project.toml and Manifest.toml.")
        return
    end
end

println("Installing the packages according to [deps]...")
try
    Pkg.instantiate()
    println("Packages installed successfully.")
catch e
    println("Error while installing packages: ", e)
    println("Ensure that Project.toml is correctly configured.")
    return
end

println("Updating the packages according to [compat]...")
try
    Pkg.resolve()
    println("Packages updated successfully.")
catch e
    println("Error while updating packages: ", e)
end

println("Versions of the packages installed:")
Pkg.status()

println("Project environment ready.")
```

A.2 System Information and Hardware Overview

It's important to know the system and hardware specifications to know the upper limits of the hardware so that we can compare the results obtained in the benchmarks.

The following code snippets were extracted from the `system_info.jl` script, which provides information about the system and hardware specifications. This script is used to gather information about the system and hardware specifications, such as the operating system, CPU, and GPU details [13].

Function `cpu_info(true)` returns the CPU max GFLOPS and function `gpu_info(true)` return the GPU max GFLOPS. Using the `false` argument leds to print the hardware information on the screen.

Listing A.2: CPU function system_info.jl

```
function cpu_info(return_max_gflops=false)
    N_cores = num_physical_cores()
    N_threads = num_virtual_cores()
    cpuid = cpuinfo()
    string_cpuid = string(cpuid)
    cpu_frequency = 3.7 # GHz (Adjust as needed)
    AVX_value = get_avx_value(string_cpuid)
    M_ops = 4 # 2 FMA, 2 operations each (multiply-add) (AVX2)

    if return_max_gflops
        if AVX_value > 0
            Theoretical_time = 1e9 / (cpu_frequency * 1e9 * AVX_value * M_ops * N_cores)
            max_gflops = 1 / Theoretical_time
            return max_gflops
        else
            return 0
        end
    else
        println("CPU Information")
        println(" ")
        println("CPU Cores: ", N_cores)
        println("CPU Threads: ", N_threads)
        println("CPU Frequency: ", cpu_frequency, " GHz (GHz adjusted manually)")
        println("AVX support: ", occursin("256 bit", string_cpuid))
        println("AVX-512 support: ", occursin("512 bit", string_cpuid))
        println("AVX Value: ", AVX_value)
        println("M_ops: ", M_ops, " (AVX2: 2 FMA, 2 operations each (multiply-add))")
        if AVX_value > 0
            Theoretical_time = 1e9 / (cpu_frequency * 1e9 * AVX_value * M_ops * N_cores)
            max_gflops = 1 / Theoretical_time
            println("CPU Theoretical max GFLOPS: ", round(max_gflops, digits=2), " GFLOPS (
                using base frequency)")
        else
            println("AVX not supported. Cannot compute theoretical GFLOPS.")
        end
    end
end
```

Listing A.3: GPU function system_info.jl

```
function gpu_info(return_max_gflops=false)
    device = CUDA.device()
    capability = CUDA.capability(device)
    sm_count = CUDA.attribute(device, CUDA.DEVICE_ATTRIBUTE_MULTIPROCESSOR_COUNT)
    cores_per_sm = cuda_cores_per_sm(capability)
    total_cuda_cores = sm_count * cores_per_sm
    clock_rate = CUDA.attribute(device, CUDA.DEVICE_ATTRIBUTE_CLOCK_RATE) / 1e6

    max_gflops = 2 * clock_rate * total_cuda_cores

    memory_clock_khz = CUDA.attribute(device, CUDA.DEVICE_ATTRIBUTE_MEMORY_CLOCK_RATE)
    bus_width_bits = CUDA.attribute(device, CUDA.DEVICE_ATTRIBUTE_GLOBAL_MEMORY_BUS_WIDTH)
    memory_clock_ghz = (memory_clock_khz / 1e6) * 2
    memory_bandwidth = memory_clock_ghz * (bus_width_bits / 8) # GB/s (8 bits = 1 byte)

    if return_max_gflops
        return max_gflops
    else
        println("GPU Information")
        println(" ")
        println("GPU Name: ", CUDA.name(device))
        println("GPU Compute Capability: ", capability.major, ".", capability.minor, " (",
            get_GPU_architecture(capability), ")")
        println("GPU Memory: ", round(CUDA.totalmem(device) / 1e9, digits=2), " GB")
        println("GPU Memory: ", round(CUDA.totalmem(device) / 2^30, digits=2), " GiB")
        println("GPU Memory Bandwidth: ", round(memory_bandwidth, digits=2), " GB/s")
        println("GPU Streaming Multiprocessor (SM) Count: ", sm_count)
        println("CUDA Cores per SM: ", cores_per_sm)
        println("Total CUDA Cores: ", total_cuda_cores)
        println("GPU Clock Rate: ", clock_rate, " GHz")
        println("GPU Theoretical max GFLOPS: ", round(max_gflops, digits=2), " GFLOPS")
    end
end
```

It's important to modify the `system_info.jl` script to include any additional hardware information you want to display and customize parameters as the `cpu_frequency` or `M_ops`.

Abbreviations and acronyms

UPM	Universidad Politécnica de Madrid
ETSIAE	Escuela Técnica Superior de Ingeniería Aeronáutica y del Espacio
CPU	Central Processing Unit
GPU	Graphics Processing Unit
RAM	Random Access Memory
BLAS	Basic Linear Algebra Subprograms
SMT	Simultaneous Multi-Threading
SIMD	Single Instruction Multiple Data
AVX	Advanced Vector Extensions
SRAM	Static Random Access Memory
DRAM	Dynamic Random Access Memory
FMA	Fused Multiply-Add
BLAS	Basic Linear Algebra Subprograms
CAS	Column Access Strobe
DDR	Double Data Rate
MKL	Math Kernel Library
FLOPS	Floating Point Operations Per Second
MxM	Matrix-Matrix Multiplication
GEMM	General Matrix to Matrix Multiplication
MxV	Matrix-Vector Multiplication
GEMV	General Matrix to Vector Multiplication
VxV	Vector-Vector Multiplication
FP32	32-bit Floating Point
FP64	64-bit Floating Point
HPC	High Performance Computing
PCIe	Peripheral Component Interconnect Express
GT/s	Gigatransfers per Second
BW	Bandwidth

CUDA Compute Unified Device Architecture

CUs Compute Units

SMs Streaming Multiprocessors

GDDR Graphics Double Data Rate

SIMT Single Instruction Multiple Threads

Bibliography

[1] S. Boehm, *Optimize a CUDA matmul kernel*, Article on CUDA optimization; published December 2022, 2022. [Online]. Available: https://siboehm.com/articles/22/CUDA-MMM.

[2] A. Casas, *CUDA tutorial: Blocks and grids*, 2024. [Online]. Available: https://blog.damavis.com/en/cuda-tutorial-blocks-and-grids/.

[3] ComputationStructures, *The memory hierarchy*, Educational resource on computer architecture, 2024. [Online]. Available: https://computationstructures.org/lectures/caches/caches.html.

[4] Cornell Center for Advanced Computing, *Understanding GPU memory types*, Educational guide on GPU memory. [Online]. Available: https://cvw.cac.cornell.edu/gpu-architecture/gpu-memory/memory_types.

[5] G. Dalle, J. Smit, and A. Hill, *Optimizing your code*, 2025. [Online]. Available: https://modernjuliaworkflows.org/optimizing/.

[6] S. Harding, *What Is CAS Latency in RAM?*, 2019. [Online]. Available: https://www.tomshardware.com/reviews/cas-latency-ram-cl-timings-glossary-definition,6011.html.

[7] J. A. Hernández Ramos and M. A. Zamecnik Barros, *Interpolación polinómica de alto orden. Métodos espectrales. Aplicación a problemas de contorno y de condiciones iniciales.* Independently Published, 2020, Spanish edition; self-published book on numerical methods; published January 11, 2020.

[8] Intel, *Fused multiply–add (fma) instructions*, 2021. [Online]. Available: https://www.intel.com/content/www/us/en/docs/cpp-compiler/developer-guide-reference/2021-8/fma-qfma.html.

[9] Intel, *Intel advanced vector extensions 2 (intel avx2)*, Datasheet, Volume 1 of 2, 2022. [Online]. Available: https://edc.intel.com/content/www/xl/es/design/ipla/software-development-platforms/client/platforms/alder-lake-desktop/12th-generation-intel-core-processors-datasheet-volume-1-of-2/004/intel-advanced-vector-extensions-2-intel-avx2/.

[10] jahrWork, *GPU-parallel github repository*. [Online]. Available: https://github.com/jahrWork/GPU-Parallel.

[11] JuliaGPU, *GemmKernels.jl*, Flexible and performant GEMM kernels in Julia, 2021. [Online]. Available: https://github.com/JuliaGPU/GemmKernels.jl/tree/master.

[12] JuliaLinearAlgebra, *MKL.jl library for Julia*, Intel MKL linear algebra backend for Julia, 2025. [Online]. Available: https://github.com/JuliaLinearAlgebra/MKL.jl.

[13] NVIDIA, *CUDA compute capability*, CUDA C++ Programming Guide, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#features-and-technical-specifications.

[14] ScienceDirect, *L1 cache*, 2023. [Online]. Available: https://www.sciencedirect.com/topics/computer-science/l1-cache.

[15] Stack Exchange Community, *Why does expressing calculations as matrix multiplications make them faster?*, 2018. [Online]. Available: https://softwareengineering.stackexchange.com/questions/312445/why-does-expressing-calculations-as-matrix-multiplications-make-them-faster.

[16] Stack Overflow Community, *Is memory operation for L2 cache significantly faster than global memory for NVIDIA GPU?*, 2021. [Online]. Available: https://stackoverflow.com/questions/66921433/is-memory-operation-for-l2-cache-significantly-faster-than-global-memory-for-nvi.

[17] Stack Overflow Community, *How does the CPU cache affect the performance?*, 2022. [Online]. Available: https://stackoverflow.com/questions/71818324/how-does-the-cpu-cache-affect-the-performance-of-a-c-program.

[18] TechPowerUp, *AMD Ryzen 5 5600X specs*, 2020. [Online]. Available: https://www.techpowerup.com/cpu-specs/ryzen-5-5600x.c2365.

[19] TechPowerUp, *NVIDIA GeForce RTX 3070 specs*, 2020. [Online]. Available: https://www.techpowerup.com/gpu-specs/geforce-rtx-3070.c3674.

[20] T. Yang, *Optimizing cache performance in matrix multiplication*, Course slides, 2017. [Online]. Available: https://sites.cs.ucsb.edu/~tyang/class/240a17/slides/Cache3.pdf.

[21] YouTube, *Fundamentals of GPU architecture*, YouTube playlist, 2019. [Online]. Available: https://www.youtube.com/playlist?list=PLxNPSjHT5qvscDTMaIAY9boOOXAJAS7y4.